

JavaScript: Beginner to Advanced

Learn JavaScript step by step by understanding it, coding it, and building with it.

script.js

```
console.log("Hello, JavaScript!");  
  
Hello, JavaScript!
```

Contents

Part 1: JavaScript fundamentals

- 1 What is JavaScript?**
What JavaScript is, where it runs, and your first program
- 2 Variables**
let, const, var, and how to name things well
- 3 Data Types**
Strings, numbers, booleans, and the other kinds of values
- 4 Operators**
Math, comparing values, combining conditions, and shortcuts
- 5 Conditions**
Making decisions with if, else, switch, and truthy values

6**Loops**

Repeating work with for, while, do...while, break, continue, for...of, and for...in

7**Functions**

Reusable blocks of code: parameters, return values, scope, arrow functions, and callbacks

P1**Mini Bank Account**

A bigger project that combines variables, conditions, loops, and functions

Part 2: Working with data

8**Arrays**

Lists of values, and the methods that add, remove, search, transform, and sort them

9**Objects**

Storing related information together with properties and methods

10**Strings**

Working with text: length, case, searching, slicing, replacing, and splitting

11**Numbers and Math**

Converting text to numbers, NaN, rounding, the Math object, and random numbers

P2**Library Book Manager**

A bigger project that combines arrays, objects, strings, and numbers

Part 3: JavaScript in the browser

12**The DOM**

Selecting, changing, creating, and removing the elements of a web page

13**Events**

Making pages react to clicks, typing, forms, the keyboard, and the mouse

P3

To-Do List App

A bigger project: a complete to-do list app with add, complete, delete, and filters

Before You Begin

Welcome! This book takes you from your very first line of JavaScript to real programs that run in the browser. You do not need any programming experience. If you know a little HTML and CSS, you are ready to start.

How every idea is taught

Each new idea in this book is explained in the same eight steps, always in the same order. Once you know the labels, you always know where to look.

Label	What you will find
What is it?	The idea in very simple words
Why do we use it?	The real purpose: what problem it solves
Syntax	The basic pattern you type
Simple example	The smallest example that works
Real-world example	How it is used in something real
Common mistakes	The traps almost every beginner falls into
Key points	Three to six things to remember
Practice	Small exercises to try yourself

Between the ideas you will find boxes with extra help, and at the end of each chapter a **mini project** where you build something real. At the end of each part there is a bigger **project** that combines everything you learned.

💡 **Type the code yourself**

Do not copy and paste. Typing each example, even a short one, teaches your fingers and your brain at the same time. When something breaks, that is good news: you are learning what the error means.

📘 **About the output blocks**

The green **Output** blocks show what the program prints when you run it with Node.js. In the browser console the same result may look a little different (for example, arrays and objects can have small arrows and extra labels). The values are always the same.

What you need

- **A web browser.** Chrome, Edge, or Firefox all work. Press F12 to open the developer tools and click the **Console** tab.
- **A code editor.** Visual Studio Code is free and popular. Any plain text editor will also work.
- **A folder for your work.** Create one folder called `js-book` and keep everything from this book inside it.
- **Node.js (optional).** It lets you run JavaScript from a terminal. You can learn every idea in this book with only a browser.

How to get the most out of this book

1. Read a section, then close the book and try to explain it in your own words.
2. Run every example, then change something and predict what will happen.
3. Do the practice exercises before you look at the sample answers.
4. When an error appears, read the message slowly. It usually tells you the problem and the line number.

5. Take breaks. Understanding one small thing well is better than rushing through five.

The road ahead

- **Part 1: JavaScript fundamentals.** Variables, data types, operators, conditions, loops, and functions. These are the building blocks of every program.
- **Part 2: Working with data.** Arrays, objects, strings, and numbers. You will learn to store, search, and transform information.
- **Part 3: JavaScript in the browser.** The DOM and events. You will make real web pages that react to clicks, typing, and more.

At the end of each part, you will also build a bigger **project** that combines everything from that part. Mini projects at the end of chapters give you smaller practice along the way.

Ready? Turn to Chapter 1.

1

Chapter 1

What is JavaScript?

Every website you use does two jobs: it shows you things, and it reacts when you do something. Clicking a heart, typing in a search box, playing a game in the browser: all of that reacting is done by JavaScript. In this chapter you will meet the language and run your first program.

What you will learn

- What JavaScript is and why people use it
- How JavaScript, HTML, and CSS are different
- Where JavaScript runs: the browser and Node.js
- How the computer reads your code
- How to print messages with `console.log()`
- How to write and run your first program

1.1 What is JavaScript?

© What is it?

JavaScript (often shortened to **JS**) is a programming language. A programming language is a way of giving instructions to a computer, using words and symbols the computer can follow.

You already know two languages of the web. Think of a web page as a house:

Language	Its job	In a house
HTML	Structure: what is on the page	Walls, doors, and rooms
CSS	Style: how the page looks	Paint, curtains, and furniture
JavaScript	Behavior: what the page does	Electricity: lights switch on, the doorbell rings

HTML and CSS *describe* a page. JavaScript makes it *act*.

🔗 Why do we use it?

We use JavaScript to make pages interactive. It can show or hide a menu, check a form before it is sent, update the score in a game, or load new posts without refreshing the page. JavaScript also runs outside the browser, so people use it to build servers, mobile apps, and desktop apps too.

<> Syntax

JavaScript goes inside a `<script>` tag in your HTML file:

HTML

```
<script>
  // JavaScript code goes here
</script>
```

Later you will keep JavaScript in its own file and link it (you will see how in Section 1.4).

✍ Simple example

Here is a tiny page where HTML, CSS, and JavaScript each do their own job:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <style>
    h1 { color: royalblue; }    /* CSS: how it looks */
  </style>
</head>
<body>
  <h1>Hello!</h1>              <!-- HTML: what is on the page -->
  <script>
    // JavaScript: what it does
    document.querySelector("h1").textContent = "Hello from JavaScript!";
  </script>
</body>
</html>
```

The HTML says "Hello!", but JavaScript changes the text the moment the page loads. You see **Hello from JavaScript!** in blue.

Do not worry yet

You cannot read `document.querySelector(...)` yet, and that is fine. You will learn it in Chapter 12. For now, just see the idea: JavaScript can change a page.

Real-world example

Have you ever seen "Password too short" appear while you type a password? That is JavaScript checking your input. Here is the idea in a few lines (you will learn `if` properly in Chapter 5):

JavaScript

```
let password = "abc";

if (password.length < 8) {
  console.log("Password too short!");
}
```

Output

Password too short!

⚠ Common mistakes

- ✗ Thinking JavaScript and Java are the same language. They are different languages that only share part of a name.
- ✗ Writing JavaScript straight into the page without a `<script>` tag. The browser will just show it as text.
- ✗ Trying to memorize everything at once. Programmers look things up all the time. Understanding matters more than memorizing.

🔑 Key points

- JavaScript is a programming language that makes web pages interactive.
- HTML is structure, CSS is style, and JavaScript is behavior.
- In an HTML page, JavaScript lives inside a `<script>` tag.
- JavaScript also runs outside the browser, for example on servers.

🔧 Practice

- 1 Explain HTML, CSS, and JavaScript to a friend using the house idea (or a better idea of your own).

- 2 Write down three things on a website that must be done by JavaScript, not by HTML or CSS.
- 3 Save the example above as `hello.html`, open it in a browser, and change the heading color and the message.

⚡ Did you know?

JavaScript was created in 1995 by Brendan Eich in about ten days, for the Netscape browser. Today it runs on billions of devices.

1.2 Where Does JavaScript Run?

🕒 What is it?

JavaScript code needs a program that can read it and follow the instructions. That program is called a **JavaScript engine**. You can find an engine in two common places:

- **The browser** (Chrome, Edge, Firefox, Safari). Every browser has an engine inside. Chrome's engine is called V8.
- **Node.js**, a free program that lets you run JavaScript on your own computer, outside the browser. Node.js uses the same V8 engine as Chrome.

The language is the same in both places. What changes is the extra tools each place gives you:

Feature	Browser	Node.js
Extra tools	<code>document</code> , <code>window</code> , <code>alert()</code> : tools to control web pages	Tools to read and write files and build servers
Good for	Interactive web pages	Servers, command-line tools, automation
How you run code	Open a page, or type in the console	<code>node file.js</code> in a terminal

🔗 Why do we use it?

Knowing where your code runs tells you which tools you can use. Browser JavaScript can change a web page. Node.js JavaScript can create files or run a web server. Beginners usually start in the browser, because you see the result right away.

<> Syntax

There are two common ways to run a JavaScript file:

HTML

```
<!-- In the browser: link a JavaScript file from your HTML page -->
<script src="script.js"></script>
```

Terminal

```
# In Node.js: run a file from the terminal
node script.js
```

You can also type JavaScript directly into the **Console** of your browser: press F12, open the Console tab, type a line, and press Enter.

Simple example

Open your browser, press F12, click the **Console** tab, type the line below, and press Enter:

```
JavaScript
```

```
2 + 2
```

```
Output
```

```
4
```

The console understands JavaScript, so it works like a very smart calculator.

Real-world example

In the **browser**, JavaScript powers the "like" button, search suggestions, and online quizzes. In **Node.js**, JavaScript can power the server that checks your login and sends the right page, or a small tool that renames a thousand files in one go.

Common mistakes

- ✗ Using browser-only tools such as `document` or `alert()` in Node.js. You will get an error, because Node.js has no web page.
 - ✗ Forgetting to save the file before running it. The old version runs instead.
 - ✗ Thinking you must install Node.js first. You can learn everything in this book with just a browser.
-

Key points

- JavaScript runs inside an engine, either in a browser or in Node.js.
- The language is the same everywhere. The extra tools are different.

- The browser is great for pages and buttons. Node.js is great for servers and file tools.
- Press F12 in your browser to open the console.

☰ Practice

- 1 Open the console in your browser and type `10 * 5`. What do you see?
- 2 Which one needs a browser: changing a heading's color, or reading a file from your computer?
- 3 (Optional) Install Node.js from nodejs.org, then type `node -v` in a terminal to see its version number.

1.3 How JavaScript Works (A High-Level Look)

🕒 What is it?

When you run JavaScript, this is what happens:

Diagram

```
Your code    ->   JavaScript engine    ->   Computer follows the steps    ->
Result
(you write)      (reads and translates)  (does the work)
(text, page changes...)
```

The engine reads your code from **top to bottom, one statement at a time**. A **statement** is one instruction, like one step in a recipe. The engine translates each step into something the computer understands, and the computer does it.

JavaScript also does **one thing at a time**, like a cashier who serves one customer at a time.

💡 Why do we use it?

This top-to-bottom rule explains many beginner errors. If you use something before you create it, the engine has not met it yet, so it does not know what you mean.

<> Syntax

Syntax pattern

```
statement 1;  
statement 2;  
statement 3;
```

Each statement usually sits on its own line and ends with a semicolon `;`.

✍ Simple example

JavaScript

```
console.log("Step 1");  
console.log("Step 2");  
console.log("Step 3");
```

Output

```
Step 1  
Step 2  
Step 3
```

The steps run in exactly the order you wrote them.

🌐 Real-world example

Think of a recipe: crack the egg, whisk it, then cook it. Change the order and you get a mess. Code is the same. Here is what happens when you use a name the engine has not met yet:

JavaScript

```
console.log(playerName); // Error: playerName is not defined
```

The fix is to create it first, then use it:

JavaScript

```
let playerName = "Ava";  
console.log(playerName);
```

Output

Ava

⚠ Common mistakes

- ✗ Using a value before creating it. You will see an error like `playerName is not defined`.
- ✗ Expecting the engine to guess what you meant. It follows exactly what you wrote, nothing more.
- ✗ Ignoring error messages. Read them slowly: they usually tell you the line number and the problem.

🔑 Key points

- The engine reads code from top to bottom, one statement at a time.
- Order matters. Create things before you use them.
- Error messages are helpers, not enemies.
- JavaScript does one thing at a time.

☰ Practice

- 1 Predict the output before you run it: three lines that print "A", then "B", then "C".
- 2 Write four `console.log` lines that describe the steps of making tea, in the right order.
- 3 Swap two of the lines. Does the program break, or does only the order of the output change?

1.4 Your First Program: `console.log()`

🕒 What is it?

`console.log()` prints a message to the **console**, a special screen where developers see what their program says. Text goes inside quotes. Text inside quotes is called a **string** (you will study strings in Chapter 3).

💡 Why do we use it?

You cannot see inside your program while it runs. `console.log()` is your window into it. You use it to check that your code works and to hunt for bugs, and you will use it almost every day.

<> Syntax

Syntax pattern

```
console.log(something);
```

`console` is the tool, `.log` is the action, the brackets `( )` hold what you want to print, and `;` ends the statement.

✍ Simple example

JavaScript

```
console.log("Hello, World!");  
console.log(2 + 3);  
console.log("I am", 15, "years old");
```

Output

```
Hello, World!  
5  
I am 15 years old
```

Text needs quotes, numbers do not, and commas let you print several things on one line, separated by spaces.

🌐 Real-world example

Programmers use `console.log()` to check their work. Imagine a shop program that calculates a bill. To see whether the total is right, print it:

JavaScript

```
let price = 250;  
let quantity = 3;  
console.log("Total:", price * quantity);
```

Output

```
Total: 750
```

You will meet `let` in the next chapter. For now, read it as "store the number 250 in a box called `price`".

⚠️ Common mistakes

✗ Writing `Console.log` or `console.Log`. JavaScript is **case-sensitive**: capital letters matter.

- ✗ Forgetting the quotes around text. In `console.log(Hello)`, JavaScript thinks `Hello` is a name and fails.
- ✗ Forgetting a closing bracket or quote. Every `(` needs a `)` and every opening quote needs a closing quote.

JavaScript

```
Console.log("Hi");    // wrong: capital C
console.log(Hello);    // wrong: no quotes around the text
console.log("Hi");    // wrong: missing )
```

🔑 Key points

- `console.log()` prints values to the console.
- Put text inside quotes. Numbers do not need quotes.
- Separate several values with commas.
- JavaScript is case-sensitive.

☰ Practice

- 1 Print your name.
- 2 Print three lines: your favorite food, your favorite color, and your favorite subject.
- 3 Print the result of `15 * 4`, and then print the sentence `"15 times 4 is"` followed by the result on the same line.

Comments: notes for humans

Anything after `//` on a line is a **comment**. The engine ignores it, so you can use comments to explain your code.

JavaScript

```
// This whole line is a comment  
console.log("Hi"); // this part is a comment too  
  
/* A comment can also  
   cover many lines. */
```

📌 Mini project: Hello, Console!

Build a tiny web page that greets you in the console. You will use two files, just like a real project.

1. Make a folder called `js-book`. Inside it, create a file named `index.html`:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My First JavaScript</title>
</head>
<body>
  <h1>Open the console!</h1>
  <script src="script.js"></script>
</body>
</html>
```

2. In the same folder, create `script.js`:

JavaScript

```
console.log("Hello, Console!");
console.log("My name is Sam."); // put your own name here
console.log("Today I start learning JavaScript.");
```

3. Open `index.html` in your browser, press F12, and click the **Console** tab. Your three messages appear.

Challenge: add three more lines about yourself. Then break something on purpose (for example, delete a closing quote), reload, and read the error message.

🔖 Chapter 1 in one minute

- JavaScript makes web pages interactive: HTML is structure, CSS is style, JavaScript is behavior.
- It runs in an engine, inside the browser or inside Node.js.
- The engine reads your code from top to bottom, one statement at a time.
- `console.log()` shows values in the console, and it is your best debugging friend.
- JavaScript is case-sensitive, and error messages are there to help you.

▼ Sample answers for Chapter 1

1.4, exercise 3:

JavaScript

```
console.log(15 * 4);  
console.log("15 times 4 is", 15 * 4);
```

Output

```
60  
15 times 4 is 60
```

1.3, exercise 2:

JavaScript

```
console.log("Boil the water");  
console.log("Add the tea leaves");  
console.log("Wait three minutes");  
console.log("Pour the tea into a cup");
```

2

Chapter 2

Variables

Programs need to remember things: your score in a game, the name you typed, the price of an item. A **variable** is how a program remembers. In this chapter you will learn how to create variables, change them, protect them, and give them good names.

What you will learn

- What a variable is and how to create one
- The difference between `let`, `const`, and `var`
- How to change (reassign) a variable
- Which values should be constants
- The rules and good habits for naming variables

2.1 Variables with let

🎯 What is it?

A **variable** is a named box in the computer's memory. The name is the label on the box, and the value is what is inside. With `let`, you can look inside the box any time, and you can also put a new value inside.

💡 Why do we use it?

A program without variables can only work with values you type directly into the code. Variables let you store a value once, use it many times, and change it while the program runs, like a score that goes up.

<> Syntax

Syntax pattern

```
let variableName = value;
```

`let` creates the variable. `variableName` is the label. The `=` sign means **store** (it does not mean "is equal to"). `value` is what goes into the box.

Simple example

JavaScript

```
let score = 0;
console.log(score);

score = 10;           // put a new value in the same box
console.log(score);
```

Output

```
0
10
```

You write `let` only the first time. After that, just use the name: `score = 10;` .

Real-world example

A game keeps a player's name and points. The points change as the game goes on:

JavaScript

```
let playerName = "Riya";
let points = 20;

points = points + 5;  // Riya scored 5 more points
console.log(playerName, "has", points, "points");
```


Output

Riya has 25 points

Read `points = points + 5` from right to left: take the old value (20), add 5, and store the answer (25) back in `points`.

⚠ Common mistakes

- ✗ Writing `let` twice for the same name. The second `let` causes an error.
- ✗ Reading `=` as "equals". In JavaScript, `=` means "store the value on the right into the box on the left".
- ✗ Using a variable before you have created it.

JavaScript

```
let score = 1;  
let score = 2; // Error: 'score' has already been declared
```

🔑 Key points

- A variable is a named box that holds a value.
- Create it with `let`, and change it later without `let`.
- `=` stores a value. It does not compare.
- A variable must be created before you use it.

☰ Practice

- 1 Create a variable called `age`, print it, change it to a new number, and print it again.
- 2 Create `cups` with the value `2`. Add `1` to it and print the result.
- 3 Create two variables, `firstName` and `lastName`, and print them on one line.

2.2 Constants with `const`

🕒 What is it?

`const` creates a variable that **cannot be changed** after you give it a value. The word comes from "constant", which means "stays the same".

💡 Why do we use it?

Some values should never change: the value of pi, a tax rate, the name of your website. With `const`, JavaScript protects you from changing them by accident, and it tells anyone reading your code, "this stays fixed". A good habit is to use `const` first, and switch to `let` only when the value really needs to change.

<> Syntax

Syntax pattern

```
const constantName = value;
```

✎ Simple example

JavaScript

```
const pi = 3.14159;  
console.log(pi);
```

Output

```
3.14159
```

If you try to change it, JavaScript stops you:

JavaScript

```
const pi = 3.14159;  
pi = 3;    // Error: Assignment to constant variable.
```

🌐 Real-world example

A shop has a fixed tax rate, but the price changes for every item:

JavaScript

```
const taxRate = 0.13;  
let price = 200;  
  
let total = price + price * taxRate;  
console.log("Total with tax:", total);
```

Output

```
Total with tax: 226
```

The tax rate is `const` because it never changes while the program runs. The price can change, so it uses `let`.

⚠ Common mistakes

- ✗ Trying to change a `const` value. You get an error, and this is the protection working.
- ✗ Creating a `const` without a value. `const x;` is an error, because you can never give it a value later.
- ✗ Using `const` for something that must change, such as a score or a counter.

🔑 Key points

- `const` creates a variable that cannot be reassigned.
- It must get its value at the moment it is created.
- Use `const` by default and `let` when the value will change.
- Errors from `const` are helpful: they show you a mistake early.

🔧 Practice

- 1 Create `const daysInWeek = 7` and print it. Then try to change it and read the error message.
- 2 Write a program that uses `const pi` and a `let radius` to print the area of a circle (`pi * radius * radius`).
- 3 Decide: should each of these be `let` or `const`? A person's birthday, the number of items in a shopping cart, the name of your school.

2.3 The Old Way: var

🎯 What is it?

`var` is the original way to create variables in JavaScript. It was the only way until 2015, when `let` and `const` arrived. It still works, but it has some

confusing behavior.

💡 Why do we use it?

You need to recognize `var` because you will see it in older tutorials and older code. But in your own code, use `let` and `const`.

<> Syntax

Syntax pattern

```
var variableName = value;
```

✍ Simple example

JavaScript

```
var fruit = "mango";  
console.log(fruit);
```

Output

```
mango
```

Here is the biggest difference. A **block** is any code between curly brackets `{ }`. A `let` variable stays inside its block, but a `var` variable leaks out:

JavaScript

```
if (true) {  
  var leaky = "I escape the block";  
  let safe = "I stay inside the block";  
}  
  
console.log(leaky);  
// console.log(safe); // this would be an error: safe is not defined
```

Output

I escape the block

🌐 Real-world example

Imagine a big program. Early on, a programmer writes `var total = 100;`. Two hundred lines later, someone else writes the same name again:

JavaScript

```
var total = 100;
// ... many lines later ...
var total = 5;           // no error! The old value is silently lost
console.log(total);
```

Output

5

With `let`, the second line would give an error and warn you. That is why modern code avoids `var`.

⚠️ Common mistakes

- ✗ Using `var` out of habit. Choose `let` or `const` instead.
- ✗ Expecting a `var` variable to stay inside `{ }`. It does not.
- ✗ Creating the same `var` twice by accident. JavaScript will not warn you.

🔑 Key points

- `var` is the old way to create variables.
- It ignores block boundaries, and it lets you create the same name twice.
- Read it, but do not write it in new code.
- Prefer `const`, and use `let` when the value changes.

☰ Practice

- 1 Rewrite `var color = "red";` using `let`, and then using `const`.
- 2 Run the "leaky" example, then change `var` to `let` and read the error.
- 3 In your own words: why is it a problem when a variable "leaks" out of its block?

2.4 Naming Variables

🕒 What is it?

A variable's name is called an **identifier**. JavaScript has strict **rules** (which you must follow) and there are good **habits** (which make your code easy to read).

Rules:

- Use letters, digits, `_`, and `$`.
- Do not start with a digit.
- No spaces or dashes.
- Do not use a reserved word such as `let`, `const`, `if`, or `class`.
- Names are case-sensitive: `age` and `Age` are two different variables.

Good habits:

- Use **camelCase**: start with a small letter and capitalize each new word (`firstName`, `totalPrice`).
- Pick names that describe the value.
- Start yes/no variables with `is` or `has` (`isLoggedIn`, `hasPaid`).
- Write fixed settings in `UPPER_CASE` with underscores if you like (`MAX_USERS`).

💡 Why do we use it?

Code is read many more times than it is written. A clear name like `totalPrice` explains itself, while `x` makes everyone guess, including you next week.

<> Syntax

Syntax pattern

```
firstName    totalPrice    isLoggedIn    MAX_USERS
```

✍ Simple example

JavaScript

```
let firstName = "Nabin";
let lastName = "Thapa";
let isStudent = true;
let totalMarks = 450;

console.log(firstName, lastName, isStudent, totalMarks);
```

Output

```
Nabin Thapa true 450
```

These names break the rules:

JavaScript

```
let 1stPlace = "Ava"; // wrong: starts with a number
let my name = "Ava";  // wrong: has a space
let let = 5;          // wrong: reserved word
```

🌐 Real-world example

Compare two versions of the same shopping calculation. The second one explains itself:

JavaScript

```
// Hard to understand
let x = 499;
let y = 3;
let z = x * y;

// Easy to understand
let itemPrice = 499;
let quantity = 3;
let totalCost = itemPrice * quantity;

console.log("Total cost:", totalCost);
```

Output

```
Total cost: 1497
```

⚠ Common mistakes

- ✗ Mixing capital letters: `totalMarks` and `totalmarks` are different names, so one of them will cause a "not defined" error.
- ✗ Using very short names (`a` , `q`) or meaningless names (`data1` , `thing`).
- ✗ Using spaces or dashes. Write `firstName` , not `first name` or `first-name` .

🔑 Key points

- Names may use letters, digits, `_` , and `$` , but cannot start with a digit.
- Names are case-sensitive.
- Use camelCase and descriptive names.
- Start yes/no names with `is` or `has` .

☞ Practice

- 1 Which of these names are wrong, and why? `user_name`, `2ndPlayer`, `total price`, `isOnline`, `class`
- 2 Write good variable names for: a student's age, the price of a pen, whether a user has logged in.
- 3 Rename this: `let a = 15; let b = 4; let c = a * b;` so it describes a rectangle's area.

🚩 Mini project: Student ID Card

Print a student ID card using variables. Some values are fixed (`const`), and others change (`let`).

JavaScript

```
const schoolName = "Green Valley School";
let studentName = "Anisha Rai";
let grade = 9;
let section = "B";
let rollNumber = 14;

console.log("=== Student ID Card ===");
console.log("School:", schoolName);
console.log("Name:", studentName);
console.log("Class:", grade, "Section:", section);
console.log("Roll number:", rollNumber);

// A new school year begins: the student moves up a class
grade = grade + 1;
rollNumber = 7;

console.log("=== Next year ===");
console.log("Class:", grade, "Section:", section);
console.log("Roll number:", rollNumber);
```

Output

```
=== Student ID Card ===
School: Green Valley School
Name: Anisha Rai
Class: 9 Section: B
Roll number: 14
=== Next year ===
Class: 10 Section: B
Roll number: 7
```

Challenge 1: put your own details in the card.

Challenge 2 (a classic puzzle): you have two boxes, `a = 5` and `b = 8`. Swap their values. You will need a third box to hold one value for a moment.

🔖 Chapter 2 in one minute

- A variable is a named box that stores a value.
- `let` for values that change, `const` for values that stay fixed, and avoid `var`.
- `=` stores a value from the right into the name on the left.
- Names are case-sensitive, use camelCase, and should describe the value.

▼ Sample answers for Chapter 2

2.1, exercise 3:

JavaScript

```
let firstName = "Sita";
let lastName = "Rana";
console.log(firstName, lastName);
```

Output

```
Sita Rana
```

2.2, exercise 2:

JavaScript

```
const pi = 3.14159;
let radius = 5;
console.log("Area:", pi * radius * radius);
```

Output

Area: 78.53975

2.4, exercise 1: `2ndPlayer` starts with a digit, `total price` has a space, and `class` is a reserved word. The others are fine.

Mini project, challenge 2 (swap):

JavaScript

```
let a = 5;
let b = 8;
let temp = a; // keep a's value safe
a = b;
b = temp;
console.log(a, b);
```

Output

8 5

3

Chapter 3

Data Types

Not every value is the same kind of thing. A name is text, an age is a number, and "Is the door open?" is a yes-or-no answer. JavaScript calls these kinds of values **data types**. Think of containers: a bottle holds water, a box holds shoes. Knowing the type of a value tells you what you can do with it.

What you will learn

- The seven basic data types: string, number, boolean, undefined, null, BigInt, and symbol
- How to write each type
- Why `"5"` and `5` are not the same
- How to check a type with `typeof`

3.1 Strings

🕒 What is it?

A **string** is text: letters, digits, spaces, and symbols inside quotes. You can use double quotes `"..."`, single quotes `'...'`, or backticks.

💡 Why do we use it?

Names, messages, emails, and addresses are all text. Almost every program works with strings.

<> Syntax

Syntax pattern

"double quotes" 'single quotes' `backticks`

Backticks are special. They let you put values inside the text with `${ }`. This is called a **template literal**.

Simple example

JavaScript

```
let name = "Sita";
let greeting = 'Hello';

console.log(greeting + ", " + name + "!"); // joining with +
console.log(`${greeting}, ${name}!`);     // template literal
```

Output

```
Hello, Sita!
Hello, Sita!
```

Joining strings with `+` is called **concatenation**. Template literals give the same result and are usually easier to read.

Real-world example

A website greets you and tells you about your messages:

JavaScript

```
let userName = "Arjun";
let unread = 3;

console.log(`Welcome back, ${userName}! You have ${unread} new messages.`);
```

Output

Welcome back, Arjun! You have 3 new messages.

⚠ Common mistakes

- ✗ Starting with one kind of quote and ending with another: `"Hello'` is an error.
- ✗ Forgetting quotes, so JavaScript treats your text as a variable name.
- ✗ Thinking `"5"` is a number. Look at what `+` does with text:

JavaScript

```
console.log("5" + "5");  
console.log(5 + 5);
```

Output

```
55  
10
```

🔑 Key points

- A string is text inside quotes (`"..."`, `'...'`, or backticks).
- Use `+` to join strings.
- Backticks with `${ }` put values inside text.
- `"5"` is text, and `5` is a number.

☰ Practice

- 1 Create a string variable with your favorite movie, and print it.
- 2 Use a template literal to print: `My name is ..., and I am ... years old.` using two variables.
- 3 Predict, then check: what does `"10" + "20"` print? What about `10 + 20`?

3.2 Numbers

🎯 What is it?

The **number** type holds every kind of number: whole numbers like `15` and decimals like `99.5`. JavaScript uses one type for both.

💡 Why do we use it?

Prices, ages, scores, distances, and every calculation use numbers.

<> Syntax

Syntax pattern

```
let age = 15;
let price = 99.5;
let big = 1_000_000;    // underscores make big numbers easier to read
```

Numbers never use quotes. Two special number values exist too: `Infinity` and `NaN` (which means "Not a Number", and you will meet it in Chapter 11).

✍ Simple example

JavaScript

```
let age = 15;
let price = 99.5;

console.log(age + 1);
console.log(price * 2);
console.log(10 / 3);
console.log(0.1 + 0.2);
```

Output

```
16
199
3.3333333333333335
0.30000000000000004
```

🌐 Real-world example

Four friends share a restaurant bill equally:

JavaScript

```
let bill = 1200;
let friends = 4;

console.log("Each person pays:", bill / friends);
```

Output

```
Each person pays: 300
```

❗ Why is $0.1 + 0.2$ not 0.3 ?

Computers store decimals in binary, and some decimals cannot be stored exactly, just like $1/3$ cannot be written exactly as a decimal. This happens in every programming language. In Chapter 11 you will learn how to round results neatly.

⚠️ Common mistakes

- ✗ Putting numbers in quotes (`"15"`). That is text, and it behaves differently.
- ✗ Writing a comma inside a number (`1,000`). Use `1000` or `1_000` .
- ✗ Being surprised by tiny decimal errors such as `0.30000000000000004` . It is normal.

🔑 Key points

- One `number` type holds whole numbers and decimals.
- Numbers do not use quotes.
- `NaN` and `Infinity` are special number values.
- Decimals can have tiny rounding errors.

🔧 Practice

- 1 Create variables for the length and width of a room (numbers), and print its area.
- 2 Print the result of `7 / 2`. Is it what you expected?
- 3 Split a bill of `950` between `5` people and print each share.

3.3 Booleans

🎯 What is it?

A **boolean** has only two possible values: `true` or `false` (no quotes). It is like a light switch: on or off.

💡 Why do we use it?

Programs make decisions, and decisions need yes-or-no answers: Is the user logged in? Is the password correct? Is the game over? Booleans store those answers.

<> Syntax

Syntax pattern

```
let isRaining = true;
let hasPassed = false;
```

✎ Simple example

JavaScript

```
let isStudent = true;
let hasPassed = false;

console.log(isStudent);
console.log(hasPassed);
console.log(10 > 5);    // a comparison gives a boolean
console.log(10 === 5);
```

Output

```
true
false
true
false
```

🌐 Real-world example

A page decides what to show, depending on a boolean (you will learn `if` in Chapter 5):

JavaScript

```
let isLoggedIn = true;

if (isLoggedIn) {
  console.log("Show my account");
} else {
  console.log("Please log in");
}
```

Output

Show my account

⚠ Common mistakes

- ✗ Writing `"true"` with quotes. That is a string, not a boolean.
- ✗ Capital letters: `True` and `FALSE` are wrong. Write `true` and `false`.
- ✗ Using confusing names. `isLoggedIn` is clearer than `login`.

🔑 Key points

- A boolean is either `true` or `false`.
- Never put quotes around them.
- Comparisons such as `10 > 5` produce booleans.
- Name booleans with `is` or `has` so they read like questions.

🔧 Practice

- 1 Create `isWeekend` and `hasHomework` variables and print them.
- 2 What does `5 > 8` print? What about `5 < 8`?
- 3 Write a boolean variable name for: "The user accepted the terms."

3.4 undefined and null

🎯 What is it?

Both mean "no value", but in different ways.

- `undefined` means the box was created, but nothing was put inside. JavaScript gives you this automatically.

- `null` means the box is **empty on purpose**. You choose to write it.

💡 Why do we use it?

Programs often need a value that means "nothing yet": a seat that no one has chosen, or a search that found no result.

<> Syntax

Syntax pattern

```
let x;           // undefined: no value given
let y = null;    // null: empty on purpose
```

✍ Simple example

JavaScript

```
let seat;
let winner = null;

console.log(seat);
console.log(winner);
```

Output

```
undefined
null
```

🌐 Real-world example

A quiz app remembers which answer the player picked. Before they choose, it is `null`:

JavaScript

```
let selectedAnswer = null;      // nobody has picked yet
console.log("Selected:", selectedAnswer);

selectedAnswer = "B";          // the player picks B
console.log("Selected:", selectedAnswer);
```

Output

```
Selected: null
Selected: B
```

⚠ Common mistakes

- ✗ Mixing them up. A good rule: `undefined` is "JavaScript did not get a value", `null` is "I decided there is no value".
- ✗ Writing `"null"` in quotes. That is a string.
- ✗ Being surprised by `undefined` when you forget to give a variable a value.

🔑 Key points

- `undefined` means a variable exists but has no value yet.
- `null` means "empty on purpose".
- Both mean "nothing", but they are different types of values.
- You usually write `null` yourself and let JavaScript create `undefined`.

🔧 Practice

- 1 Create a variable with no value and print it. What do you see?
- 2 Create `currentUser` with the value `null`, and then give it the value `"Sam"`.
- 3 Explain the difference between `undefined` and `null` in one sentence.

3.5 BigInt and Symbol

🎯 What is it?

These are two special types that you will not need often.

- **BigInt** holds very large whole numbers. Write an `n` at the end: `123n`.
 - **Symbol** creates a value that is always unique. Even two symbols with the same description are different.
-

💡 Why do we use it?

Normal numbers lose exact accuracy above about 9 quadrillion (9,007,199,254,740,991). BigInt keeps every digit exact. Symbols help advanced programs avoid name clashes.

<> Syntax

Syntax pattern

```
let big = 123456789012345678901234567890n;  
let id = Symbol("description");
```

✍ Simple example

JavaScript

```
let big = 9007199254740993n;  
  
console.log(big + 1n);           // BigInt keeps every digit  
console.log(9007199254740993 + 1); // a normal number loses accuracy  
  
let a = Symbol("id");  
let b = Symbol("id");  
console.log(a === b);           // every symbol is unique
```


Output

```
9007199254740994n
9007199254740992
false
```

🌐 Real-world example

A social app gives every post a 19-digit ID. As a normal number, the last digits could be wrong. As a BigInt, the ID stays exact:

JavaScript

```
let postId = 1234567890123456789n;
console.log("Post ID:", postId);
```

Output

```
Post ID: 1234567890123456789n
```

⚠️ Common mistakes

- ✗ Mixing BigInt and normal numbers: `5n + 1` gives an error. Use `5n + 1n`.
- ✗ Forgetting the `n` at the end of a BigInt.
- ✗ Expecting two symbols with the same description to be equal. They never are.

🔑 Key points

- BigInt (`123n`) stores huge whole numbers exactly.
- Symbol creates a unique value.
- You cannot mix BigInt with normal numbers in one calculation.
- Both are rare in beginner programs. Just know they exist.

☰ Practice

- 1 Create a BigInt with the value `500n` and add `1n` to it.
- 2 What happens when you run `console.log(5n + 1)`? Read the error message.
- 3 Create two symbols with the same description and compare them with `===`.

3.6 Checking a Type with typeof

🕒 What is it?

`typeof` is a tool that tells you the type of a value. It gives its answer as text.

🔍 Why do we use it?

When a program misbehaves, the cause is often a wrong type, such as `"5"` (text) instead of `5` (number). `typeof` lets you check quickly.

<> Syntax

Syntax pattern

```
typeof value
```

Simple example

JavaScript

```
console.log(typeof "hello");
console.log(typeof 42);
console.log(typeof true);
console.log(typeof undefined);
console.log(typeof null);
console.log(typeof 10n);
console.log(typeof Symbol("x"));
```

Output

```
string
number
boolean
undefined
object
bigint
symbol
```

A famous quirk

`typeof null` gives `"object"`. This is an old mistake in JavaScript that was never fixed, because fixing it would break old websites. `null` is **not** really an object. Just remember this special case.

Real-world example

Values that come from a web form are always text, even if the user typed a number. `typeof` helps you notice:

JavaScript

```
let ageFromForm = "15";

console.log(typeof ageFromForm);
console.log(typeof Number(ageFromForm)); // Number() converts text to a number
```

Output

```
string
number
```

⚠ Common mistakes

- ✗ Expecting `typeof null` to say `"null"`.
- ✗ Forgetting that the answer is a string. To compare, write `typeof x === "number"`, with quotes.
- ✗ Expecting `typeof` to separate arrays from objects. Both give `"object"` (you will see better tools in Chapter 8).

🔑 Key points

- `typeof` returns the type of a value as text.
- The types you know: `"string"`, `"number"`, `"boolean"`, `"undefined"`, `"bigint"`, `"symbol"`, and `"object"`.
- `typeof null` is `"object"`, an old quirk.
- Check types when a result looks strange.

☞ Practice

- 1 Use `typeof` on your name, your age, and `false`.
- 2 What does `typeof "42"` print? Why is it not `"number"`?
- 3 Predict the output of `console.log(typeof typeof 5)`, then run it.

📌 Mini project: Data Type Detective

Guess each type before you run the program. Then run it and check how many you got right.

JavaScript

```
let mystery1 = "42";
let mystery2 = 42;
let mystery3 = true;
let mystery4;
let mystery5 = null;
let mystery6 = 3.14;

console.log("mystery1 is", typeof mystery1);
console.log("mystery2 is", typeof mystery2);
console.log("mystery3 is", typeof mystery3);
console.log("mystery4 is", typeof mystery4);
console.log("mystery5 is", typeof mystery5);
console.log("mystery6 is", typeof mystery6);

// Bonus: what does + do with different types?
console.log(mystery1 + mystery2);
console.log(mystery2 + mystery2);
```

Output

```
mystery1 is string
mystery2 is number
mystery3 is boolean
mystery4 is undefined
mystery5 is object
mystery6 is number
4242
84
```

Challenge: add three more mystery values of your own, such as a `BigInt` and a string with backticks, and predict their types.

🔖 Chapter 3 in one minute

- Every value has a type: string, number, boolean, undefined, null, BigInt, or symbol.
- Strings are text in quotes, and "5" is not the same as 5.
- `undefined` means "no value yet", and `null` means "empty on purpose".
- `typeof` tells you the type of a value. `typeof null` is "object", an old quirk.

▼ Sample answers for Chapter 3

3.1, exercise 3:

JavaScript

```
console.log("10" + "20");  
console.log(10 + 20);
```

Output

```
1020  
30
```

3.2, exercise 1:

JavaScript

```
let length = 6;  
let width = 4;  
console.log("Area:", length * width);
```

Output

```
Area: 24
```

3.6, exercise 3: `typeof 5` gives `"number"`, which is a string, so the outer `typeof` gives `"string"`.

JavaScript

```
console.log(typeof typeof 5);
```

Output

```
string
```


4

Chapter 4

Operators

Think about the buttons on a calculator: plus, minus, times, divide. In JavaScript, symbols that do work with values are called **operators**. The values they work on are called **operands**. In `5 + 3`, the `+` is the operator, and `5` and `3` are the operands. In this chapter you will learn the operators you will use in almost every program.

What you will learn

- Do math with arithmetic operators, including modulus and exponentiation
- Update variables quickly with assignment operators and `++` / `--`
- Compare values, and know why `===` is better than `==`
- Combine yes-or-no answers with logical operators
- Choose between two values with the ternary operator

4.1 Arithmetic Operators

🕒 What is it?

Arithmetic operators do math. Two of them may be new to you: **modulus** (`%`) gives the *remainder* after a division, and **exponentiation** (`**`) means "to the power of".

Operator	Meaning	Example	Result
+	Add	10 + 3	13
-	Subtract	10 - 3	7
*	Multiply	10 * 3	30
/	Divide	10 / 4	2.5
%	Remainder (modulus)	10 % 3	1
**	Power	2 ** 3	8

🔗 Why do we use it?

Every calculation in a program uses these: prices, totals, scores, areas. The remainder operator is surprisingly useful. It helps you find even and odd numbers, repeat patterns, and share things fairly.

<> Syntax

Syntax pattern

a + b a - b a * b a / b a % b a ** b

✍ Simple example

JavaScript

```
console.log(10 + 3);
console.log(10 / 4);
console.log(10 % 3);
console.log(2 ** 3);

// Order of operations: * and / come before + and -
console.log(2 + 3 * 4);
console.log((2 + 3) * 4); // brackets go first
```

Output

```
13
2.5
1
8
14
20
```

🌐 Real-world example

Seventeen sweets are shared between five children. Each child gets 3 sweets, and the remainder tells you how many are left. Also, any even number has a remainder of 0 when divided by 2:

JavaScript

```
let sweets = 17;
let children = 5;
console.log("Sweets left over:", sweets % children);

console.log("10 % 2 =", 10 % 2); // 0 means even
console.log("7 % 2 =", 7 % 2);  // 1 means odd
```

Output

```
Sweets left over: 2
10 % 2 = 0
7 % 2 = 1
```

⚠️ Common mistakes

- ✗ Using `^` for power. In JavaScript, `2 ^ 3` is a different operator and gives `1`. Use `2 ** 3`.
- ✗ Forgetting the order of operations. Use brackets when you are not sure.
- ✗ Thinking `%` means percent. It means remainder.
- ✗ Dividing by zero does not crash the program. It gives `Infinity`.

🔑 Key points

- `+` `-` `*` `/` work as you expect. `%` gives the remainder, and `**` is "to the power of".
- Multiplication and division happen before addition and subtraction. Brackets go first.
- `number % 2 === 0` is a common way to test for an even number.
- `+` also joins strings.

🔧 Practice

- 1 Print the result of `(8 + 2) * 5` and of `8 + 2 * 5`. Why are they different?
- 2 What is `25 % 4`? Check with the console.
- 3 Print the area of a square with side `9` using `**`.

4.2 Assignment and Increment/Decrement Operators

🎯 What is it?

You already know `=`, which stores a value in a variable. **Assignment shortcuts** like `+=` update a variable using its own old value. The **increment** `++` adds 1 and the **decrement** `--` subtracts 1.

💡 Why do we use it?

Updating a variable is so common that programmers made shortcuts for it. `score += 5` is shorter and easier to read than `score = score + 5`. Counting up or down with `++` and `--` is used everywhere.

<> Syntax

Syntax pattern

```
x += 5;    // x = x + 5
x -= 2;    // x = x - 2
x *= 3;    // x = x * 3
x /= 2;    // x = x / 2
x++;       // x = x + 1
x--;       // x = x - 1
```

🔗 Simple example

JavaScript

```
let score = 10;

score += 5;
console.log(score);
score -= 3;
console.log(score);
score *= 2;
console.log(score);
score++;
console.log(score);
score--;
console.log(score);
```

Output

```
15
12
24
25
24
```

++ before or after?

`x++` gives the old value first and then adds 1. `++x` adds 1 first and then gives the new value.

JavaScript

```
let a = 5;
console.log(a++); // prints 5, then a becomes 6
console.log(a);
console.log(++a); // a becomes 7, then prints 7
```

Output

```
5
6
7
```

To stay safe, use `x++` on its own line, and avoid mixing it into other expressions.

Real-world example

A game tracks the player's lives and coins:

JavaScript

```
let lives = 3;
lives--; // the player got hit
lives--; // hit again
console.log("Lives left:", lives);

let coins = 0;
coins += 10; // collected a coin
coins += 10;
console.log("Coins:", coins);
```

Output

```
Lives left: 1  
Coins: 20
```

⚠ Common mistakes

- ✗ Writing `x =+ 5` instead of `x += 5`. The first one just sets `x` to positive 5.
- ✗ Using `++` or `+=` on a `const` variable. It gives an error, because it changes the value.
- ✗ Mixing `x++` into bigger expressions, which makes code hard to understand.

🔑 Key points

- `+=`, `-=`, `*=`, and `/=` update a variable using its old value.
- `++` adds 1, and `--` subtracts 1.
- The variable must be created with `let`, not `const`.
- Keep `++` and `--` on their own line.

🔧 Practice

- 1 Start with `let count = 0`. Use `++` three times and print the result.
- 2 Start with `let price = 100`. Use `*=` to double it, then `-=` to take off 20.
- 3 What are `a` and `b` after `let a = 3; let b = a++;`? Predict first, then run it.

4.3 Comparison Operators

🕒 What is it?

Comparison operators compare two values and always give a boolean (`true` or `false`).

Operator	Meaning
<code>></code> and <code><</code>	Greater than, less than
<code>>=</code> and <code><=</code>	Greater than or equal, less than or equal
<code>===</code>	Strict equal: same value and same type
<code>!==</code>	Strict not equal
<code>==</code> and <code>!=</code>	Loose equal and not equal (avoid them)

The loose `==` tries to change the types before it compares, which gives surprising results. `5 == "5"` is `true`. The strict `===` does not change anything, so `5 === "5"` is `false`.

💡 Why do we use it?

Decisions depend on comparisons: Is the age at least 18? Does the password match? Is the score higher than the best score?

<> Syntax

Syntax pattern

`a > b` `a < b` `a >= b` `a <= b` `a === b` `a !== b`

Simple example

JavaScript

```
console.log(10 > 5);  
console.log(10 <= 5);  
console.log(5 === 5);  
console.log(5 !== 3);  
console.log(5 == "5");    // loose: true  
console.log(5 === "5");  // strict: false
```

Output

```
true  
false  
true  
true  
true  
false
```

🌐 Real-world example

A ticket machine checks age, and a lock checks a PIN. Notice how `===` catches a PIN that is stored as text:

JavaScript

```
let age = 16;
console.log("Can vote?", age >= 18);

let enteredPin = "1234"; // text, typed by the user
let correctPin = 1234;    // a number
console.log("Strict match:", enteredPin === correctPin);
console.log("Loose match:", enteredPin == correctPin);
```

Output

```
Can vote? false
Strict match: false
Loose match: true
```

⚠️ Common mistakes

- ✗ Using one `=` when you want to compare. `=` stores a value, and `===` compares.
- ✗ Using `==` out of habit. Always write `===` and `!==`.
- ✗ Trusting decimal comparisons: `0.1 + 0.2 === 0.3` is `false`, because of tiny rounding errors.

JavaScript

```
console.log(0.1 + 0.2 === 0.3);
```

Output

```
false
```

🔑 Key points

- Comparisons give `true` or `false`.
- Use `===` and `!==`, and avoid `==` and `!=`.
- `===` checks the value **and** the type.
- One `=` stores, and three `===` compare.

🔧 Practice

- 1 Print the result of `7 >= 7` and of `7 > 7`.
- 2 Predict, then check: `"10" === 10` and `"10" == 10`.
- 3 Store your age in a variable and print whether you are older than 13.

4.4 Logical Operators

🕒 What is it?

Logical operators combine yes-or-no answers.

- `&&` (AND): true only if **both** sides are true.
- `||` (OR): true if **at least one** side is true.
- `!` (NOT): flips true to false, and false to true.

A	B	A && B	A B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

💡 Why do we use it?

Real decisions have more than one condition: "The user is logged in **and** has paid", or "It is Saturday **or** Sunday".

<> Syntax

Syntax pattern

`a && b` `a || b` `!a`

✍ Simple example

JavaScript

```
let age = 15;
let hasTicket = true;

console.log(age >= 12 && hasTicket);
console.log(age >= 18 || hasTicket);
console.log(!hasTicket);
```

Output

```
true
true
false
```

🌐 Real-world example

A cinema gives a discount to students or seniors, and a weekend check compares a day with two possible values:

JavaScript

```
let isStudent = true;
let isSenior = false;
console.log("Gets discount?", isStudent || isSenior);

let day = "Sunday";
let isWeekend = day === "Saturday" || day === "Sunday";
console.log("Weekend?", isWeekend);
```

Output

```
Gets discount? true
Weekend? true
```

⚠ Common mistakes

- ✗ Using a single `&` or `|`. Those are different operators. Use `&&` and `||`.
- ✗ Writing `day === "Saturday" || "Sunday"`. Each side of `||` needs its own full comparison, as in the example above.
- ✗ Forgetting brackets when mixing `&&` and `||`. `&&` is done before `||`, so add brackets to be clear.

🔑 Key points

- `&&` needs both sides true.
- `||` needs at least one side true.
- `!` flips a boolean.
- Each side of `&&` and `||` should be a complete comparison.

🔧 Practice

- 1 Write a condition that is true when `age` is between 13 and 19 (including both).

- 2 What is `!(5 > 3)` ? Predict, then check.
- 3 Write a condition that is true if `day` is `"Saturday"` or `"Sunday"`.

4.5 The Ternary Operator

🎯 What is it?

The **ternary operator** is a one-line shortcut for choosing between two values. It is the only operator with three parts, which is why it is called "ternary".

💡 Why do we use it?

It keeps small decisions short and neat, such as choosing a piece of text depending on a condition.

<> Syntax

Syntax pattern

```
condition ? valueIfTrue : valueIfFalse
```

Read it as: "If the condition is true, use the first value, otherwise use the second."

✍ Simple example

JavaScript

```
let age = 20;  
let message = age >= 18 ? "Adult" : "Minor";  
console.log(message);
```

Output

Adult

🌐 Real-world example

A shopping site shows "1 new message" or "3 new messages", with the right grammar:

JavaScript

```
let unread = 3;
let label = unread === 1 ? "1 new message" : `${unread} new messages`;
console.log(label);
```

Output

3 new messages

⚠️ Common mistakes

- ✗ Forgetting the `:` part. `condition ? value` alone is an error.
- ✗ Chaining many ternaries together. It becomes very hard to read, so use `if / else` instead (Chapter 5).
- ✗ Using it to run several actions. A ternary is for choosing a **value**.

🔑 Key points

- `condition ? a : b` gives `a` when the condition is true, otherwise `b`.
- It is a short form of a simple `if / else`.
- Use it for small choices only.

🔧 Practice

- 1 Use a ternary to store "Even" or "Odd" for the number 7.

- 2 Use a ternary to print "Pass" when `marks >= 40` and "Fail" otherwise.
- 3 Rewrite the `unread` example for a value of `0`, so it says "No new messages". (Hint: you may need two questions.)

🚩 Mini project: Shopping Bill Calculator

A shop applies a member discount, adds tax, and gives free delivery for big bills. This uses almost every operator from this chapter.

JavaScript

```
const taxRate = 0.1;
let price = 250;
let quantity = 4;
let isMember = true;

let subtotal = price * quantity;
let discount = isMember ? subtotal * 0.2 : 0;
let afterDiscount = subtotal - discount;
let tax = afterDiscount * taxRate;
let total = afterDiscount + tax;

console.log("Subtotal:", subtotal);
console.log("Discount:", discount);
console.log("Tax:", tax);
console.log("Total:", total);
console.log("Free delivery?", total >= 500);
console.log("Even quantity?", quantity % 2 === 0);
```

Output

```
Subtotal: 1000
Discount: 200
Tax: 80
Total: 880
Free delivery? true
Even quantity? true
```

Challenge: change `isMember` to `false` and `quantity` to `1`. Predict the new output before you run it. Then add a variable `tip` and use `+=` to add it to `total`.

📖 Chapter 4 in one minute

- Arithmetic: `+` `-` `*` `/` `%` `**` . Modulus gives the remainder.
- `+=` , `-=` , `++` , and `--` update a variable in a shorter way.
- Compare with `===` and `!==` . Avoid `==` .
- `&&` , `||` , and `!` combine yes-or-no answers.
- `condition ? a : b` chooses between two values.

▼ Sample answers for Chapter 4

4.3, exercise 1:

JavaScript

```
console.log(7 >= 7);  
console.log(7 > 7);
```

Output

```
true  
false
```

4.4, exercise 1 (age between 13 and 19):

JavaScript

```
let age = 16;  
console.log(age >= 13 && age <= 19);
```

Output

```
true
```

4.5, exercise 1:

JavaScript

```
let number = 7;  
let type = number % 2 === 0 ? "Even" : "Odd";  
console.log(type);
```

Output

Odd

5

Chapter 5

Conditions

Every day you make decisions: if it is raining, take an umbrella; otherwise, wear sunglasses. Programs make decisions in the same way. **Conditions** let your code choose different paths depending on the situation. This is where programs start to feel smart.

What you will learn

- Run code only when a condition is true, using `if` and `else`
- Check many options with `else if`
- Put one decision inside another (nested conditions)
- Choose between two values with the ternary operator
- Match exact values with `switch`
- Understand truthy and falsy values

5.1 if and else

🎯 What is it?

An `if` **statement** runs a block of code only when a condition is `true`. An optional `else` block runs when the condition is `false`. A **block** is the code between curly brackets `{ }`.

💡 Why do we use it?

Without conditions, a program would do exactly the same thing every time. With conditions, it can react: show a welcome or an error, allow or block, give

a discount or not.

<> Syntax

Syntax pattern

```
if (condition) {  
    // runs when the condition is true  
} else {  
    // runs when the condition is false  
}
```

The condition goes inside brackets, and it should give `true` or `false`.

Simple example

JavaScript

```
let age = 15;  
  
if (age >= 18) {  
    console.log("You can vote.");  
} else {  
    console.log("You are too young to vote.");  
}
```

Output

```
You are too young to vote.
```

Real-world example

An age checker for tickets. A museum charges less for children under 12:

JavaScript

```
let age = 10;
let ticketPrice;

if (age < 12) {
  ticketPrice = 100;
} else {
  ticketPrice = 200;
}

console.log("Ticket price:", ticketPrice);
```

Output

```
Ticket price: 100
```

⚠ Common mistakes

- ✗ Using one `=` instead of `===`. In `if (age = 18)` you store 18 into `age`, and the condition is always true.
- ✗ Adding a semicolon right after the condition: `if (age > 18);` This ends the `if` early, so the block below always runs.
- ✗ Forgetting or mismatching curly brackets. Every `{` needs its `}`.

JavaScript

```
if (age = 18) {           // wrong: this stores 18, it does not compare
  console.log("Adult");
}
```

🔑 Key points

- `if` runs a block only when the condition is true.
- `else` is the "otherwise" part, and it is optional.
- Use `===` to compare, never a single `=`.
- Indent the code inside `{ }` so that it is easy to read.

☰ Practice

- 1 Write an `if / else` that prints `"Pass"` if `marks` is 40 or more, and `"Fail"` otherwise.
- 2 Write an `if` that prints `"It is hot!"` when `temperature` is above 30. Test it with 35 and with 20.
- 3 Store `true` in `isRaining`. Print `"Take an umbrella"` or `"Enjoy the sun"` accordingly.

5.2 else if

🕒 What is it?

With `else if` you can check several conditions, one after the other. JavaScript checks from top to bottom. The **first** condition that is true runs, and all the rest are skipped.

💡 Why do we use it?

Many decisions have more than two outcomes: grades A, B, C, or D; hot, warm, or cold; small, medium, or large.

<> Syntax

Syntax pattern

```
if (condition1) {  
  // ...  
} else if (condition2) {  
  // ...  
} else {  
  // ...  
}
```

🔗 Simple example

JavaScript

```
let temperature = 28;  
  
if (temperature > 35) {  
  console.log("Very hot");  
} else if (temperature > 25) {  
  console.log("Warm");  
} else if (temperature > 15) {  
  console.log("Pleasant");  
} else {  
  console.log("Cold");  
}
```

Output

Warm

Since 28 is not above 35, JavaScript moves on. Since 28 **is** above 25, it prints "Warm" and skips everything after it.

🌐 Real-world example

A grade calculator. A school turns marks into grades:

JavaScript

```
let marks = 82;
let grade;

if (marks >= 90) {
  grade = "A+";
} else if (marks >= 80) {
  grade = "A";
} else if (marks >= 70) {
  grade = "B";
} else if (marks >= 60) {
  grade = "C";
} else if (marks >= 40) {
  grade = "D";
} else {
  grade = "Fail";
}

console.log("Marks:", marks, "Grade:", grade);
```

Output

Marks: 82 Grade: A

⚠ Common mistakes

- ✗ Putting the conditions in the wrong order. If `marks >= 40` came first, then a score of 95 would get a "D", because that check is true and stops the chain.
- ✗ Using separate `if` statements when you meant `else if`. Separate `if` statements are all checked, and several can run.
- ✗ Forgetting the last `else` for the "none of the above" case.

🔑 Key points

- `else if` lets you check many conditions in order.
- Only the first true block runs.

- The order of the conditions matters: go from the most specific to the most general.
- End with `else` to catch everything else.

Practice

- 1 Write a program that prints `"Child"` for age under 13, `"Teenager"` for 13 to 19, and `"Adult"` otherwise.
- 2 Change the grade calculator to give `"Excellent"` for 90 or more, `"Good"` for 60 to 89, and `"Try again"` below that.
- 3 Print `"Positive"`, `"Negative"`, or `"Zero"` for a number stored in a variable.

5.3 Nested Conditions

What is it?

A **nested condition** is an `if` inside another `if`. It is like a door with a second door behind it: you only reach the second check if you pass the first.

Why do we use it?

Some decisions come in steps. First you check whether a user is logged in, and then you check what kind of user they are.

<> Syntax

Syntax pattern

```
if (condition1) {  
  if (condition2) {  
    // runs when both are true  
  }  
}
```

✎ Simple example

JavaScript

```
let isLoggedIn = true;  
let isAdmin = false;  
  
if (isLoggedIn) {  
  if (isAdmin) {  
    console.log("Welcome, admin!");  
  } else {  
    console.log("Welcome, user!");  
  }  
} else {  
  console.log("Please log in first.");  
}
```

Output

```
Welcome, user!
```

🌐 Real-world example

A login check. The program checks the username first, and only then the password:

JavaScript

```
const correctUsername = "student01";
const correctPassword = "js2026";

let usernameEntered = "student01";
let passwordEntered = "js2025";

if (usernameEntered === correctUsername) {
  if (passwordEntered === correctPassword) {
    console.log("Login successful. Welcome!");
  } else {
    console.log("Wrong password. Try again.");
  }
} else {
  console.log("Username not found.");
}
```

Output

Wrong password. Try again.

A safer login message

Real websites usually say "Username or password is incorrect" and do not tell you which one was wrong. That way, an attacker cannot learn which usernames exist.

Common mistakes

- ✗ Nesting too deeply. If your code goes four or five levels in, look for a simpler way.
 - ✗ Mixing up which `}` closes which block. Indent carefully.
 - ✗ Nesting when `&&` would be simpler. `if (isLoggedIn && isAdmin)` is one check instead of two.
-

🔑 Key points

- A nested condition is an `if` inside an `if`.
- The inner check only runs when the outer one is true.
- Use `&&` when both conditions lead to the same result.
- Keep nesting shallow so that your code stays readable.

🔧 Practice

- 1 Write a nested condition: if `hasTicket` is true, then check `age`. Print "Enter" when `age >= 12`, otherwise "Too young". If there is no ticket, print "Buy a ticket".
- 2 Rewrite the admin example using `&&`, so that it prints "Welcome, admin!" or "Access denied".
- 3 Add a third check to the login example: an `isBlocked` variable that stops everything if it is true.

5.4 The Ternary Operator for Decisions

🎯 What is it?

You met the ternary operator in Chapter 4. It is a one-line `if / else` that **produces a value**. Here you learn when to choose it.

💡 Why do we use it?

It makes small decisions short and neat. Use it when you just need to pick one of two values. Use `if / else` when you need several lines of code in each branch.

<> Syntax

Syntax pattern

```
let result = condition ? valueIfTrue : valueIfFalse;
```

✍ Simple example

JavaScript

```
let age = 17;  
let status = age >= 18 ? "an adult" : "a minor";  
console.log("You are", status);
```

Output

```
You are a minor
```

🌐 Real-world example

A website button changes its label depending on whether you are logged in:

JavaScript

```
let isLoggedIn = false;  
let buttonText = isLoggedIn ? "Log out" : "Log in";  
console.log(buttonText);
```

Output

```
Log in
```

⚠ Common mistakes

- ✗ Chaining three or more ternaries. It is very hard to read, so switch to `else if`.
- ✗ Using a ternary to run several actions. It is meant to choose a value.
- ✗ Forgetting the `: else` part. A ternary always needs both values.

🔑 Key points

- A ternary is a compact `if / else` that gives a value.
- Use it for simple, two-way choices.
- Use `if / else` when the logic grows.

🔗 Practice

- 1 Use a ternary to store `"Weekend"` or `"Weekday"` in a variable, depending on a boolean `isWeekend`.
- 2 Print `"Even"` or `"Odd"` for a number, using `%` and a ternary.
- 3 When is `if / else` a better choice than a ternary? Write one sentence.

5.5 The switch Statement

🎯 What is it?

A `switch` statement compares one value against many exact options, called **cases**. Think of a vending machine keypad: press `B2` and you get exactly the item in `B2`.

💡 Why do we use it?

When you compare the same variable against many exact values (days, menu choices, commands), `switch` is cleaner than a long `else if` chain.

<> Syntax

Syntax pattern

```
switch (value) {  
  case option1:  
    // code  
    break;  
  case option2:  
    // code  
    break;  
  default:  
    // runs when no case matched  
}
```

`break` stops the switch after a match. `default` is like the final `else`.

Simple example

JavaScript

```
let day = 3;  
let dayName;  
  
switch (day) {  
  case 1:  
    dayName = "Monday";  
    break;  
  case 2:  
    dayName = "Tuesday";  
    break;  
  case 3:  
    dayName = "Wednesday";  
    break;  
  default:  
    dayName = "Another day";  
}  
  
console.log(dayName);
```


Output

Wednesday

🌐 Real-world example

A tiny calculator picks the operation from a text command. You can also let several cases share one block by leaving out `break` on purpose:

JavaScript

```
let operation = "multiply";
let a = 6;
let b = 7;

switch (operation) {
  case "add":
    console.log(a + b);
    break;
  case "subtract":
    console.log(a - b);
    break;
  case "multiply":
    console.log(a * b);
    break;
  default:
    console.log("Unknown operation");
}

let today = "Sunday";
switch (today) {
  case "Saturday":
  case "Sunday":
    console.log("Weekend!");
    break;
  default:
    console.log("Weekday");
}
```

Output

42
Weekend!

⚠ Common mistakes

- ✗ Forgetting `break`. JavaScript then "falls through" and runs the next case as well:

JavaScript

```
let n = 1;

switch (n) {
  case 1:
    console.log("one");    // no break here!
  case 2:
    console.log("two");
    break;
}
```

Output

one
two

- ✗ Trying to use ranges such as `case > 5`. A `switch` only matches exact values, so use `if / else if` for ranges.
- ✗ Forgetting `default`. Then nothing happens when no case matches.
- ✗ Forgetting that `switch` uses strict comparison (`===`): the number `1` does not match the text `"1"`.

🔑 Key points

- `switch` compares one value against many exact cases.
- End each case with `break`, unless you want to fall through on purpose.

- `default` handles all the other values.
- Use `if / else if` for ranges and complex conditions.

Practice

- 1 Write a `switch` that turns numbers 1 to 3 into `"Small"`, `"Medium"`, and `"Large"`, with `"Unknown"` as the default.
- 2 Make a traffic light: `"red"` prints `"Stop"`, `"yellow"` prints `"Get ready"`, `"green"` prints `"Go"`.
- 3 Remove one `break` on purpose and see what changes.

5.6 Truthy and Falsy Values

What is it?

When JavaScript needs a `true` or `false` (for example, inside `if ( )`), it turns any value into a boolean. Values that become `false` are **falsy**. Everything else is **truthy**.

The falsy values are: `false`, `0`, `""` (an empty string), `null`, `undefined`, and `NaN`. Every other value is truthy, including `"0"`, `"false"`, and `[]`.

Why do we use it?

It lets you write short checks. `if (name)` means "if `name` has something in it". You will use it very often, especially with user input.

<> Syntax

Syntax pattern

```
if (value) {  
  // runs when the value is truthy  
}
```

✍ Simple example

`Boolean(value)` shows you how JavaScript sees any value:

JavaScript

```
console.log(Boolean("hello"));  
console.log(Boolean(""));   
console.log(Boolean(0));  
console.log(Boolean(42));  
console.log(Boolean(null));  
console.log(Boolean("0"));
```

Output

```
true  
false  
false  
true  
false  
true
```

🌐 Real-world example

A form checks that the user typed a name. It also uses `||` to fill in a default, because `||` picks the first truthy value:

JavaScript

```
let userName = "";

if (userName) {
  console.log("Hello,", userName);
} else {
  console.log("Please enter your name.");
}

let displayName = userName || "Guest";
console.log("Showing:", displayName);
```

Output

```
Please enter your name.
Showing: Guest
```

⚠ Common mistakes

- ✗ Forgetting that `0` is falsy. A score of `0` is a real score, but `if (score)` treats it as "no score":

JavaScript

```
let score = 0;

if (score) {
  console.log("Has a score");
} else {
  console.log("No score yet"); // wrong! 0 is a real score
}

if (score !== null) {
  console.log("Score is", score); // better: check for null on purpose
}
```

Output

```
No score yet  
Score is 0
```

- ✗ Thinking `"false"` (text) is falsy. It is a non-empty string, so it is truthy.
- ✗ Thinking an empty array `[]` is falsy. It is truthy.

🔑 Key points

- Falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`.
- Everything else is truthy.
- `if (value)` is a short way to check that a value exists.
- Be careful with `0` and `""`, which can be valid values.

☰ Practice

- 1 Use `Boolean()` to test `" "` (a space), `[]`, and `NaN`. Predict first.
- 2 Write a check that prints `"Empty"` when a string variable has no text.
- 3 Use `||` to give `nickname` the default value `"Friend"` when the variable is empty.

📌 Mini project: Cinema Ticket Machine

Use `if / else if`, `switch`, and logical operators together. The price depends on age, the day, and whether the customer is a student.

JavaScript

```
let age = 15;
let day = "Saturday";
let isStudent = true;

let price;

// Step 1: base price depends on age
if (age < 5) {
  price = 0;
} else if (age < 12) {
  price = 100;
} else {
  price = 200;
}

// Step 2: weekend extra charge
switch (day) {
  case "Saturday":
  case "Sunday":
    price += 50;
    break;
  default:
    price += 0;
}

// Step 3: student discount (not for free tickets)
if (isStudent && price > 0) {
  price = price - 30;
}

console.log("Age:", age, "| Day:", day, "| Student:", isStudent);
console.log("Ticket price:", price);
```

Output

```
Age: 15 | Day: Saturday | Student: true  
Ticket price: 220
```

Challenge: add a senior discount: customers aged 60 or over pay half of the final price. Then test with the ages 3, 9, 15, and 65.

🔖 Chapter 5 in one minute

- `if / else if / else` chooses one path, and the first true condition wins.
- Nested conditions put one decision inside another, so keep them shallow.
- The ternary condition `? a : b` is a short `if / else` for values.
- `switch` matches exact values, and do not forget `break`.
- Falsy values are `false`, `0`, `""`, `null`, `undefined`, and `NaN`. Everything else is truthy.

▼ Sample answers for Chapter 5

5.1, exercise 1:

JavaScript

```
let marks = 55;  
  
if (marks >= 40) {  
  console.log("Pass");  
} else {  
  console.log("Fail");  
}
```


Output

Pass

5.2, exercise 1:

JavaScript

```
let age = 15;

if (age < 13) {
  console.log("Child");
} else if (age <= 19) {
  console.log("Teenager");
} else {
  console.log("Adult");
}
```

Output

Teenager

5.5, exercise 2:

JavaScript

```
let light = "green";

switch (light) {
  case "red":
    console.log("Stop");
    break;
  case "yellow":
    console.log("Get ready");
    break;
  case "green":
    console.log("Go");
    break;
  default:
    console.log("Broken light");
}
```

Output

Go

5.6, exercise 3:

JavaScript

```
let nickname = "";
console.log(nickname || "Friend");
```

Output

Friend

6

Chapter 6

Loops

Imagine writing "I will not talk in class" one hundred times on a board. You would get tired quickly, but a computer would not. A **loop** repeats a block of code again and again, so you write the instructions once and the computer does the boring part. Loops are how programs count, search, add up, and process lists of things.

What you will learn

- Repeat code a fixed number of times with `for`
- Repeat code until something changes with `while` and `do...while`
- Spot and avoid infinite loops
- Stop a loop early with `break` and skip a step with `continue`
- Go through the items of a list with `for...of`
- Go through the keys of an object with `for...in`

6.1 The for Loop

🕒 What is it?

A `for` **loop** repeats a block of code a set number of times. It keeps a counter, checks the counter each time, and changes the counter after every round. One round of a loop is called an **iteration**.

💡 Why do we use it?

When you already know how many times something must happen (10 times, 100 times, once for every student), `for` is the neatest tool. It keeps the counting in one line.

<> Syntax

Syntax pattern

```
for (start; condition; update) {  
  // code to repeat  
}
```

The three parts inside the brackets are separated by semicolons:

- **start** runs once, before the loop begins (usually `let i = 1`).
 - **condition** is checked before every round. If it is `true`, the block runs. If it is `false`, the loop ends.
 - **update** runs after every round (usually `i++`).
-

✍ Simple example

JavaScript

```
for (let i = 1; i <= 5; i++) {  
  console.log("Round", i);  
}
```

Output

```
Round 1  
Round 2  
Round 3  
Round 4  
Round 5
```

Here is what JavaScript does: it sets `i` to 1, checks `i <= 5`, runs the block, adds 1 to `i`, checks again, and so on. When `i` becomes 6, the condition is `false` and the loop stops.

🌐 Real-world example

A times table. A quiz app prints the table of 7 with one loop:

JavaScript

```
const number = 7;

for (let i = 1; i <= 5; i++) {
  console.log(`${number} x ${i} = ${number * i}`);
}
```

Output

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
```

You can also put a loop **inside** another loop. The inner loop finishes all its rounds for every single round of the outer loop:

JavaScript

```
for (let row = 1; row <= 3; row++) {
  let line = "";
  for (let col = 1; col <= 3; col++) {
    line += row * col + " ";
  }
  console.log(line);
}
```

Output

```
1 2 3
2 4 6
3 6 9
```

⚠ Common mistakes

- ✗ **Off-by-one errors.** Writing `i <= 3` when you meant `i < 3` (or the other way around) gives one round too many or too few. Count carefully.
- ✗ Forgetting the update part (`i++`). The counter never changes, so the loop never ends.
- ✗ Using `const` for the counter. `for (const i = 0; ...; i++)` fails, because `i++` tries to change a constant. Use `let`.
- ✗ Using commas instead of semicolons between the three parts.

JavaScript

```
// Starting at 0 with <= 3 gives FOUR rounds, not three
for (let i = 0; i <= 3; i++) {
  console.log(i);
}
```

Output

```
0
1
2
3
```

🔑 Key points

- A `for` loop has three parts: start, condition, update.
- The condition is checked before every round.
- Use it when you know how many rounds you need.
- Use `let` for the counter, and be careful with `<` and `<=`.

☰ Practice

- 1 Print the numbers 1 to 10 with a `for` loop.
- 2 Print the even numbers from 2 to 20. (Hint: change the update part to `i += 2`.)
- 3 Print a countdown from 5 to 1, then print `"Go!"` after the loop. (Hint: use `i--`.)

6.2 The while Loop

🕒 What is it?

A `while` **loop** repeats a block of code **as long as** a condition is `true`. It does not have a built-in counter. It only asks one question before each round: "Should I go again?"

💡 Why do we use it?

Sometimes you do not know in advance how many rounds you need. You might repeat until a bank balance reaches a goal, until a number gets small enough, or until the user types the right answer.

<> Syntax

Syntax pattern

```
while (condition) {  
    // code to repeat  
}
```

You must change something inside the block so that the condition eventually becomes `false`.

✎ Simple example

JavaScript

```
let cups = 1;  
  
while (cups <= 3) {  
    console.log("Drinking cup number", cups);  
    cups++;  
}  
  
console.log("Done!");
```

Output

```
Drinking cup number 1  
Drinking cup number 2  
Drinking cup number 3  
Done!
```

🌐 Real-world example

A savings goal. You save 500 every month, and you want to buy a phone that costs 3200. How many months will it take? You do not know the answer beforehand, so `while` is perfect:

JavaScript

```
let saved = 0;
let months = 0;
const goal = 3200;

while (saved < goal) {
  saved += 500;
  months++;
}

console.log("Months needed:", months);
console.log("Total saved:", saved);
```

Output

```
Months needed: 7
Total saved: 3500
```

⚠ Common mistakes

- ✗ Forgetting to change the variable in the condition, which creates an infinite loop (see 6.4).
- ✗ Starting with a condition that is already `false`. Then the loop runs zero times.
- ✗ Using `=` instead of `===` in the condition.

🔑 Key points

- `while` repeats as long as its condition is `true`.
- The condition is checked **before** each round.
- Use it when you do not know how many rounds you need.
- Always change something inside the loop that moves it toward the end.

☰ Practice

- 1 Use a `while` loop to print the numbers 1 to 5.
- 2 Start with `let n = 100;` and keep halving it (`n = n / 2`) while `n` is greater than 1. Print `n` each time.
- 3 Change the savings goal example to save 750 per month. How many months now?

6.3 The do...while Loop

🕒 What is it?

A `do...while` **loop** is like `while`, but it checks the condition **after** the block runs. That means the block always runs **at least once**.

💡 Why do we use it?

Some tasks must happen once before you can decide whether to repeat them. A menu must be shown before the user can choose to leave. A password box must appear before it can say "wrong password".

<> Syntax

Syntax pattern

```
do {  
    // code to repeat  
} while (condition);
```

Notice the semicolon at the very end.

✍ Simple example

Compare the two loops when the condition is `false` from the start:

JavaScript

```
let n = 10;

while (n < 5) {
  console.log("while: this never prints");
}

do {
  console.log("do...while: this prints once");
} while (n < 5);
```

Output

```
do...while: this prints once
```

🌐 Real-world example

A PIN screen. The screen must ask at least once. We pretend that the user types the wrong PIN twice and then the right one:

JavaScript

```
let pin = 0;
let tries = 0;

do {
  tries++;
  pin = tries === 3 ? 1234 : 1111;
  console.log("Try", tries, "- typed", pin);
} while (pin !== 1234);

console.log("Unlocked after", tries, "tries");
```

Output

```
Try 1 - typed 1111
Try 2 - typed 1111
Try 3 - typed 1234
Unlocked after 3 tries
```

⚠ Common mistakes

- ✗ Forgetting the semicolon after `while (condition)`.
- ✗ Using `do...while` when the block should not run at all for some values. Then use `while`.
- ✗ Forgetting to change the value that the condition checks.

🔑 Key points

- `do...while` runs the block first and checks the condition afterwards.
- The block always runs at least once.
- Use it for "ask first, then decide" situations.

☰ Practice

- 1 Write a `do...while` loop that prints `"Hello"` three times using a counter.
- 2 Make a `do...while` whose condition is false at the start. How many times does it run?
- 3 In your own words: when would you pick `do...while` over `while`?

6.4 Infinite Loops

🎯 What is it?

An **infinite loop** is a loop that never stops, because its condition never becomes `false`. The program keeps repeating forever and cannot do anything else.

🔗 Why do we use it?

You do not want infinite loops by accident, so you must be able to recognize them. In a browser they freeze the tab. In Node.js they run until you press `Ctrl + C`. A few programs, such as games and servers, run on purpose "for ever", but they always have a way to stop.

<> Syntax

There is no special syntax. An infinite loop is a normal loop with a mistake. Here are the two most common ones (do **not** run these):

JavaScript

```
// Mistake 1: the counter never changes
let i = 1;
while (i <= 3) {
  console.log(i);
}

// Mistake 2: the condition is always true
for (let n = 1; n > 0; n++) {
  console.log(n);
}
```

✍ Simple example

The fix for Mistake 1 is one line. Change the counter so the condition can become `false`:

JavaScript

```
let i = 1;

while (i <= 3) {
  console.log(i);
  i++;
}
```

Output

```
1  
2  
3
```

🌐 Real-world example

A loading screen that never ends. Imagine a game that shows "Loading..." until the files arrive. If the code forgets to check the files, or the check can never become true, the player stares at the screen forever. Programmers protect against this by adding a **safety limit**:

JavaScript

```
let attempts = 0;  
let ready = false;  
  
while (!ready && attempts < 3) {  
  attempts++;  
  console.log("Loading... attempt", attempts);  
}  
  
console.log(ready ? "Ready!" : "Gave up after " + attempts + " attempts");
```

Output

```
Loading... attempt 1  
Loading... attempt 2  
Loading... attempt 3  
Gave up after 3 attempts
```

⚠️ If your browser freezes

If a page freezes because of an infinite loop, close that tab and open the page again. In a terminal running Node.js, press `Ctrl + C`. Before you run a loop, ask yourself: "What makes this stop?"

⚠ Common mistakes

- ✗ Forgetting `i++` (or `i--`) inside a `while` loop.
- ✗ Counting in the wrong direction: `for (let i = 10; i > 0; i++)` counts up, so `i > 0` is always true.
- ✗ Making the exit condition impossible, such as waiting for a number to equal `5.5` while you add whole numbers.

🔑 Key points

- An infinite loop never ends because its condition never becomes `false`.
- Always know what will make your loop stop.
- Check that the counter moves in the right direction.
- A safety limit is a good habit for loops that wait for something.

🔧 Practice

- 1 Find the bug: `let x = 5; while (x > 0) { console.log(x); }`. Then fix it.
- 2 Find the bug: `for (let i = 10; i >= 0; i++) { ... }`. What should the update be?
- 3 Write a `while` loop with a safety limit of 5 attempts.

6.5 break and continue

🎯 What is it?

Two small keywords give you control inside a loop:

- `break` ends the whole loop immediately.

- `continue` skips the rest of the current round and jumps to the next one.

💡 Why do we use it?

`break` saves time when you have already found what you are looking for.

`continue` lets you skip items that you do not want to handle, without wrapping everything in an `if`.

<> Syntax

Syntax pattern

```
for (let i = 1; i <= 10; i++) {  
  if (something) {  
    break;      // leave the loop completely  
  }  
  if (somethingElse) {  
    continue;   // skip to the next round  
  }  
  // rest of the code  
}
```

📝 Simple example

JavaScript

```
for (let i = 1; i <= 10; i++) {  
  if (i === 4) {  
    continue;  
  }  
  if (i === 7) {  
    break;  
  }  
  console.log(i);  
}
```


Output

```
1
2
3
5
6
```

When `i` is 4, `continue` skips the print. When `i` is 7, `break` stops the loop, so 7 to 10 are never printed.

🌐 Real-world example

Finding the first match. A shop wants the first number from 1 upward whose square is bigger than 200. As soon as it finds it, it stops looking:

JavaScript

```
let found = 0;

for (let n = 1; n <= 100; n++) {
  if (n * n > 200) {
    found = n;
    break;
  }
}

console.log("First number:", found, "with square", found * found);
```

Output

```
First number: 15 with square 225
```

Skipping the weekend. Days 6 and 7 are Saturday and Sunday. We skip them with `continue`:

JavaScript

```
for (let day = 1; day <= 7; day++) {  
  if (day >= 6) {  
    continue;  
  }  
  console.log("Day", day, "is a school day");  
}
```

Output

```
Day 1 is a school day  
Day 2 is a school day  
Day 3 is a school day  
Day 4 is a school day  
Day 5 is a school day
```

🔗 A loop that stops on purpose

`while (true)` looks like an infinite loop, but if it has a `break` inside, it is a safe way to say "keep going until I say stop". You will see this pattern in many programs.

⚠️ Common mistakes

- ✗ Using `break` outside of a loop (or a `switch`). It causes a syntax error.
- ✗ Using `continue` in a `while` loop **before** the counter is updated. The counter never moves and the loop never ends.
- ✗ Expecting `break` to stop an outer loop. It only leaves the **innermost** loop that contains it.

🔑 Key points

- `break` leaves the loop completely.
- `continue` skips to the next round.

- Both are usually placed inside an `if`.
- In nested loops, `break` only affects the loop it is in.

Practice

- 1 Print 1 to 20, but stop when you reach a number divisible by 8.
- 2 Print 1 to 15, but skip all multiples of 3.
- 3 Find the first number from 1 upward that is divisible by both 6 and 8. Use `break`.

6.6 for...of

What is it?

A `for...of` **loop** goes through the items of a list, one by one. You do not need a counter. JavaScript hands you each item in turn.

A list is called an **array** and is written with square brackets: `["red", "green", "blue"]`. You will learn arrays properly in Chapter 8. For now, just know that it is a list of values. `for...of` also works on the letters of a string.

Why do we use it?

Most of the time, you want to do something with every item: print each name, add up each price, check each answer. `for...of` says exactly that, and there is no counter to get wrong.

<> Syntax

Syntax pattern

```
for (const item of collection) {  
  // use item here  
}
```

`item` is a new variable name of your choice. It holds one value per round.

✎ Simple example

JavaScript

```
const fruits = ["mango", "banana", "apple"];  
  
for (const fruit of fruits) {  
  console.log("I like", fruit);  
}  
  
for (const letter of "JS") {  
  console.log(letter);  
}
```

Output

```
I like mango  
I like banana  
I like apple  
J  
S
```

🌐 Real-world example

A shopping cart total. The prices of the items in a cart are added up:

JavaScript

```
const prices = [250, 80, 120, 45];
let total = 0;

for (const price of prices) {
  total += price;
}

console.log("Total to pay:", total);
```

Output

Total to pay: 495

⚠ Common mistakes

- ✗ Using `in` instead of `of`. `for...in` gives you the **positions** (0, 1, 2), not the values.
- ✗ Trying to use `for...of` on a plain object. It gives an error, because objects are not lists. Use `for...in` for objects (next section).
- ✗ Changing the list while looping over it. That can skip items.

🔑 Key points

- `for...of` visits every value in an array or string.
- No counter is needed.
- The loop variable holds one item per round.
- Use `const` for the loop variable unless you need to change it.

🔧 Practice

- 1 Make an array of five of your favourite foods and print each one with `for...of`.

- 2 Loop through the string `"hello"` and print each letter in capitals. (Hint: `letter.toUpperCase()`.)
- 3 Add up the numbers in `[4, 8, 15, 16, 23, 42]` using `for...of`.

6.7 for...in

🕒 What is it?

A `for...in` **loop** goes through the **keys** (names) of an object. An **object** stores information as `key: value` pairs inside curly brackets, like `{ name: "Asha", age: 15 }`. Chapter 9 covers objects in full.

💡 Why do we use it?

When you want to look at every piece of information inside an object, `for...in` gives you each key. Then you can use the key to get its value.

<> Syntax

Syntax pattern

```
for (const key in object) {  
  // key is the name, object[key] is the value  
}
```

Use square brackets `object[key]` to read the value, because `key` is a variable.

Simple example

JavaScript

```
const student = { name: "Asha", age: 15, grade: 10 };

for (const key in student) {
  console.log(key + ": " + student[key]);
}
```

Output

```
name: Asha
age: 15
grade: 10
```

Real-world example

A price list. A canteen keeps its prices in an object. The loop prints every item and also adds up the price of one of everything:

JavaScript

```
const menu = { samosa: 20, tea: 15, momo: 60 };
let total = 0;

for (const item in menu) {
  console.log(item, "costs", menu[item]);
  total += menu[item];
}

console.log("One of each:", total);
```

Output

```
samosa costs 20
tea costs 15
momo costs 60
One of each: 95
```

⚠ Common mistakes

- ✗ Writing `student.key` instead of `student[key]`. The dot looks for a property literally named "key", so you get `undefined`.
- ✗ Using `for...in` on an array. The keys of an array are text like `"0"` and `"1"`, which can give surprising results:

JavaScript

```
const nums = [10, 20, 30];
let sum = 0;

for (const i in nums) {
  sum += i;      // adds the position as TEXT, not the value
}

console.log(sum);
```

Output

0012

- ✗ Forgetting the difference: `in` gives keys, `of` gives values.

🔖 of vs in

of gives you the **objects** (the values in a list). **in** gives you the **indexes** or keys (the names). Use `for...of` for arrays and strings, and `for...in` for objects.

🔑 Key points

- `for...in` goes through the keys of an object.
- Use `object[key]` with square brackets to read each value.
- Do not use it on arrays. Use `for...of` instead.

☰ Practice

- 1 Make an object about a book with `title`, `author`, and `pages`. Print every key and value.
- 2 Make an object of three subjects and their marks. Add up all the marks with `for...in`.
- 3 Explain in one sentence why `student.key` does not work inside the loop.

🚩 Mini project: Loop Playground

Three classic loop puzzles in one program: a star triangle (nested loops), FizzBuzz (`for` with conditions), and a sum with `while`.

JavaScript

```
// 1. A triangle of stars
const rows = 5;
for (let i = 1; i <= rows; i++) {
  let line = "";
  for (let j = 1; j <= i; j++) {
    line += "*";
  }
  console.log(line);
}

// 2. FizzBuzz for 1 to 15
console.log("--- FizzBuzz ---");
for (let n = 1; n <= 15; n++) {
  if (n % 15 === 0) {
    console.log("FizzBuzz");
  } else if (n % 3 === 0) {
    console.log("Fizz");
  } else if (n % 5 === 0) {
    console.log("Buzz");
  } else {
    console.log(n);
  }
}

// 3. Add the numbers from 1 to 100 using while
let k = 1;
let sum = 0;
while (k <= 100) {
  sum += k;
  k++;
}
console.log("Sum of 1 to 100:", sum);
```

Output

```
*
**
***
****
*****
--- FizzBuzz ---
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
Sum of 1 to 100: 5050
```

Challenge: make the triangle upside down (5 stars first, then 4, and so on). Then change FizzBuzz to count up to 30.

🔖 Chapter 6 in one minute

- `for` repeats with a counter: `for (let i = 0; i < n; i++)` .
- `while` repeats while a condition is true. `do...while` runs at least once.
- Infinite loops never end. Always know what stops your loop.
- `break` leaves the loop. `continue` skips to the next round.
- `for...of` gives the values of a list. `for...in` gives the keys of an object.

▼ Sample answers for Chapter 6

6.1, exercise 3 (countdown):

JavaScript

```
for (let i = 5; i >= 1; i--) {  
  console.log(i);  
}  
console.log("Go!");
```

Output

```
5  
4  
3  
2  
1  
Go!
```

6.2, exercise 2 (halving):

JavaScript

```
let n = 100;  
  
while (n > 1) {  
  n = n / 2;  
  console.log(n);  
}
```

Output

```
50  
25  
12.5  
6.25  
3.125  
1.5625  
0.78125
```

6.4, exercise 1 (the fix): the loop never changes `x`, so add `x--;` inside the block.

JavaScript

```
let x = 5;  
  
while (x > 0) {  
  console.log(x);  
  x--;  
}
```

Output

```
5  
4  
3  
2  
1
```

6.5, exercise 3 (first number divisible by 6 and 8):

JavaScript

```
for (let n = 1; n <= 100; n++) {  
  if (n % 6 === 0 && n % 8 === 0) {  
    console.log("Found:", n);  
    break;  
  }  
}
```

Output

Found: 24

6.6, exercise 3 (add up the numbers):

JavaScript

```
let total = 0;  
  
for (const num of [4, 8, 15, 16, 23, 42]) {  
  total += num;  
}  
  
console.log(total);
```

Output

108

6.7, exercise 2 (marks total):

JavaScript

```
const marks = { maths: 80, science: 72, english: 65 };  
let total = 0;  
  
for (const subject in marks) {  
  total += marks[subject];  
}  
  
console.log("Total marks:", total);
```

Output

```
Total marks: 217
```

7

Chapter 7

Functions

Think of a juice machine. You put in fruit, press a button, and out comes juice. You do not need to know how the machine works inside, and you can use it as many times as you like. A **function** is that machine for code. You write the steps once, give them a name, and run them whenever you need. Functions are the most important tool for keeping programs short, tidy, and easy to fix.

What you will learn

- Create a function and call it
- Pass information in with parameters and arguments
- Get a result out with `return`
- Understand local and global scope
- Write function expressions and arrow functions
- Use default parameters and rest parameters
- Pass a function into another function (a callback)

7.1 Function Declarations

🕒 What is it?

A **function** is a named block of code that does one job. Creating it is called **declaring** the function. Running it is called **calling** (or invoking) the function.

💡 Why do we use it?

Without functions, you would copy and paste the same code again and again. If you later needed to fix something, you would have to fix it in every copy. With a function, you change it in one place.

<> Syntax

Syntax pattern

```
function functionName() {  
    // code to run  
}  
  
functionName(); // call it
```

Function names follow the same rules as variable names, and they usually start with a verb: `showMenu`, `calculateTotal`, `sayHello`.

✍ Simple example

JavaScript

```
function sayHello() {  
    console.log("Hello, students!");  
}  
  
sayHello();  
sayHello();
```

Output

```
Hello, students!  
Hello, students!
```

Declaring the function does nothing by itself. The code only runs when you call it with `sayHello()`.

🌐 Real-world example

A report header. A school program prints the same header at the top of every report:

JavaScript

```
function printHeader() {  
  console.log("=====");  
  console.log("  Green Valley School");  
  console.log("=====");  
}  
  
printHeader();  
console.log("Report card for Asha");  
printHeader();  
console.log("Report card for Ravi");
```

Output

```
=====  
  Green Valley School  
=====  
Report card for Asha  
=====  
  Green Valley School  
=====  
Report card for Ravi
```

💡 Declarations can be called early

JavaScript reads function declarations before it runs the rest of the code. So you can call a declared function **above** the place where you wrote it:

JavaScript

```
welcome();  
  
function welcome() {  
  console.log("Welcome!");  
}
```

Output

Welcome!

⚠️ Common mistakes

- ✗ Forgetting the round brackets when calling. `sayHello;` does nothing, but `sayHello();` runs the function.
- ✗ Declaring a function and never calling it.
- ✗ Misspelling the name when you call it. `sayhello()` is not the same as `sayHello()`.
- ✗ Giving a function a vague name like `doStuff`. A good name tells you what the function does.

🔑 Key points

- A function is a reusable block of code with a name.
- Declare it with `function name() { }` and call it with `name()`.
- Functions save you from repeating code.
- Use clear verb names for your functions.

☰ Practice

- 1 Write a function `showTitle` that prints your favourite book title. Call it three times.
- 2 Write a function `printLine` that prints 20 dashes (`-`). Use it to draw a line above and below a message.
- 3 Declare a function and call it **before** its declaration. Does it work?

7.2 Parameters and Arguments

🎯 What is it?

Functions become much more useful when they can receive information. A **parameter** is a variable name inside the function's brackets. An **argument** is the real value you give when you call the function.

Think of a parameter as an empty box with a label, and the argument as the thing you put in the box.

💡 Why do we use it?

With parameters, one function can work with different data each time: greet any name, add any two numbers, calculate the area of any rectangle.

<> Syntax

Syntax pattern

```
function functionName(parameter1, parameter2) {  
    // use the parameters here  
}  
  
functionName(argument1, argument2);
```

The arguments are matched to the parameters **by position**: first with first, second with second.

Simple example

JavaScript

```
function greet(name) {  
  console.log("Hello, " + name + "!");  
}  
  
greet("Asha");  
greet("Ravi");
```

Output

```
Hello, Asha!  
Hello, Ravi!
```

Real-world example

A rectangle area helper. A room-planning tool needs the area of many different rooms:

JavaScript

```
function printArea(width, height) {  
  const area = width * height;  
  console.log(`A ${width} x ${height} room has an area of ${area}.`);  
}  
  
printArea(4, 5);  
printArea(3, 3);
```

Output

```
A 4 x 5 room has an area of 20.  
A 3 x 3 room has an area of 9.
```

⚠ Common mistakes

✗ Forgetting an argument. The missing parameter becomes `undefined`:

JavaScript

```
function greet(name) {  
  console.log("Hello, " + name + "!");  
}  
  
greet();
```

Output

```
Hello, undefined!
```

- ✗ Giving arguments in the wrong order. `divide(2, 10)` and `divide(10, 2)` give different answers.
- ✗ Mixing up the two words. **Parameters** are in the definition. **Arguments** are in the call.

🔑 Key points

- Parameters are the names inside the brackets when you declare a function.
- Arguments are the values you pass when you call it.
- They match by position.
- A missing argument makes the parameter `undefined`.

🔧 Practice

- 1 Write `printSquare(n)` that prints the square of `n`. Call it with 4 and 9.
- 2 Write `introduce(name, age)` that prints `"I am ..., and I am ... years old."`.

- 3 What does `introduce(15, "Asha")` print if you swap the arguments by mistake?

7.3 The return Statement

🕒 What is it?

So far, our functions have only printed things. The `return` statement sends a value **back** to the place where the function was called. That value can be stored in a variable or used in another calculation.

🔗 Why do we use it?

Printing is only for people to read. A returned value can be used by the program itself: saved, checked, added, or passed to another function. Most real functions return something.

<> Syntax

Syntax pattern

```
function functionName(parameters) {  
  return value;  
}  
  
const result = functionName(arguments);
```

When JavaScript reaches `return`, the function ends immediately.

Simple example

JavaScript

```
function add(a, b) {  
  return a + b;  
}  
  
const total = add(7, 5);  
console.log(total);  
console.log(add(1, 2) * 10);
```

Output

```
12  
30
```

Real-world example

A price calculator. A shop calculates the final price after a discount, and the result is used in more than one place:

JavaScript

```
function finalPrice(price, quantity, discountPercent) {  
  const subtotal = price * quantity;  
  const discount = subtotal * discountPercent / 100;  
  return subtotal - discount;  
}  
  
const bill = finalPrice(200, 3, 10);  
console.log("You pay:", bill);  
console.log("With delivery:", bill + 50);
```

Output

```
You pay: 540  
With delivery: 590
```


⚠ Common mistakes

✗ Forgetting `return`. The function gives back `undefined`:

JavaScript

```
function double(n) {  
  n * 2;      // no return!  
}  
  
console.log(double(4));
```

Output

undefined

- ✗ Writing code after `return`. It never runs, because the function has already ended.
- ✗ Using `console.log` when you needed `return`. Printing a value does not give it back.

🔑 Key points

- `return` sends a value back to the caller.
- The function stops as soon as it returns.
- A function without `return` gives back `undefined`.
- Return values let you store and reuse results.

🔗 Practice

- 1 Write `multiply(a, b)` that returns the product. Print `multiply(6, 7)`.
- 2 Write `isAdult(age)` that returns `true` when `age` is 18 or more, and `false` otherwise.
- 3 Write `average(a, b, c)` that returns the average of three numbers.

7.4 Scope: Local and Global Variables

🎯 What is it?

Scope means "where a variable can be seen". A variable made **inside** a function is **local**: only that function can see it. A variable made **outside** all functions is **global**: everything can see it.

💡 Why do we use it?

Local variables keep functions independent, so one function cannot accidentally break another. Scope is also the reason for many confusing errors, so it is worth understanding early.

<> Syntax

Syntax pattern

```
const globalVariable = "everyone can see me";

function example() {
  const localVariable = "only this function can see me";
}
```

✍ Simple example

JavaScript

```
const school = "Green Valley";

function showInfo() {
  const room = 12;
  console.log(school, "room", room);
}

showInfo();
console.log(typeof room);
```

Output

```
Green Valley room 12  
undefined
```

The function can read `school` because it is global. But `room` lives only inside the function, so outside it does not exist. (`typeof` gives `"undefined"` for a name that does not exist, instead of an error.)

🌐 Real-world example

A game score. Several functions share one global score:

JavaScript

```
let score = 0;  
  
function addPoints(points) {  
  score += points;  
}  
  
addPoints(10);  
addPoints(5);  
console.log("Score:", score);
```

Output

```
Score: 15
```

⚠️ Common mistakes

- ✗ Trying to use a local variable outside its function. It does not exist there.
- ✗ Creating a local variable with the **same name** as a global one. The local one hides the global one inside the function:

JavaScript

```
let count = 1;

function test() {
  let count = 99;
  console.log("inside:", count);
}

test();
console.log("outside:", count);
```

Output

```
inside: 99
outside: 1
```

- ✗ Using too many global variables. Any function can change them, which makes bugs hard to find.
- ✗ Forgetting that `let` and `const` also stay inside any block `{ }`, such as an `if` or a loop.

🔑 Key points

- Variables made inside a function are local to it.
- Global variables can be seen everywhere.
- A local variable with the same name hides the global one.
- Prefer local variables and pass data in with parameters.

🔧 Practice

- 1 Create a function with a local variable `secret`. Try to print `secret` outside the function using `typeof`.
- 2 Write a global `taxRate` and a function `addTax(price)` that uses it.

- 3 Why is it safer to pass data through parameters than to use many global variables?

7.5 Function Expressions

🎯 What is it?

A **function expression** creates a function and stores it in a variable. In JavaScript, a function is a value, just like a number or a string, so you can put it in a variable.

💡 Why do we use it?

It shows an important idea: functions can be stored, passed around, and returned. You will need this for callbacks (7.9) and for many browser tools later in this book.

<> Syntax

Syntax pattern

```
const functionName = function (parameters) {  
    return value;  
};
```

The function on the right has no name of its own, so it is called an **anonymous function**. Notice the semicolon at the end, because this is a normal `const` statement.

Simple example

JavaScript

```
const square = function (n) {  
  return n * n;  
};  
  
console.log(square(9));  
console.log(typeof square);
```

Output

```
81  
function
```

Real-world example

A validator. A signup form has small checking functions stored in variables:

JavaScript

```
const isValidAge = function (age) {  
  return age >= 13 && age <= 100;  
};  
  
console.log("Age 12:", isValidAge(12));  
console.log("Age 25:", isValidAge(25));
```

Output

```
Age 12: false  
Age 25: true
```

Common mistakes

- ✗ Calling a function expression **before** it is created. Unlike declarations, expressions are not available early:

JavaScript

```
sayHi(); // Error: Cannot access 'sayHi' before initialization

const sayHi = function () {
  console.log("Hi");
};
```

- ✗ Forgetting the semicolon after the closing `}`. It usually still works, but it is a good habit.
- ✗ Forgetting the brackets when calling: `square` is the function itself, `square(9)` runs it.

🔑 Key points

- A function can be stored in a variable.
- A function expression is not available before its line runs.
- Function expressions are usually written with `const`.
- Storing functions in variables makes it possible to pass them around.

☰ Practice

- 1 Write a function expression `cube` that returns a number multiplied by itself twice.
- 2 Store a function in `isEven` that returns `true` for even numbers.
- 3 What does `console.log(square)` (without brackets) show, compared with `console.log(square(3))`?

7.6 Arrow Functions

🎯 What is it?

An **arrow function** is a shorter way to write a function expression. It uses an arrow `=>` instead of the word `function`.

💡 Why do we use it?

Modern JavaScript uses arrow functions everywhere, especially for short helper functions. Less typing means clearer code.

<> Syntax

Syntax pattern

```
const name = (parameters) => {  
  return value;  
};  
  
// Shorter: one expression, no braces, the value is returned automatically  
const name = (parameters) => value;  
  
// One parameter: the brackets are optional  
const name = parameter => value;
```

✍ Simple example

The same function written in three ways:

JavaScript

```
const add1 = function (a, b) {  
  return a + b;  
};  
  
const add2 = (a, b) => {  
  return a + b;  
};  
  
const add3 = (a, b) => a + b;  
  
console.log(add1(2, 3), add2(2, 3), add3(2, 3));
```


Output

5 5 5

🌐 Real-world example

Small helpers for a shop:

JavaScript

```
const addTax = price => price + price / 10;
const isExpensive = price => price > 1000;
const greet = () => console.log("Welcome to the shop!");

greet();
console.log("With tax:", addTax(200));
console.log("Expensive?", isExpensive(1500));
```

Output

```
Welcome to the shop!
With tax: 220
Expensive? true
```

⚠ Common mistakes

- ✗ Forgetting `return` when you use curly braces. With `{ }`, you must write `return` yourself:

JavaScript

```
const half = n => { n / 2 };           // no return: gives undefined
const half2 = n => { return n / 2 };    // correct
const half3 = n => n / 2;               // shortest and correct

console.log(half(10), half2(10), half3(10));
```

Output

```
undefined 5 5
```

- ✗ Writing `=>` the wrong way round. It is an equals sign followed by a greater-than sign: `=>`.
- ✗ Using empty brackets incorrectly. A function with no parameters needs `()`: `() => "hello"`.

🔑 Key points

- An arrow function is a short function expression: `(a, b) => a + b`.
- Without curly braces, the value is returned automatically.
- With curly braces, you must use `return`.
- Use `()` when there are no parameters.

🔧 Practice

- 1 Rewrite `const square = function (n) { return n * n; };` as an arrow function.
- 2 Write an arrow function `isPositive` that returns `true` for numbers above 0.
- 3 Write an arrow function with two parameters that returns the larger of them (use a ternary).

7.7 Default Parameters

🎯 What is it?

A **default parameter** gives a parameter a value to use when no argument is given (or when it is `undefined`).

💡 Why do we use it?

Some values are usually the same, such as a shipping fee or a greeting. Defaults let callers skip them, and they also protect you from `undefined` surprises.

<> Syntax

Syntax pattern

```
function name(parameter = defaultValue) {  
  // ...  
}
```

✍ Simple example

JavaScript

```
function greet(name = "friend") {  
  console.log("Hello, " + name + "!");  
}  
  
greet("Asha");  
greet();
```

Output

```
Hello, Asha!  
Hello, friend!
```

🌐 Real-world example

An online store. Delivery costs 50 unless the caller says otherwise:

JavaScript

```
function totalCost(price, delivery = 50) {  
  return price + delivery;  
}  
  
console.log("Normal:", totalCost(300));  
console.log("Free delivery:", totalCost(300, 0));
```

Output

```
Normal: 350  
Free delivery: 300
```

⚠️ Common mistakes

✗ Thinking that the default is used for every empty-looking value. It is used **only** when the argument is missing or `undefined`:

JavaScript

```
function show(value = "default") {  
  console.log(value);  
}  
  
show(undefined);  
show(null);  
show(0);
```

Output

```
default  
null  
0
```

- ✗ Putting the default parameter first and a normal one after it. Put default parameters at the end.
- ✗ Forgetting that the default is only a fallback. Passing an argument replaces it.

🔑 Key points

- A default parameter is written as `name = value`.
- It is used when the argument is missing or `undefined`.
- `null`, `0`, and `""` are real values, so they do not trigger the default.
- Put default parameters last.

🔧 Practice

- 1 Write `power(base, exponent = 2)` that returns `base ** exponent`. Call it with one and with two arguments.
- 2 Write `welcome(name, city = "Paris")` that prints a message using both.
- 3 What does `show(false)` print in the example above? Why?

7.8 Rest Parameters

🎯 What is it?

A **rest parameter** collects any number of extra arguments into one array. You write it with three dots `...` before the last parameter name.

💡 Why do we use it?

Sometimes you do not know how many values will come: the sum of any number of prices, the average of any number of marks. A rest parameter lets

one function handle them all.

<> Syntax

Syntax pattern

```
function name(...items) {  
  // items is an array of all the arguments  
}
```

✎ Simple example

JavaScript

```
function sum(...numbers) {  
  let total = 0;  
  for (const n of numbers) {  
    total += n;  
  }  
  return total;  
}  
  
console.log(sum(1, 2, 3));  
console.log(sum(10, 20, 30, 40, 50));  
console.log(sum());
```

Output

```
6  
150  
0
```

🌐 Real-world example

An average marks calculator. Students can have any number of subjects:

JavaScript

```
function average(...marks) {  
  let total = 0;  
  for (const m of marks) {  
    total += m;  
  }  
  return total / marks.length;  
}  
  
console.log(average(80, 90, 70));  
console.log(average(60, 100));
```

Output

```
80  
80
```

The same dots can also spread

When you put `...` in a function **call**, it does the opposite job: it **spreads** an array into separate arguments. For example, `sum(...[1, 2, 3])` works like `sum(1, 2, 3)`. You will use this again in the array and Math chapters.

Common mistakes

- ✗ Putting the rest parameter anywhere except last. `function f(...items, last)` is a syntax error.
- ✗ Using more than one rest parameter in the same function.
- ✗ Forgetting that `...numbers` is an **array**. You can loop over it and use `.length`, but it is not a single number.

Key points

- `...name` collects the remaining arguments into an array.

- It must be the last parameter.
- It works with any number of arguments, even none.
- Loop over it with `for...of`.

Practice

- 1 Write `maxOf(...numbers)` that returns the biggest number using a loop.
- 2 Write `joinWords(...words)` that returns all the words joined with a space between them. (Hint: start with `" "` and add.)
- 3 What does `average()` return in the example above when no marks are given? Try it.

7.9 Callback Functions

What is it?

A **callback** is a function that you pass into another function, so that the other function can call it later. Think of leaving your phone number at a shop: "Call me when my order is ready."

Why do we use it?

Callbacks let you say **what to do** without deciding **when** or **how often** it happens. The other function decides that. Callbacks are used for timers, button clicks, loading data, and looping over lists.

<> Syntax

Syntax pattern

```
function doSomething(callback) {  
  // ...  
  callback();  
}  
  
doSomething(function () { /* ... */ });  
doSomething(() => { /* ... */ });
```

🔗 Simple example

JavaScript

```
function repeat(times, action) {  
  for (let i = 1; i <= times; i++) {  
    action(i);  
  }  
}  
  
repeat(3, function (n) {  
  console.log("Round", n);  
});  
  
repeat(2, n => console.log("Arrow round", n));
```

Output

```
Round 1  
Round 2  
Round 3  
Arrow round 1  
Arrow round 2
```

The function `repeat` does not know what the action is. It only knows **when** to call it.

🌐 Real-world example

A timer. `setTimeout` is a built-in function that takes a callback and a delay in milliseconds. After the delay, it calls your function:

JavaScript

```
console.log("Making tea...");

setTimeout(() => {
  console.log("Tea is ready!");
}, 500);

console.log("Meanwhile, I read a book.");
```

Output

```
Making tea...
Meanwhile, I read a book.
Tea is ready!
```

The book line prints before the tea line, because JavaScript does not wait. It schedules the callback and moves on.

⚠️ Common mistakes

❌ **Calling** the function instead of **passing** it. Compare:

JavaScript

```
setTimeout(sayHi, 1000);    // correct: passes the function
setTimeout(sayHi(), 1000);  // wrong: sayHi runs NOW, and its result is passed
```

- ❌ Forgetting that a callback is just a normal function. You can write it inline or give it a name and pass the name.
- ❌ Expecting the code after `setTimeout` to wait. It does not.

🔑 Key points

- A callback is a function passed to another function.
- The other function decides when to run it.
- Pass the function itself (`sayHi`), not its result (`sayHi()`).
- Callbacks are used everywhere: timers, events, and array methods.

🔧 Practice

- 1 Write `runTwice(action)` that calls `action` two times. Pass it a function that prints `"Hi!"`.
- 2 Use `setTimeout` to print `"Time is up!"` after 300 milliseconds.
- 3 Write `calculate(a, b, operation)` where `operation` is a function. Call it with `(x, y) => x + y` and with `(x, y) => x * y`.

📌 Mini project: Temperature Converter

Build a small toolbox of functions with parameters, return values, and an arrow function. Then use them together.

JavaScript

```
const toFahrenheit = celsius => celsius * 9 / 5 + 32;
const toCelsius = fahrenheit => (fahrenheit - 32) * 5 / 9;

function describe(celsius) {
  if (celsius <= 0) {
    return "freezing";
  } else if (celsius < 20) {
    return "cool";
  } else if (celsius < 30) {
    return "warm";
  }
  return "hot";
}

function report(celsius) {
  const f = toFahrenheit(celsius);
  console.log(`${celsius}°C is ${f}°F and feels ${describe(celsius)}.`);
}

report(0);
report(15);
report(25);
report(40);
console.log("212°F is", toCelsius(212), "°C");
```

Output

```
0°C is 32°F and feels freezing.
15°C is 59°F and feels cool.
25°C is 77°F and feels warm.
40°C is 104°F and feels hot.
212°F is 100 °C
```

Challenge: add a `formatTemp(value, unit = "C")` function with a default parameter, and use it inside `report`.

🔖 Chapter 7 in one minute

- A function is a reusable block: declare it once, call it many times.
- Parameters are the names in the definition. Arguments are the values you pass.
- `return` sends a value back and ends the function.
- Local variables live inside a function. Global variables live outside.
- Arrow functions are short function expressions: `(a, b) => a + b`.
- Default parameters give fallback values. Rest parameters collect extras into an array.
- A callback is a function you pass to another function.

▼ Sample answers for Chapter 7

7.1, exercise 2 (`printLine`):

JavaScript

```
function printLine() {  
  console.log("-----");  
}  
  
printLine();  
console.log("Welcome to JavaScript");  
printLine();
```

Output

```
-----  
Welcome to JavaScript  
-----
```

7.2, exercise 2 (introduce):

JavaScript

```
function introduce(name, age) {  
  console.log("I am " + name + ", and I am " + age + " years old.");  
}  
  
introduce("Asha", 15);
```

Output

```
I am Asha, and I am 15 years old.
```

7.3, exercise 2 (isAdult):

JavaScript

```
function isAdult(age) {  
  return age >= 18;  
}  
  
console.log(isAdult(20));  
console.log(isAdult(12));
```

Output

```
true  
false
```

7.6, exercise 3 (larger of two):

JavaScript

```
const larger = (a, b) => (a > b ? a : b);  
  
console.log(larger(4, 9));  
console.log(larger(12, 3));
```

Output

9
12

7.7, exercise 1 (power):

JavaScript

```
function power(base, exponent = 2) {  
  return base ** exponent;  
}  
  
console.log(power(5));  
console.log(power(2, 10));
```

Output

25
1024

7.8, exercise 1 (maxOf):

JavaScript

```
function maxOf(...numbers) {  
  let biggest = numbers[0];  
  for (const n of numbers) {  
    if (n > biggest) {  
      biggest = n;  
    }  
  }  
  return biggest;  
}  
  
console.log(maxOf(3, 17, 8, 12));
```

Output

17

7.9, exercise 3 (calculate):

JavaScript

```
function calculate(a, b, operation) {  
  return operation(a, b);  
}  
  
console.log(calculate(4, 5, (x, y) => x + y));  
console.log(calculate(4, 5, (x, y) => x * y));
```

Output

9
20

1

Project 1

Mini Bank Account

You have learned variables, data types, operators, conditions, loops, and functions. Now you will use **all** of them in one program: a tiny bank that keeps a balance, accepts deposits, allows withdrawals, and refuses the ones that are not allowed. Build it slowly, one step at a time, and run the code after each step.

What you will learn

- Plan a program before writing it
- Keep a shared value in a global variable
- Write small functions that each do one job
- Use `return` values to report success or failure
- Combine `if`, `for...of`, and functions in one program

Step 1: Plan the bank

Before you type any code, decide what the bank must do. Write it as a short list:

1. It starts with a **balance**.
2. A **deposit** adds money.
3. A **withdrawal** removes money, but only if there is enough money.
4. Every action prints a message, so we can follow what happens.
5. We feed it a list of transactions: a positive number is a deposit, a negative number is a withdrawal, and `0` is an invalid entry.

Plan first

Professional programmers plan before they code. A five-line plan like this one saves you from getting lost halfway through.

Step 2: The balance and the deposit function

The balance is a **global** variable, because every function needs to see and change it. A failed-transaction counter goes next to it.

JavaScript

```
let balance = 1000;
let failed = 0;

function deposit(amount) {
  balance += amount;
  console.log(`Deposited ${amount}. Balance: ${balance}`);
}
```

Step 3: The withdraw function

A withdrawal must be checked first. If the amount is bigger than the balance, we refuse. The function **returns** `true` or `false`, so the caller knows what happened.

JavaScript

```
function withdraw(amount) {
  if (amount > balance) {
    console.log(`Cannot withdraw ${amount}. Balance is only ${balance}.`);
    return false;
  }
  balance -= amount;
  console.log(`Withdrew ${amount}. Balance: ${balance}`);
  return true;
}
```

Step 4: One function to handle any transaction

This function reads the number and decides what to do. It also returns `true` or `false`.

JavaScript

```
function processTransaction(amount) {  
  if (amount === 0) {  
    console.log("Invalid: the amount is 0");  
    return false;  
  }  
  if (amount > 0) {  
    deposit(amount);  
    return true;  
  }  
  return withdraw(-amount);  
}
```

Notice `withdraw(-amount)`. A withdrawal arrives as a negative number such as `-200`. The minus sign in front turns it into a positive `200` before we pass it on.

Step 5: Run a list of transactions

Finally, a `for...of` loop feeds every transaction into the bank and counts the failures.

JavaScript

```
const transactions = [500, -200, -5000, 0, 250, -1550];

for (const t of transactions) {
  const ok = processTransaction(t);
  if (!ok) {
    failed++;
  }
}

console.log("---");
console.log("Final balance:", balance);
console.log("Failed transactions:", failed);
```

The complete program

Put all the steps together in one file and run it:

JavaScript

```
let balance = 1000;
let failed = 0;

function deposit(amount) {
  balance += amount;
  console.log(`Deposited ${amount}. Balance: ${balance}`);
}

function withdraw(amount) {
  if (amount > balance) {
    console.log(`Cannot withdraw ${amount}. Balance is only ${balance}.`);
    return false;
  }
  balance -= amount;
  console.log(`Withdrew ${amount}. Balance: ${balance}`);
  return true;
}

function processTransaction(amount) {
  if (amount === 0) {
    console.log("Invalid: the amount is 0");
    return false;
  }
  if (amount > 0) {
    deposit(amount);
    return true;
  }
  return withdraw(-amount);
}

const transactions = [500, -200, -5000, 0, 250, -1550];

for (const t of transactions) {
  const ok = processTransaction(t);
  if (!ok) {
    failed++;
  }
}

console.log("---");
```

```
console.log("Final balance:", balance);
console.log("Failed transactions:", failed);
```

Output

```
Deposited 500. Balance: 1500
Withdrew 200. Balance: 1300
Cannot withdraw 5000. Balance is only 1300.
Invalid: the amount is 0
Deposited 250. Balance: 1550
Withdrew 1550. Balance: 0
---
Final balance: 0
Failed transactions: 2
```

Follow the numbers by hand: $1000 + 500 = 1500$, minus $200 = 1300$. The 5000 withdrawal is refused, and so is the 0. Then $1300 + 250 = 1550$, and the last withdrawal takes everything.

Your turn: challenges

1. **Add interest.** Write an arrow function `addInterest(ratePercent)` that adds interest to the balance and returns the interest earned.
2. **Add an overdraft.** Let the customer go down to `-500` before a withdrawal is refused.
3. **Add a fee.** Charge 10 for every withdrawal (but not for deposits).
4. **Add a transfer.** Write `transfer(amount)` that withdraws money and reports whether it worked.
5. **Change the data.** Try a longer list of transactions with your own numbers, and predict the final balance before you run it.

▼ Hint for challenge 1

The interest is `balance * ratePercent / 100`. Add it to the balance and return it:

JavaScript

```
let balance = 2000;

const addInterest = ratePercent => {
  const interest = balance * ratePercent / 100;
  balance += interest;
  return interest;
};

console.log("Interest earned:", addInterest(5));
console.log("New balance:", balance);
```

Output

```
Interest earned: 100
New balance: 2100
```

🔖 What you practised

- Planning a program in plain words before coding.
- A global variable (`balance`) shared by several functions.
- Small functions that each do one job, and `return` values that report success.
- `if` checks to protect the program from bad input.
- A `for...of` loop that processes a whole list.



Chapter 8

Arrays

A shopping list, a class register, the songs in a playlist: real life is full of lists. Until now, each variable held only one value. An **array** holds a whole list of values in one variable, in order. Arrays are one of the most used tools in JavaScript, and the array methods you will learn here (`map`, `filter`, `reduce`, and friends) will change the way you write code.

What you will learn

- Create arrays and read items by their index
- Add and remove items with `push`, `pop`, `shift`, and `unshift`
- Copy and cut arrays with `slice` and `splice`
- Search with `includes` and `indexOf`, and join items into text
- Loop with `forEach`
- Transform lists with `map`, `filter`, `find`, `some`, `every`, and `reduce`
- Sort arrays correctly

8.1 Creating Arrays and Using Indexes

🕒 What is it?

An **array** is an ordered list of values inside square brackets, separated by commas. Each value is called an **element** or **item**. Each item has a position number called an **index**, and counting starts at **0**, not 1.

🔗 Why do we use it?

Without arrays you would need a separate variable for every item (`student1`, `student2`, `student3`, ...). With an array, you keep them together and can loop over them, search them, and sort them.

<> Syntax

Syntax pattern

```
const arrayName = [item1, item2, item3];

arrayName[0]           // first item
arrayName[1]           // second item
arrayName.length       // how many items
arrayName[0] = value;  // change an item
```

Simple example

JavaScript

```
const fruits = ["mango", "banana", "apple"];

console.log(fruits[0]);
console.log(fruits[2]);
console.log(fruits.length);
console.log(fruits[fruits.length - 1]);

fruits[1] = "orange";
console.log(fruits);
```

Output

```
mango
apple
3
apple
[ 'mango', 'orange', 'apple' ]
```

The last item is always at index `length - 1`. An array can also hold mixed types, such as `["Asha", 15, true]`, but most arrays hold one kind of value.

Real-world example

A playlist. A music app keeps song names in an array and shows the first song and the number of songs:

JavaScript

```
const playlist = ["Sunrise", "River Song", "Night Bus", "Monsoon"];

console.log("Now playing:", playlist[0]);
console.log("Songs in playlist:", playlist.length);
console.log("Last song:", playlist[playlist.length - 1]);
console.log("Song 10:", playlist[9]);
```

Output

```
Now playing: Sunrise  
Songs in playlist: 4  
Last song: Monsoon  
Song 10: undefined
```

⚠ Common mistakes

- ✗ Forgetting that counting starts at 0. In an array of 3 items, the indexes are 0, 1, and 2. There is no item at index 3.
- ✗ Reading an index that does not exist. You get `undefined`, not an error.
- ✗ Thinking that `const` makes the array frozen. `const` only stops you from putting a **different** array in the variable. You can still change the items inside.
- ✗ Using `typeof` to check for an array. It says `"object"`. Use `Array.isArray()` instead:

JavaScript

```
const list = [1, 2, 3];  
  
console.log(typeof list);  
console.log(Array.isArray(list));
```

Output

```
object  
true
```

🔑 Key points

- An array is an ordered list inside `[ ]`.
- The first item has index 0, and the last has index `length - 1`.
- `array.length` tells you how many items there are.

- You can change an item with `array[index] = value`.
- Use `Array.isArray()` to check whether something is an array.

Practice

- 1 Make an array of your five favourite foods. Print the first and the last one.
- 2 Change the second item to something else, then print the whole array.
- 3 What does `console.log(fruits[fruits.length])` print? Why?

8.2 Adding and Removing Items

What is it?

Four methods change the ends of an array:

Method	What it does
<code>push(x)</code>	adds <code>x</code> to the end
<code>pop()</code>	removes the last item and gives it back
<code>unshift(x)</code>	adds <code>x</code> to the start
<code>shift()</code>	removes the first item and gives it back

A **method** is a function that belongs to a value. You write it after a dot:
`array.push(...)`.

Why do we use it?

Lists change over time: people join a queue, songs are added, tasks are finished. These four methods handle almost every everyday change.

<> Syntax

Syntax pattern

```
array.push(item);      // add to end
array.pop();           // remove from end
array.unshift(item);   // add to start
array.shift();         // remove from start
```

✎ Simple example

JavaScript

```
const tasks = ["wash", "cook"];

tasks.push("study");
console.log(tasks);

tasks.unshift("wake up");
console.log(tasks);

const last = tasks.pop();
console.log("Removed:", last);

const first = tasks.shift();
console.log("Removed:", first);

console.log(tasks);
```

Output

```
[ 'wash', 'cook', 'study' ]
[ 'wake up', 'wash', 'cook', 'study' ]
Removed: study
Removed: wake up
[ 'wash', 'cook' ]
```

🌐 Real-world example

A ticket queue. People join at the back (`push`), and the person at the front is served first (`shift`):

JavaScript

```
const queue = [];  
  
queue.push("Asha");  
queue.push("Ravi");  
queue.push("Sita");  
console.log("Waiting:", queue.length);  
  
const served = queue.shift();  
console.log("Now serving:", served);  
console.log("Still waiting:", queue);
```

Output

```
Waiting: 3  
Now serving: Asha  
Still waiting: [ 'Ravi', 'Sita' ]
```

⚠ Common mistakes

- ✗ Mixing up `push` (end) and `unshift` (start), or `pop` (end) and `shift` (start). Remember: **push and pop** work at the **end**, like a stack of plates.
- ✗ Forgetting that these methods **change the original array**.
- ✗ Expecting `push` to give back the array. It gives back the **new length**:

JavaScript

```
const items = ["a", "b"];  
const result = items.push("c");  
  
console.log(result);
```

Output

```
3
```

- ✗ Using `pop()` on an empty array. It gives back `undefined`.

🔑 Key points

- `push` and `pop` work at the end. `unshift` and `shift` work at the start.
- `pop` and `shift` give back the removed item.
- All four change the original array.
- `push` and `unshift` give back the new length.

🔧 Practice

- 1 Start with an empty array. Add three names with `push`, then remove the first one with `shift`.
- 2 Add `"urgent"` to the very start of `["email", "call"]`.
- 3 Use `pop` to remove the last item and print what was removed.

8.3 Copying and Cutting: slice and splice

🎯 What is it?

These two look alike, but they behave very differently:

- `slice(start, end)` makes a **copy** of a part of the array. It does **not** change the original. The item at `end` is **not** included.
- `splice(start, deleteCount, ...newItems)` **changes** the original array. It can remove items, insert items, or replace items.

Memory trick: **splice** is like surgery (it changes the patient), and **slice** is like a photocopy.

💡 Why do we use it?

`slice` is safe when you need a piece of a list but want to keep the whole list.
`splice` is how you remove a cancelled order from the middle of a list or add

an item at an exact position.

<> Syntax

Syntax pattern

```
array.slice(start, end)           // copy from start up to (not
including) end
array.splice(start, count)        // remove count items
array.splice(start, 0, item)      // insert item at start
array.splice(start, count, item)  // replace
```

✎ Simple example

JavaScript

```
const letters = ["a", "b", "c", "d", "e"];

console.log(letters.slice(1, 3));
console.log(letters.slice(-2));
console.log(letters);
```

Output

```
[ 'b', 'c' ]
[ 'd', 'e' ]
[ 'a', 'b', 'c', 'd', 'e' ]
```

`slice(-2)` counts from the end and gives the last two items. The original array is untouched.

Now the same array with `splice`:

JavaScript

```
const letters = ["a", "b", "c", "d", "e"];

const removed = letters.splice(1, 2);
console.log("Removed:", removed);
console.log("Left:", letters);

letters.splice(1, 0, "X");
console.log("After insert:", letters);

letters.splice(0, 1, "Z");
console.log("After replace:", letters);
```

Output

```
Removed: [ 'b', 'c' ]
Left: [ 'a', 'd', 'e' ]
After insert: [ 'a', 'X', 'd', 'e' ]
After replace: [ 'Z', 'X', 'd', 'e' ]
```

Real-world example

A shopping cart. The customer removes one item and the app adds a free gift in the middle:

JavaScript

```
const cart = ["pen", "book", "bag", "bottle"];

cart.splice(2, 1);
console.log("After removing the bag:", cart);

cart.splice(1, 0, "free sticker");
console.log("With gift:", cart);

const firstTwo = cart.slice(0, 2);
console.log("First two:", firstTwo);
```

Output

```
After removing the bag: [ 'pen', 'book', 'bottle' ]  
With gift: [ 'pen', 'free sticker', 'book', 'bottle' ]  
First two: [ 'pen', 'free sticker' ]
```

⚠ Common mistakes

- ✗ Mixing up `slice` and `splice`. Only `splice` changes the original.
- ✗ Forgetting that the `end` index in `slice` is **not** included. `slice(1, 3)` gives the items at index 1 and 2.
- ✗ Forgetting the second argument of `splice`. `splice(1)` removes everything from index 1 to the end.
- ✗ Forgetting that `splice` gives back the **removed** items, not the changed array.

🔑 Key points

- `slice(start, end)` copies a part and leaves the original alone.
- `splice(start, count, ...items)` removes, inserts, or replaces in the original array.
- The `end` index of `slice` is not included.
- Negative numbers in `slice` count from the end.

☰ Practice

- 1 Use `slice` to get the middle three items of `[1, 2, 3, 4, 5]`.
- 2 Use `splice` to remove `"Tuesday"` from `["Mon", "Tuesday", "Wed"]`.
- 3 Use `splice` to insert `"Cherry"` between `"Apple"` and `"Mango"` in `["Apple", "Mango"]`.

8.4 Searching and Joining

🎯 What is it?

Three helpful tools:

- `includes(x)` tells you `true` or `false`: is `x` in the array?
 - `indexOf(x)` gives the index of the first `x`, or `-1` if it is not there.
 - `join(separator)` joins all the items into one string, with the separator between them.
-

💡 Why do we use it?

Programs constantly ask "is this already in the list?" and "where is it?". Joining turns a list into readable text for the screen.

<> Syntax

Syntax pattern

```
array.includes(item)    // true or false
array.indexOf(item)     // index, or -1 when not found
array.join(separator)  // a string
```

✍ Simple example

JavaScript

```
const colors = ["red", "green", "blue"];

console.log(colors.includes("green"));
console.log(colors.includes("pink"));
console.log(colors.indexOf("blue"));
console.log(colors.indexOf("pink"));
console.log(colors.join(" - "));
```

Output

```
true
false
2
-1
red - green - blue
```

🌐 Real-world example

A username check and a tag list. A signup page checks whether a name is taken, and a blog shows its tags as one line:

JavaScript

```
const takenNames = ["asha", "ravi", "sita"];
const wanted = "ravi";

if (takenNames.includes(wanted)) {
  console.log(`Sorry, "${wanted}" is already taken.`);
} else {
  console.log(`"${wanted}" is available!`);
}

const tags = ["js", "beginner", "arrays"];
console.log("Tags: " + tags.join(", "));
```

Output

```
Sorry, "ravi" is already taken.
Tags: js, beginner, arrays
```

⚠️ Common mistakes

- ✗ Using `indexOf` inside an `if` without comparing. The index `0` is falsy, and `-1` is truthy, so the check is backwards:

JavaScript

```
const names = ["asha", "ravi"];

if (names.indexOf("asha")) {
  console.log("found");           // does NOT print, because the index is 0
} else {
  console.log("not found");
}

if (names.indexOf("zoe")) {
  console.log("found");           // prints, because -1 is truthy!
}
```

Output

```
not found
found
```

- ✗ Forgetting that searching is case-sensitive. "Asha" and "asha" are different.
- ✗ Using `indexOf` when you only need a yes or no. Use `includes`.

🔑 Key points

- `includes` answers yes or no.
- `indexOf` gives the position, or `-1` when the item is missing.
- Compare with `-1` (or use `includes`), never test the index directly.
- `join` turns an array into a string.

🔧 Practice

- 1 Check whether "Sunday" is in an array of weekday names. Print `true` or `false`.
- 2 Find the index of "blue" in `["red", "blue", "green"]`.

3 Join `["2026", "09", "20"]` with `" - "` to make a date string.

8.5 Looping with `forEach`

🕒 What is it?

`forEach` runs a function once for every item in the array. You give it a callback (Chapter 7), and it hands over each item, and optionally the item's index.

💡 Why do we use it?

It is a tidy way to do something with every item: print it, show it on the page, or check it. There is no counter to manage.

<> Syntax

Syntax pattern

```
array.forEach((item, index) => {  
  // use item and index here  
});
```

✍ Simple example

JavaScript

```
const names = ["Asha", "Ravi", "Sita"];  
  
names.forEach((name, index) => {  
  console.log(index + ": " + name);  
});
```

Output

```
0: Asha
1: Ravi
2: Sita
```

🌐 Real-world example

A numbered menu. A restaurant shows its dishes with numbers starting from 1:

JavaScript

```
const menu = ["Momo", "Thukpa", "Dal Bhat"];

console.log("Today's menu:");
menu.forEach((dish, i) => {
  console.log(`${i + 1}. ${dish}`);
});
```

Output

```
Today's menu:
1. Momo
2. Thukpa
3. Dal Bhat
```

⚠️ Common mistakes

- ✗ Expecting `forEach` to give back a new array. It always gives back `undefined`. Use `map` when you need a new array.
- ✗ Trying to stop it with `break`. That does not work inside `forEach`. Use a `for...of` loop if you may need to stop early.
- ✗ Forgetting that the callback receives the **item first** and the **index second**.

🔑 Key points

- `forEach` calls your function once for each item.
- The callback receives `(item, index)`.
- It returns nothing (`undefined`).
- Use `for...of` if you need `break` or `continue`.

🔧 Practice

- 1 Use `forEach` to print every number of `[3, 6, 9]` multiplied by 2.
- 2 Print `"Item 1: ..."`, `"Item 2: ..."` for an array of three things.
- 3 Use `forEach` and a variable outside the loop to add up all the numbers in `[5, 10, 15]`.

8.6 Transforming with map

🎯 What is it?

`map` creates a **new array** by running a function on every item. The new array has the same length as the old one, and the original is not changed. Think of a machine on a factory belt that changes every item that passes.

💡 Why do we use it?

Very often you need a changed version of a list: prices with tax added, names in capital letters, temperatures in Fahrenheit. `map` does this in one line.

<> Syntax

Syntax pattern

```
const newArray = array.map((item, index) => {  
  return newValue;  
});  
  
// short form  
const newArray = array.map(item => newValue);
```

✎ Simple example

JavaScript

```
const numbers = [1, 2, 3, 4];  
const doubled = numbers.map(n => n * 2);  
  
console.log(doubled);  
console.log(numbers);
```

Output

```
[ 2, 4, 6, 8 ]  
[ 1, 2, 3, 4 ]
```

🌐 Real-world example

Adding tax and formatting names. A shop adds 10% tax to every price, and a school writes names in capital letters:

JavaScript

```
const prices = [100, 250, 80];  
const withTax = prices.map(p => p + p / 10);  
console.log(withTax);  
  
const students = ["asha", "ravi"];  
const shouty = students.map(name => name.toUpperCase());  
console.log(shouty);
```

Output

```
[ 110, 275, 88 ]  
[ 'ASHA', 'RAVI' ]
```

⚠ Common mistakes

- ✗ Forgetting `return` when you use curly braces. Every item becomes `undefined`:

JavaScript

```
const nums = [1, 2, 3];  
  
const broken = nums.map(n => { n * 2 });  
const fixed = nums.map(n => { return n * 2 });  
  
console.log(broken);  
console.log(fixed);
```

Output

```
[ undefined, undefined, undefined ]  
[ 2, 4, 6 ]
```

- ✗ Using `map` just to print things. That is a job for `forEach`.
- ✗ Forgetting to store the result. `numbers.map(...)` on its own changes nothing. Save it in a variable.

🔑 Key points

- `map` returns a new array of the same length.
- The original array stays the same.
- The callback must **return** the new value for each item.
- Use `map` to transform, and `forEach` to just do something.

☰ Practice

- 1 Use `map` to turn `[1, 2, 3]` into their squares.
- 2 Turn `["a", "b", "c"]` into `["a!", "b!", "c!"]`.
- 3 Given prices in `[100, 200, 300]`, create a new array where each price has a 20% discount.

8.7 Selecting with filter and find

🕒 What is it?

- `filter` creates a **new array** with only the items that pass a test. The test is a function that returns `true` (keep) or `false` (drop).
- `find` gives back the **first** item that passes the test, or `undefined` when nothing passes.

💡 Why do we use it?

Searching and selecting are everyday jobs: show only the products under a price, list only the students who passed, find the first empty seat.

<> Syntax

Syntax pattern

```
const keepers = array.filter(item => condition);  
const firstMatch = array.find(item => condition);
```

Simple example

JavaScript

```
const numbers = [5, 12, 8, 21, 3, 16];

const big = numbers.filter(n => n > 10);
const firstBig = numbers.find(n => n > 10);
const none = numbers.find(n => n > 100);

console.log(big);
console.log(firstBig);
console.log(none);
```

Output

```
[ 12, 21, 16 ]
12
undefined
```

Real-world example

A price filter. A shop shows only the items that fit a budget of 300:

JavaScript

```
const prices = [450, 120, 300, 80, 999, 250];
const affordable = prices.filter(price => price <= 300);

console.log("Within budget:", affordable);
console.log("How many:", affordable.length);
```

Output

```
Within budget: [ 120, 300, 80, 250 ]
How many: 4
```

⚠ Common mistakes

- ✗ Expecting `filter` to give one item. It always gives an **array**, even when only one item matches or none do (`[]`).
- ✗ Using `=` instead of `===` inside the test.
- ✗ Forgetting that `find` returns `undefined` when nothing matches. Check for that before using the result.
- ✗ Forgetting to return a `true` or `false` when using curly braces.

🔑 Key points

- `filter` returns a new array with all the matching items.
- `find` returns the first matching item, or `undefined`.
- The test function should return `true` or `false`.
- The original array is not changed.

🔧 Practice

- 1 Keep only the even numbers of `[1, 2, 3, 4, 5, 6]`.
- 2 From `["apple", "kiwi", "banana", "fig"]`, keep only the words with more than 4 letters.
- 3 Use `find` to get the first number above 50 in `[10, 60, 30, 90]`.

8.8 Checking with some and every

🎯 What is it?

- `some` gives `true` if **at least one** item passes the test.
 - `every` gives `true` only if **all** the items pass the test.
-

💡 Why do we use it?

They answer yes-or-no questions about a whole list: "Did anyone fail?", "Is everything filled in?", "Is there any item in stock?".

<> Syntax

Syntax pattern

```
array.some(item => condition)    // true if any item passes
array.every(item => condition)    // true if all items pass
```

✍ Simple example

JavaScript

```
const marks = [72, 85, 38, 90];

console.log(marks.some(m => m < 40));
console.log(marks.every(m => m >= 40));
console.log(marks.every(m => m >= 30));
```

Output

```
true
false
true
```

🌐 Real-world example

A form check. A signup form is valid only when every field has something typed in:

JavaScript

```
const fields = ["Asha", "asha@example.com", ""];

const allFilled = fields.every(value => value !== "");
const anyEmpty = fields.some(value => value === "");

console.log("All filled?", allFilled);
console.log("Any empty?", anyEmpty);
```

Output

```
All filled? false
Any empty? true
```

⚠️ Common mistakes

- ✗ Mixing them up. **some** = at least one. **every** = all of them.
- ✗ Forgetting the empty-array case: `every` on an empty array is `true` (nothing failed), and `some` on an empty array is `false`:

JavaScript

```
console.log([].every(n => n > 10));
console.log([].some(n => n > 10));
```

Output

```
true
false
```

- ✗ Using `filter` and checking the length when `some` would be simpler and faster.

🔑 Key points

- `some` asks "is there at least one?".
- `every` asks "are they all?".
- Both give `true` or `false`.
- They stop early as soon as the answer is known.

🔧 Practice

- 1 Check whether `[3, 7, 10]` contains at least one even number.
- 2 Check whether all the numbers in `[4, 8, 12]` are even.
- 3 What do you expect `[1, 2, 3].every(n => n > 0)` to give? Try it.

8.9 Combining with reduce

🎯 What is it?

`reduce` goes through an array and combines all the items into **one single value**. It keeps a running result, called the **accumulator**, and updates it with every item. It starts from a value you choose.

💡 Why do we use it?

Adding up a cart, finding the biggest number, counting things: all of these turn a whole list into one answer.

<> Syntax

Syntax pattern

```
const result = array.reduce((accumulator, item) => {  
  return newAccumulator;  
}, startValue);
```

The **start value** comes last, after the function.

✎ Simple example

JavaScript

```
const numbers = [10, 20, 30];  
  
const total = numbers.reduce((sum, n) => sum + n, 0);  
console.log(total);
```

Output

60

Step by step: `sum` starts at 0. Then $0 + 10 = 10$, then $10 + 20 = 30$, then $30 + 30 = 60$.

🌐 Real-world example

A cart total and the highest price:

JavaScript

```
const cart = [250, 80, 120, 45];  
  
const total = cart.reduce((sum, price) => sum + price, 0);  
const highest = cart.reduce((max, price) => (price > max ? price : max),  
  cart[0]);  
  
console.log("Total:", total);  
console.log("Most expensive:", highest);
```

Output

Total: 495
Most expensive: 250

⚠ Common mistakes

- ✗ Forgetting the start value. It can work for numbers, but it fails on an empty array:

JavaScript

```
[].reduce((sum, n) => sum + n);    // TypeError: Reduce of empty array with no  
initial value  
[].reduce((sum, n) => sum + n, 0); // 0 (safe)
```

- ✗ Forgetting to **return** the new accumulator when you use curly braces.
- ✗ Mixing up the order of the parameters. The accumulator always comes **first**, the item second.

🔑 Key points

- `reduce` turns a whole array into a single value.
- The callback receives `(accumulator, item)`.
- Always give a start value as the second argument.
- The callback must return the new accumulator.

🔧 Practice

- 1 Use `reduce` to add up `[5, 10, 15, 20]`.
- 2 Use `reduce` to multiply all the numbers in `[1, 2, 3, 4]` together. What start value should you use?
- 3 Use `reduce` to count how many words in `["a", "bb", "ccc"]` have more than one letter.

8.10 Sorting Arrays

🕒 What is it?

`sort` puts the items of an array in order. `reverse` flips the order. Both **change the original array**.

By default, `sort` treats every item as **text** and sorts alphabetically. That is fine for words, but it goes wrong for numbers. To sort numbers properly, you give `sort` a **compare function**: `(a, b) => a - b` for small to big, and `(a, b) => b - a` for big to small.

🔍 Why do we use it?

Leaderboards, price lists, and name lists all need order.

<> Syntax

Syntax pattern

```
array.sort();           // alphabetical (as text)
array.sort((a, b) => a - b); // numbers, small to big
array.sort((a, b) => b - a); // numbers, big to small
array.reverse();        // flip the order
```

Simple example

JavaScript

```
const names = ["Sita", "Asha", "Ravi"];
names.sort();
console.log(names);

const marks = [72, 95, 38, 60];
marks.sort((a, b) => a - b);
console.log("Low to high:", marks);

marks.sort((a, b) => b - a);
console.log("High to low:", marks);
```

Output

```
[ 'Asha', 'Ravi', 'Sita' ]
Low to high: [ 38, 60, 72, 95 ]
High to low: [ 95, 72, 60, 38 ]
```

Real-world example

A leaderboard. A game keeps the original score order but shows a sorted copy. `slice()` with no arguments makes a copy first:

JavaScript

```
const scores = [40, 90, 15, 70];
const leaderboard = scores.slice().sort((a, b) => b - a);

console.log("Leaderboard:", leaderboard);
console.log("Original:", scores);
```

Output

```
Leaderboard: [ 90, 70, 40, 15 ]
Original: [ 40, 90, 15, 70 ]
```

⚠ Common mistakes

- ✗ Sorting numbers without a compare function. They are sorted as text, so `10` comes before `9`:

JavaScript

```
const nums = [10, 9, 1, 100];  
  
console.log(nums.sort());  
console.log(nums.sort((a, b) => a - b));
```

Output

```
[ 1, 10, 100, 9 ]  
[ 1, 9, 10, 100 ]
```

- ✗ Forgetting that `sort` changes the original array. Copy it first with `slice()` if you need the original order.
- ✗ Writing the compare function as `a > b`. It must return a number (negative, zero, or positive), so use `a - b`.
- ✗ Expecting capital and small letters to sort together. Capital letters come before small letters.

🔑 Key points

- `sort()` alone sorts as text.
- For numbers, use `sort((a, b) => a - b)` or `sort((a, b) => b - a)`.
- `sort` and `reverse` change the original array.
- Copy with `slice()` first if you want to keep the original.

🔧 Practice

- 1 Sort `["mango", "apple", "banana"]` alphabetically.
- 2 Sort `[30, 4, 100, 25]` from biggest to smallest.

- 3 Make a sorted copy of `[3, 1, 2]` and check that the original did not change.

🚩 Mini project: Class Marks Analyzer

A teacher has two arrays: student names and marks. Use `forEach`, `reduce`, `filter`, `every`, `indexOf`, and `sort` to build a report. (In Chapter 9, you will learn a neater way to keep this data together.)

JavaScript

```
const students = ["Asha", "Ravi", "Sita", "Kiran", "Maya"];
const marks = [78, 45, 92, 33, 67];

// 1. A report line for every student
students.forEach((name, i) => {
  const result = marks[i] >= 40 ? "Pass" : "Fail";
  console.log(`${i + 1}. ${name}: ${marks[i]} (${result})`);
});

// 2. Class statistics
const total = marks.reduce((sum, m) => sum + m, 0);
const average = total / marks.length;
const highest = marks.reduce((best, m) => (m > best ? m : best),
marks[0]);
const topStudent = students[marks.indexOf(highest)];
const passed = marks.filter(m => m >= 40);

console.log("---");
console.log("Total:", total);
console.log("Average:", average);
console.log("Top student:", topStudent, "with", highest);
console.log("Passed:", passed.length, "of", marks.length);

// 3. Ranking (a sorted copy keeps the original order)
const ranking = marks.slice().sort((a, b) => b - a);
console.log("Ranking:", ranking);
console.log("Everyone passed?", marks.every(m => m >= 40));
```

Output

```
1. Asha: 78 (Pass)
2. Ravi: 45 (Pass)
3. Sita: 92 (Pass)
4. Kiran: 33 (Fail)
5. Maya: 67 (Pass)
---
Total: 315
Average: 63
Top student: Sita with 92
Passed: 4 of 5
Ranking: [ 92, 78, 67, 45, 33 ]
Everyone passed? false
```

Challenge: add a sixth student, find the lowest mark, and print the names of everyone who scored above the class average.

🔖 Chapter 8 in one minute

- An array is an ordered list in `[ ]`. Indexes start at 0, and `length` counts the items.
- `push` and `pop` work at the end. `unshift` and `shift` work at the start.
- `slice` copies a part. `splice` changes the original.
- `includes` says yes or no. `indexOf` gives a position or `-1`.
- `forEach` does something for each item. `map` makes a new transformed array.
- `filter` keeps matching items. `find` gives the first match.
- `some` asks "at least one?". `every` asks "all of them?". `reduce` combines everything into one value.
- `sort` needs `(a, b) => a - b` for numbers, and it changes the original array.

▼ Sample answers for Chapter 8

8.2, exercise 1 (queue):

JavaScript

```
const people = [];  
  
people.push("Asha");  
people.push("Ravi");  
people.push("Sita");  
people.shift();  
  
console.log(people);
```

Output

```
[ 'Ravi', 'Sita' ]
```

8.3, exercise 1 (slice the middle):

JavaScript

```
const nums = [1, 2, 3, 4, 5];  
  
console.log(nums.slice(1, 4));
```

Output

```
[ 2, 3, 4 ]
```

8.4, exercise 3 (join a date):

JavaScript

```
const parts = ["2026", "09", "20"];  
  
console.log(parts.join("-"));
```

Output

2026-09-20

8.6, exercise 3 (discount):

JavaScript

```
const prices = [100, 200, 300];
const discounted = prices.map(p => p - p * 20 / 100);

console.log(discounted);
```

Output

[80, 160, 240]

8.7, exercise 2 (long words):

JavaScript

```
const words = ["apple", "kiwi", "banana", "fig"];
const longWords = words.filter(word => word.length > 4);

console.log(longWords);
```

Output

['apple', 'banana']

8.9, exercise 2 (multiply everything): the start value must be `1`, because multiplying by 0 would always give 0.

JavaScript

```
const product = [1, 2, 3, 4].reduce((result, n) => result * n, 1);  
  
console.log(product);
```

Output

24

8.10, exercise 2 (big to small):

JavaScript

```
const nums = [30, 4, 100, 25];  
  
nums.sort((a, b) => b - a);  
console.log(nums);
```

Output

[100, 30, 25, 4]

9

Chapter 9

Objects

An array is a list, but some things are not just a list of values. A student has a name, an age, a class, and marks. A product has a title, a price, and a stock count. An **object** groups all the information about one thing together, and gives every piece a name. Objects are everywhere in JavaScript: the page, the browser, and even `console` are objects.

What you will learn

- Create objects and read their properties with dot and bracket notation
- Change, add, delete, and check properties
- Write methods (functions inside objects)
- Understand what `this` means inside a method
- Work with objects inside objects
- Handle lists of objects with `map`, `filter`, `find`, and `sort`
- Use `Object.keys`, `Object.values`, and `Object.entries`

9.1 Creating Objects and Reading Properties

🕒 What is it?

An **object** stores information as **key: value** pairs inside curly brackets `{ }`. Each pair is called a **property**. The key is the property's name, and the value can be anything: a number, a string, an array, even another object.

🔗 Why do we use it?

An object keeps everything about one thing in one place. Compare two loose variables, `studentName` and `studentAge`, with one object `student`. The object is easier to pass around and easier to understand.

<> Syntax

Syntax pattern

```
const objectName = {  
  key1: value1,  
  key2: value2  
};  
  
objectName.key1           // dot notation  
objectName["key1"]        // bracket notation
```

Properties are separated by **commas**. A colon `:` joins each key to its value.

✍ Simple example

JavaScript

```
const student = { name: "Asha", age: 15, isPresent: true };  
  
console.log(student.name);  
console.log(student["age"]);  
console.log(student);
```

Output

```
Asha  
15  
{ name: 'Asha', age: 15, isPresent: true }
```

Dot notation is shorter and used most of the time. **Bracket notation** is needed when the key is stored in a variable, or when it has spaces:

JavaScript

```
const student = { name: "Asha", age: 15, "home town": "Pokhara" };

const wanted = "age";
console.log(student[wanted]);
console.log(student["home town"]);
```

Output

```
15
Pokhara
```

🌐 Real-world example

A product in a shop. The shop shows its details on the screen:

JavaScript

```
const product = {
  id: 101,
  title: "Notebook",
  price: 60,
  inStock: true
};

console.log(`${product.title} costs ${product.price}.`);
console.log("Available:", product.inStock);
console.log("Discount:", product.discount);
```

Output

```
Notebook costs 60.
Available: true
Discount: undefined
```

⚠️ Common mistakes

- ✗ Using a semicolon or forgetting the comma between properties. Use commas.

- ✗ Writing `student.wanted` when you mean the key stored in the variable `wanted`. Use `student[wanted]`.
- ✗ Reading a property that does not exist. You get `undefined`, not an error.
- ✗ Forgetting the quotes on keys that have spaces or dashes. Those keys need `"..."`, and bracket notation to read them.

🔑 Key points

- An object is a collection of `key: value` properties in `{ }`.
- Read a property with `object.key` or `object["key"]`.
- Use brackets when the key is in a variable or has spaces.
- A missing property gives `undefined`.

☰ Practice

- 1 Make an object `book` with `title`, `author`, and `pages`. Print each one.
- 2 Store the text `"pages"` in a variable, and use it to read `book.pages` with brackets.
- 3 Print `book.publisher`. What do you get, and why?

9.2 Changing, Adding, and Deleting Properties

🕒 What is it?

Objects are not fixed. You can **change** a property by assigning a new value, **add** a property by assigning to a new key, and **delete** a property with the `delete` keyword. The `in` operator checks whether a property exists.

💡 Why do we use it?

Information changes: a user gets older, changes their email, or removes a phone number. The object must be able to change with it.

<> Syntax

Syntax pattern

```
object.key = newValue;      // change (or add)
object.newKey = value;      // add
delete object.key;          // remove
"key" in object              // true or false
```

✍ Simple example

JavaScript

```
const student = { name: "Asha", age: 15 };

student.age = 16;
student.city = "Pokhara";
delete student.name;

console.log(student);
console.log("age" in student);
console.log("name" in student);
```

Output

```
{ age: 16, city: 'Pokhara' }
true
false
```

🌐 Real-world example

Updating a profile. A user changes their email and adds a phone number:

JavaScript

```
const profile = { username: "asha01", email: "old@example.com" };

profile.email = "new@example.com";
profile.phone = "9800000000";

console.log(profile.email);
console.log(profile.phone);
```

Output

```
new@example.com
9800000000
```

⚠ Common mistakes

- ✗ Thinking `const` stops you from changing an object. It only stops you from replacing the whole object with a different one. The properties inside can change.
- ✗ Making a typo. `student.nmae = "Ravi"` does not give an error. It quietly adds a **new** property called `nmae`.
- ✗ Thinking that `const copy = original` makes a copy. Both names point to the **same** object:

JavaScript

```
const original = { score: 10 };
const alias = original;
alias.score = 99;
console.log(original.score);

const copy = { ...original };
copy.score = 5;
console.log(original.score, copy.score);
```

Output

```
99
99 5
```

The three dots `{ ...original }` **spread** all the properties of `original` into a brand-new object, which is how you make a real copy.

🔑 Key points

- Assign to a key to change or add a property.
- `delete object.key` removes a property.
- `"key" in object` checks that a key exists.
- `const` objects can still change inside.
- `const b = a` does not copy an object. Use `{ ...a }`.

☰ Practice

- 1 Create `car = { brand: "Toyota", year: 2015 }`. Change the year, add a `color`, and print the object.
- 2 Delete the `brand` and check with `in` that it is gone.
- 3 Make a real copy of `car` with the spread dots, change the copy's `color`, and show that the original did not change.

9.3 Methods

🎯 What is it?

A **method** is a function stored as a property of an object. It describes something the object can **do**. Properties are what an object **has**, and methods are what an object **does**.

💡 Why do we use it?

Grouping actions with data keeps related code together. A `dog` object can have a name and also a `bark` action. You have already used methods without knowing: `console.log(...)` is the `log` method of the `console` object.

<> Syntax

Syntax pattern

```
const objectName = {
  method1: function () {
    // code
  },
  method2() {
    // shorter way to write the same thing
  }
};

objectName.method1(); // call it with brackets
```

Simple example

JavaScript

```
const dog = {
  name: "Bruno",
  bark: function () {
    console.log("Woof!");
  },
  sit() {
    console.log("Bruno sits down.");
  }
};

dog.bark();
dog.sit();
```

Output

```
Woof!
Bruno sits down.
```

Real-world example

A small calculator object keeps its operations together:

JavaScript

```
const calculator = {
  add(a, b) {
    return a + b;
  },
  multiply(a, b) {
    return a * b;
  }
};

console.log(calculator.add(5, 3));
console.log(calculator.multiply(5, 3));
```

Output

8
15

⚡ Objects are everywhere

`console` is an object and `log` is its method. `Math` is an object too, and `Math.round()` is one of its methods. You will meet `Math` in Chapter 11. In Chapter 12, the whole web page becomes an object named `document`.

⚠ Common mistakes

- ✗ Forgetting the brackets when calling a method. `dog.bark` is the function itself (not a call), and `dog.bark()` runs it:

JavaScript

```
const dog = { bark() { console.log("Woof!"); } };  
  
console.log(dog.bark);  
dog.bark();
```

Output

[Function: bark]
Woof!

- ✗ Forgetting the comma between one method and the next one.
- ✗ Writing `dog.bark()` when the method does not exist. That gives an error, because `undefined` is not a function.

🔑 Key points

- A method is a function inside an object.

- Write it as `name() { ... }` or `name: function () { ... }`.
- Call it with brackets: `object.method()`.
- `console.log` and `Math.round` are methods too.

☰ Practice

- 1 Make an object `robot` with a method `greet` that prints `"Beep boop!"`.
- 2 Add a method `add(a, b)` to `robot` that returns the sum.
- 3 Print `robot.greet` without brackets, then call it with brackets. What is the difference?

9.4 The `this` Keyword

🕒 What is it?

Inside a method, `this` means "the object that this method belongs to". It lets a method read and change the object's own properties.

💡 Why do we use it?

Without `this`, a method would have to hard-code the name of its object. With `this`, the same method works correctly for whichever object is using it.

<> Syntax

Syntax pattern

```
const objectName = {
  property: value,
  method() {
    this.property    // reads the object's own property
  }
};
```

Simple example

JavaScript

```
const person = {
  name: "Asha",
  age: 15,
  introduce() {
    console.log(`Hi, I am ${this.name} and I am ${this.age}.`);
  },
  birthday() {
    this.age++;
    console.log(`Happy birthday! Now ${this.age}.`);
  }
};

person.introduce();
person.birthday();
person.introduce();
```

Output

```
Hi, I am Asha and I am 15.
Happy birthday! Now 16.
Hi, I am Asha and I am 16.
```

Real-world example

A bank account object. The balance lives inside the object, and the methods use `this` to reach it:

JavaScript

```
const account = {
  owner: "Asha",
  balance: 1000,
  deposit(amount) {
    this.balance += amount;
    console.log(`${this.owner} deposited ${amount}. Balance: ${this.balance}`);
  },
  withdraw(amount) {
    if (amount > this.balance) {
      console.log("Not enough money.");
      return;
    }
    this.balance -= amount;
    console.log(`${this.owner} withdrew ${amount}. Balance: ${this.balance}`);
  }
};

account.deposit(500);
account.withdraw(200);
account.withdraw(5000);
```

Output

```
Asha deposited 500. Balance: 1500
Asha withdrew 200. Balance: 1300
Not enough money.
```

Compare this with Project 1, where the balance was a global variable. Here, the data and the actions live together in one tidy object.

⚠ Common mistakes

- ✗ Forgetting `this.` in front of a property. Writing just `balance` inside the method looks for a normal variable, and gives an error.
- ✗ Using an arrow function as a method. Arrow functions do **not** get their own `this`, so `this.name` will not find the object's name:

JavaScript

```
const person = {  
  name: "Asha",  
  hello: () => console.log(this.name)  // wrong: this is not the person  
};
```

Use the `hello() { ... }` form for methods.

✗ Thinking `this` always means the same thing. It depends on **how** the function is called. In a method call like `person.introduce()`, it is the object before the dot.

🔑 Key points

- `this` inside a method is the object that owns the method.
- Use `this.property` to read or change the object's own data.
- Write methods with `name() { }`, not with arrow functions.

☰ Practice

- 1 Make a `counter` object with `count: 0` and a method `increment()` that adds 1 to `this.count`. Call it three times and print the count.
- 2 Add a `reset()` method to `counter` that sets the count back to 0.
- 3 Explain in one sentence why an arrow function is a poor choice for an object method.

9.5 Nested Objects

🎯 What is it?

A property's value can itself be an object. This is called a **nested object**. To reach the inner values, chain the dots: `student.address.city`.

💡 Why do we use it?

Real information has layers. A student has an address, and the address has a city and a street. A nested object keeps those layers organised.

<> Syntax

Syntax pattern

```
const outer = {  
  inner: {  
    key: value  
  }  
};  
  
outer.inner.key
```

✍ Simple example

JavaScript

```
const student = {  
  name: "Asha",  
  address: { city: "Pokhara", street: "Lake Road" },  
  marks: { maths: 80, science: 72 }  
};  
  
console.log(student.address.city);  
console.log(student.marks["maths"]);  
  
student.marks.english = 90;  
console.log(student.marks);
```

Output

```
Pokhara
80
{ maths: 80, science: 72, english: 90 }
```

🌐 Real-world example

A user with settings. An app stores preferences inside the user object, and uses `?.` (optional chaining) to read a value that might not exist:

JavaScript

```
const user = {
  name: "Ravi",
  settings: { theme: "dark", language: "en" }
};

console.log(user.settings.theme);
console.log(user.contact?.phone);
console.log(user.contact && user.contact.phone);
```

Output

```
dark
undefined
undefined
```

The `?.` operator means "if the thing before me exists, go on. Otherwise, stop and give `undefined`."

⚠️ Common mistakes

- ✗ Reading a value from something that does not exist. This is one of the most common errors in JavaScript:

JavaScript

```
const user = { name: "Ravi" };

console.log(user.contact.phone);
// TypeError: Cannot read properties of undefined (reading 'phone')
```

`user.contact` is `undefined`, and you cannot look inside `undefined`. Use `?.` or check first.

- ✗ Losing track of the levels. Read from left to right: `student.address.city` means "the student, then the address, then the city".
- ✗ Making objects nested more than two or three levels deep. It becomes hard to read.

🔑 Key points

- A property can hold another object.
- Chain the dots to go deeper: `a.b.c`.
- `?.` avoids errors when a middle part is missing.
- `Cannot read properties of undefined` means an earlier step was `undefined`.

☰ Practice

- 1 Make a `school` object with a nested `principal` object (`name` and `phone`). Print the principal's name.
- 2 Add a `year` property inside `principal`.
- 3 Print `school.library.books` using `?.` so that it does not crash.

9.6 Arrays of Objects

🎯 What is it?

Real programs usually keep **many** objects in one **array**: a list of students, a list of products, a list of messages. All the array methods from Chapter 8 work on them.

💡 Why do we use it?

This is the most common shape of data in real websites and apps. When you read a list from a server, it usually arrives as an array of objects.

<> Syntax

Syntax pattern

```
const list = [
  { key: value },
  { key: value }
];

list[0].key           // the first object's property
list.map(item => item.key)
```

Simple example

JavaScript

```
const students = [  
  { name: "Asha", marks: 80 },  
  { name: "Ravi", marks: 45 },  
  { name: "Sita", marks: 92 }  
];  
  
console.log(students[0].name);  
console.log(students[2].marks);  
  
for (const s of students) {  
  console.log(s.name + " scored " + s.marks);  
}
```

Output

```
Asha  
92  
Asha scored 80  
Ravi scored 45  
Sita scored 92
```

Real-world example

A product list. A shop uses `map`, `filter`, `find`, `reduce`, and `sort` on its products:

JavaScript

```
const products = [
  { id: 1, name: "Pen", price: 10, inStock: true },
  { id: 2, name: "Notebook", price: 60, inStock: false },
  { id: 3, name: "Bag", price: 500, inStock: true },
  { id: 4, name: "Ruler", price: 15, inStock: true }
];

const names = products.map(p => p.name);
const available = products.filter(p => p.inStock).map(p => p.name);
const bag = products.find(p => p.id === 3);
const totalValue = products.reduce((sum, p) => sum + p.price, 0);
const byPrice = products.slice().sort((a, b) => b.price - a.price);

console.log(names);
console.log("In stock:", available);
console.log("Product 3:", bag.name, bag.price);
console.log("Total value:", totalValue);
console.log("Most expensive:", byPrice[0].name);
```

Output

```
[ 'Pen', 'Notebook', 'Bag', 'Ruler' ]
In stock: [ 'Pen', 'Bag', 'Ruler' ]
Product 3: Bag 500
Total value: 585
Most expensive: Bag
```

⚠ Common mistakes

- ✗ Forgetting the dot: `p.price`, not `p`. Each item in the array is a whole object.
- ✗ Returning an object from an arrow function without extra brackets. JavaScript thinks `{` starts a block. Wrap the object in `( )`:

JavaScript

```
const items = [{ name: "Pen", price: 10 }, { name: "Bag", price: 500 }];

const wrong = items.map(i => { name: i.name });
const right = items.map(i => ({ name: i.name, price: i.price * 2 }));

console.log(wrong);
console.log(right);
```

Output

```
[ undefined, undefined ]
[ { name: 'Pen', price: 20 }, { name: 'Bag', price: 1000 } ]
```

- ✗ Forgetting that `find` gives back the **same** object, not a copy. Changing it changes the array's object too.
- ✗ Sorting objects without telling `sort` which property to compare.

🔑 Key points

- An array of objects is the most common data shape.
- Reach a value with `list[index].key`.
- `map`, `filter`, `find`, `reduce`, and `sort` all work with objects.
- Wrap an object in brackets when an arrow function returns it: `x => ({ ... })`.

🔧 Practice

- 1 Make an array of three books (`title` and `pages`). Print the title of the last one.
- 2 Use `filter` to keep the books with more than 200 pages.
- 3 Use `reduce` to add up the pages of all the books.

9.7 Object.keys, Object.values, and Object.entries

🕒 What is it?

Three built-in helpers turn an object into arrays, so you can use all the array tools on it:

- `Object.keys(obj)` gives an array of the keys.
 - `Object.values(obj)` gives an array of the values.
 - `Object.entries(obj)` gives an array of `[key, value]` pairs.
-

💡 Why do we use it?

You often need to count properties, add up all the values, or loop through everything in an object. These helpers make that easy and safer than `for...in`.

<> Syntax

Syntax pattern

```
Object.keys(object)
Object.values(object)
Object.entries(object)
```

✍ Simple example

JavaScript

```
const marks = { maths: 80, science: 72, english: 90 };

console.log(Object.keys(marks));
console.log(Object.values(marks));
console.log(Object.entries(marks));
console.log("Subjects:", Object.keys(marks).length);
```

Output

```
[ 'maths', 'science', 'english' ]  
[ 80, 72, 90 ]  
[ [ 'maths', 80 ], [ 'science', 72 ], [ 'english', 90 ] ]  
Subjects: 3
```

🌐 Real-world example

A report card. Add up the marks with `values`, and print every subject with `entries`. The `[subject, mark]` in the loop **unpacks** each pair into two variables (this is called destructuring):

JavaScript

```
const marks = { maths: 80, science: 72, english: 90 };  
  
const total = Object.values(marks).reduce((sum, m) => sum + m, 0);  
console.log("Total:", total);  
  
for (const [subject, mark] of Object.entries(marks)) {  
  console.log(`${subject}: ${mark}`);  
}
```

Output

```
Total: 242  
maths: 80  
science: 72  
english: 90
```

💡 See an object as text

`JSON.stringify(object)` turns an object into a single line of text. It is handy for looking inside an object: `console.log(JSON.stringify({ name: "Asha", age: 15 }))` prints `{"name": "Asha", "age": 15}`. You will meet JSON properly when you learn how programs share data.

⚠ Common mistakes

- ✗ Using `.length` on an object. Objects have no `length`. Use `Object.keys(obj).length`.
- ✗ Forgetting the capital `O` in `Object`.
- ✗ Expecting `Object.entries` to give an object. It gives an **array of pairs**.

🔑 Key points

- `Object.keys` gives the keys, `Object.values` the values, and `Object.entries` the pairs.
- All three give arrays, so `map`, `filter`, `reduce`, and `for...of` work on them.
- Count the properties with `Object.keys(obj).length`.

🔧 Practice

- 1 Print all the keys of `{ a: 1, b: 2, c: 3 }`.
- 2 Find the sum of the values of `{ a: 1, b: 2, c: 3 }` using `Object.values` and `reduce`.
- 3 Loop over `Object.entries` of a small `prices` object and print `"item costs price"` for each.

🚩 Mini project: Student Report Cards

Chapter 8's marks analyzer used two separate arrays. Now the data is tidier: each student is an object with a nested `marks` object. You will combine `Object.values`, `Object.entries`, `map`, `sort`, and `filter`.

JavaScript

```
const students = [
  { name: "Asha", marks: { maths: 80, science: 72, english: 88 } },
  { name: "Ravi", marks: { maths: 45, science: 36, english: 54 } },
  { name: "Sita", marks: { maths: 95, science: 88, english: 90 } }
];

function bestSubject(marks) {
  let best = "";
  let bestMark = -1;
  for (const [subject, mark] of Object.entries(marks)) {
    if (mark > bestMark) {
      best = subject;
      bestMark = mark;
    }
  }
  return best;
}

function summarize(student) {
  const scores = Object.values(student.marks);
  const total = scores.reduce((sum, s) => sum + s, 0);
  return {
    name: student.name,
    total: total,
    average: total / scores.length,
    best: bestSubject(student.marks),
    passedAll: scores.every(s => s >= 40)
  };
}

const report = students.map(summarize).sort((a, b) => b.average - a.average);

report.forEach((s, i) => {
  console.log(`${i + 1}. ${s.name}: total ${s.total}, average ${s.average}, best subject ${s.best}`);
});

const struggling = report.filter(s => !s.passedAll);
console.log("Needs help:", struggling.map(s => s.name).join(", "));
```

Output

```
1. Sita: total 273, average 91, best subject maths
2. Asha: total 240, average 80, best subject english
3. Ravi: total 135, average 45, best subject english
Needs help: Ravi
```

Challenge: add a `city` property to each student. Then print the names of the students from one city. Finally, add a method `describe()` to each student that returns a sentence with the name and the average.

📖 Chapter 9 in one minute

- An object stores `key: value` properties inside `{ }`.
- Read with `obj.key` or `obj["key"]`. Change, add, and delete properties freely.
- A method is a function inside an object. `this` is the object that owns the method.
- Objects can be nested. Use `?.` when a middle part might be missing.
- An array of objects is the most common shape of real data.
- `Object.keys`, `Object.values`, and `Object.entries` turn an object into arrays.
- `const b = a` does not copy an object. Use `{ ...a }`.

▼ Sample answers for Chapter 9

9.1, exercise 1 (book):

JavaScript

```
const book = { title: "Wings of Fire", author: "A. P. J. Abdul Kalam",  
pages: 180 };  
  
console.log(book.title);  
console.log(book.author);  
console.log(book.pages);
```

Output

```
Wings of Fire  
A. P. J. Abdul Kalam  
180
```

9.2, exercise 3 (a real copy):

JavaScript

```
const car = { brand: "Toyota", year: 2015, color: "white" };  
const copy = { ...car };  
  
copy.color = "red";  
console.log(car.color, copy.color);
```

Output

```
white red
```

9.4, exercise 1 (counter):

JavaScript

```
const counter = {  
  count: 0,  
  increment() {  
    this.count++;  
  }  
};  
  
counter.increment();  
counter.increment();  
counter.increment();  
  
console.log(counter.count);
```

Output

3

9.6, exercise 3 (total pages):

JavaScript

```
const books = [  
  { title: "A", pages: 120 },  
  { title: "B", pages: 250 },  
  { title: "C", pages: 310 }  
];  
  
const totalPages = books.reduce((sum, b) => sum + b.pages, 0);  
console.log(totalPages);
```

Output

680

9.7, exercise 2 (sum of values):

JavaScript

```
const nums = { a: 1, b: 2, c: 3 };  
const sum = Object.values(nums).reduce((total, n) => total + n, 0);  
  
console.log(sum);
```

Output

6

10

Chapter 10

Strings

Almost everything you see on a website is text: names, messages, titles, search words, passwords. In JavaScript, text is stored in **strings**, and strings come with a big toolbox of built-in methods. In this chapter you will learn how to measure text, clean it, search inside it, cut pieces out of it, and change it. These skills are used in nearly every real project.

What you will learn

- Find the length of a string and read single characters
- Write special characters with escape sequences and multi-line text
- Change case and remove extra spaces
- Search with `includes`, `startsWith`, `endsWith`, and `indexOf`
- Cut out parts with `slice` and `substring`
- Replace text with `replace` and `replaceAll`
- Turn text into arrays and back with `split` and `join`

10.1 String Length and Characters

🕒 What is it?

A string is a sequence of **characters** (letters, digits, spaces, and symbols). Like an array, each character has an **index** that starts at 0. The `length` property tells you how many characters the string has.

🔗 Why do we use it?

You need the length to check a password, limit a username, or find the last letter. Reading single characters lets you look at the first letter, or go through a word one letter at a time.

<> Syntax

Syntax pattern

```
string.length           // number of characters
string[0]               // first character
string[string.length - 1] // last character
string.charAt(index)    // another way to read one character
```

📝 Simple example

JavaScript

```
const word = "JavaScript";

console.log(word.length);
console.log(word[0]);
console.log(word[4]);
console.log(word[word.length - 1]);
console.log(word.charAt(4));
```

Output

```
10
J
S
t
S
```

🌐 Real-world example

A username rule. A signup page accepts usernames that are 3 to 12 characters long:

JavaScript

```
const username = "asha_01";

if (username.length < 3 || username.length > 12) {
  console.log("Username must have 3 to 12 characters.");
} else {
  console.log(`"${username}" is OK. It starts with "${username[0]}".`);
}
```

Output

```
"asha_01" is OK. It starts with "a".
```

⚠ Common mistakes

- ✗ Writing `word.length()` with brackets. `length` is a **property**, not a method, so it has no brackets.
- ✗ Forgetting that spaces count as characters. `"a b"` has a length of 3.
- ✗ Forgetting that strings cannot be changed by index. The line below is ignored:

JavaScript

```
let word = "cat";
word[0] = "b";

console.log(word);
```

Output

```
cat
```

To get a different string, you must build a **new** one and store it.

🔑 Key points

- `string.length` gives the number of characters (no brackets).

- Characters have indexes starting at 0.
- The last character is at index `length - 1`.
- Strings cannot be changed in place. Methods return **new** strings.

☰ Practice

- 1 Print the length of your full name, including the space.
- 2 Print the first and the last letter of the word `"programming"`.
- 3 Write a check that prints `"Too short"` when a password has fewer than 8 characters.

10.2 Special Characters and Multi-line Text

🕒 What is it?

Some characters are hard to type inside quotes: a quote mark, a new line, or a backslash. You write them with an **escape sequence**: a backslash `\` followed by a letter or symbol. Backticks (template literals) can also hold text over several lines.

Escape	Meaning
<code>\n</code>	new line
<code>\t</code>	tab (a wide space)
<code>\"</code>	a double quote
<code>\'</code>	a single quote
<code>\\</code>	a backslash

🔗 Why do we use it?

Without escapes, a quote inside a string would end the string too early. Escapes also let you print messages over more than one line.

<> Syntax

Syntax pattern

```
"She said \"Hello\""    // quotes inside quotes
"Line 1\nLine 2"        // new line
`Line 1
Line 2`                // multi-line with backticks
```

✍ Simple example

JavaScript

```
console.log("She said \"Hello\"");
console.log('It\'s a nice day');
console.log("First line\nSecond line");
console.log("C:\\Users\\Asha");
```

Output

```
She said "Hello"
It's a nice day
First line
Second line
C:\Users\Asha
```

🌐 Real-world example

An order message. Template literals keep line breaks exactly as you type them, and can hold values with `${ }`:

JavaScript

```
const name = "Asha";  
const total = 540;  
  
const message = `Dear ${name},  
Your order is ready.  
Total to pay: ${total}  
Thank you!`;   
  
console.log(message);
```

Output

```
Dear Asha,  
Your order is ready.  
Total to pay: 540  
Thank you!
```

⚠ Common mistakes

- ✗ Using the same kind of quote inside the string without escaping it. This is a syntax error:

JavaScript

```
const broken = "She said "Hello"";    // the string ends after "She said "
```

Use `\`, or choose a different outer quote, such as `'She said "Hello"'`.

- ✗ Writing `/n` (forward slash) instead of `\n` (backslash). The text `/n` is printed as it is.
- ✗ Using a single backslash in a path: `"C:\Users"` does not work as you expect. Write `"C:\\Users"`.

🔑 Key points

- A backslash starts an escape sequence: `\n`, `\t`, `\"`, `\'`, `\\`.

- `\n` makes a new line.
- Backtick strings can span several lines and hold `${ }` values.
- To use a quote inside a string, escape it or use a different outer quote.

Practice

- 1 Print `He said "JavaScript is fun"` with the double quotes included.
- 2 Print three lines using a single string with `\n`.
- 3 Write a multi-line greeting with backticks that includes your name in `${ }`.

10.3 Changing Case and Trimming

What is it?

- `toUpperCase()` makes all letters capitals. `toLowerCase()` makes them all small.
- `trim()` removes spaces (and new lines) from the **start and end** of a string. Spaces in the middle stay.

Why do we use it?

Users type in every way: `ASHA` , `asha` , `Asha` . To compare or store text properly, you first clean it into one standard form.

<> Syntax

Syntax pattern

```
string.toUpperCase()  
string.toLowerCase()  
string.trim()
```

📝 Simple example

JavaScript

```
const name = "  Asha Sharma  ";  
  
console.log "[" + name + " ]";  
console.log "[" + name.trim() + " ]";  
console.log(name.trim().toUpperCase());  
console.log(name.trim().toLowerCase());
```

Output

```
[  Asha Sharma  ]  
[Asha Sharma]  
ASHA SHARMA  
asha sharma
```

You can chain methods with dots. Each one works on the result of the one before it.

🌐 Real-world example

A login check. A user types their email with capital letters and extra spaces. The program cleans it before comparing:

JavaScript

```
const typed = "  Asha@Example.COM ";  
const saved = "asha@example.com";  
  
console.log("Raw match:", typed === saved);  
console.log("Cleaned match:", typed.trim().toLowerCase() === saved);
```

Output

```
Raw match: false  
Cleaned match: true
```

⚠ Common mistakes

- ✗ Forgetting that these methods **return a new string** and do not change the old one:

JavaScript

```
let city = "pokhara";  
  
city.toUpperCase();  
console.log(city);  
  
city = city.toUpperCase();  
console.log(city);
```

Output

```
pokhara  
POKHARA
```

- ✗ Expecting `trim()` to remove the spaces **inside** the text. It only cleans the two ends.
- ✗ Forgetting the brackets: `name.trim` is the function itself, and `name.trim()` runs it.

🔑 Key points

- `toUpperCase()` and `toLowerCase()` change the case.
- `trim()` removes spaces from both ends.
- Methods return a new string. Store it in a variable to keep it.
- Chain methods: `text.trim().toLowerCase()` .

🔗 Practice

- 1 Turn `"hello world"` into `"HELLO WORLD"` .
- 2 Clean `" Kiaan "` so that it has no spaces, and print its length.
- 3 Compare `"YES"` and `"yes"` in a way that treats them as equal.

10.4 Searching Inside Strings

🕒 What is it?

Four methods look for text inside a string:

- `includes(text)` : does it contain this text? (`true` or `false`)
- `startsWith(text)` : does it begin with this text?
- `endsWith(text)` : does it end with this text?
- `indexOf(text)` : at which index does it first appear? (`-1` if it does not.)

💡 Why do we use it?

Searching is behind search boxes, spam filters, form checks, and file-type checks.

<> Syntax

Syntax pattern

```
string.includes(text)
string.startsWith(text)
string.endsWith(text)
string.indexOf(text)
```

🔗 Simple example

JavaScript

```
const sentence = "JavaScript is fun";

console.log(sentence.includes("Script"));
console.log(sentence.includes("script"));
console.log(sentence.startsWith("Java"));
console.log(sentence.endsWith("fun"));
console.log(sentence.indexOf("is"));
console.log(sentence.indexOf("xyz"));
```

Output

```
true
false
true
true
11
-1
```

Notice that `"script"` (small `s`) is not found: searching is **case-sensitive**.

🌐 Real-world example

File types and a simple email check.

JavaScript

```
const files = ["notes.pdf", "photo.jpg", "report.pdf", "song.mp3"];

for (const file of files) {
  if (file.endsWith(".pdf")) {
    console.log(file, "is a PDF document");
  }
}

const email = "asha@example.com";
const looksValid = email.includes("@") && email.endsWith(".com");
console.log("Email looks valid:", looksValid);
```

Output

```
notes.pdf is a PDF document
report.pdf is a PDF document
Email looks valid: true
```

⚠ Common mistakes

- ✗ Forgetting that searches are case-sensitive. Use `toLowerCase()` on both sides when case should not matter.
- ✗ Testing `indexOf` directly in an `if`. The index `0` is falsy, and `-1` is truthy. Compare with `-1`, or use `includes`.
- ✗ Thinking that this simple email check is enough. Real email checking needs more rules.

🔑 Key points

- `includes`, `startsWith`, and `endsWith` give `true` or `false`.
- `indexOf` gives a position, or `-1` when the text is missing.
- All of them are case-sensitive.
- Combine them with `&&` and `||` for real checks.

☰ Practice

- 1 Check whether `"I love coding"` contains the word `"love"`.
- 2 Check whether a phone number string starts with `"98"`.
- 3 Find the position of the `@` in `"asha@example.com"`.

10.5 Extracting Parts: slice and substring

🕒 What is it?

`slice(start, end)` copies a part of a string, from `start` up to (but **not** including) `end`. If you leave out `end`, it goes to the end of the string.

`substring(start, end)` works almost the same way.

💡 Why do we use it?

Cutting pieces out of text is how you get initials, the domain of an email address, the last digits of a number, or the first 100 letters of a long article.

<> Syntax

Syntax pattern

```
string.slice(start, end)
string.slice(start)      // to the end
string.slice(-3)         // the last 3 characters
string.substring(start, end)
```

Simple example

JavaScript

```
const word = "JavaScript";

console.log(word.slice(0, 4));
console.log(word.slice(4));
console.log(word.slice(-6));
console.log(word.substring(0, 4));
```

Output

```
Java
Script
Script
Java
```

Real-world example

Initials, domain, and a hidden number.

JavaScript

```
const first = "Asha";
const last = "Sharma";
console.log("Initials:", first[0] + last[0]);

const email = "asha@example.com";
const at = email.indexOf("@");
console.log("Domain:", email.slice(at + 1));

const id = "1234567890";
console.log("Hidden ID:", "*****" + id.slice(-4));

const city = "pokhara";
console.log("Capitalized:", city[0].toUpperCase() + city.slice(1));
```

Output

```
Initials: AS
Domain: example.com
Hidden ID: *****7890
Capitalized: Pokhara
```

⚠ Common mistakes

- ✗ Forgetting that the `end` index is **not** included. `slice(0, 4)` gives 4 characters: indexes 0, 1, 2, and 3.
- ✗ Forgetting the `+ 1` when you start after a found character, so the `@` ends up in your result.
- ✗ Using negative numbers with `substring`. It treats them as 0. Use `slice` for that.
- ✗ Expecting the original string to change.

🔑 Key points

- `slice(start, end)` copies a part. The end is not included.
- `slice(start)` goes to the end. `slice(-n)` takes the last `n` characters.
- Combine `indexOf` and `slice` to cut around a known character.
- The original string is never changed.

🔧 Practice

- 1 Print the first 3 letters of `"Wednesday"`.
- 2 Print the last 4 letters of `"Wednesday"`.
- 3 From `"asha@example.com"`, print only the part before the `@`.

10.6 Replacing Text

🎯 What is it?

- `replace(old, new)` replaces the **first** match of `old` with `new`.
- `replaceAll(old, new)` replaces **every** match.

Both give back a new string.

💡 Why do we use it?

Cleaning and changing text is a daily job: hide bad words, turn spaces into dashes for a web address, or fix a spelling everywhere.

<> Syntax

Syntax pattern

```
string.replace(oldText, newText)
string.replaceAll(oldText, newText)
```

✍ Simple example

JavaScript

```
const text = "I like cats. Cats are cute. cats!";

console.log(text.replace("cats", "dogs"));
console.log(text.replaceAll("cats", "dogs"));
```

Output

```
I like dogs. Cats are cute. cats!
I like dogs. Cats are cute. dogs!
```

The middle word `Cats` (with a capital `C`) is not changed, because matching is case-sensitive.

🌐 Real-world example

A web address slug. A blog turns a title into a clean address, and a chat app hides a rude word:

JavaScript

```
const title = "My First Blog Post";
const slug = title.toLowerCase().replaceAll(" ", "-");
console.log("/blog/" + slug);

const message = "That was a stupid idea";
console.log(message.replace("stupid", "*****"));
```

Output

```
/blog/my-first-blog-post
That was a ***** idea
```

⚠️ Common mistakes

- ✗ Using `replace` and expecting every match to change. Only the first one does. Use `replaceAll`.
- ✗ Forgetting that the original string stays the same. Save the result.
- ✗ Forgetting about capital letters. `"Cats"` and `"cats"` are different.

🔑 Key points

- `replace` changes the first match. `replaceAll` changes every match.
- Both return a new string.
- Matching is case-sensitive.
- Chain them with `toLowerCase()` and `trim()` to clean text.

☰ Practice

- 1 Replace "red" with "blue" in "red car, red bike", first with `replace`, then with `replaceAll`.
- 2 Turn "2026-09-20" into "2026/09/20".
- 3 Make a slug from "Learn JavaScript Fast".

10.7 Splitting and Joining

🕒 What is it?

`split(separator)` cuts a string into pieces and gives you an **array** of the pieces. The separator is the text where you want to cut. `join` (from Chapter 8) does the opposite.

💡 Why do we use it?

Data often arrives as one long string: a list separated by commas, a sentence made of words. Splitting lets you count, sort, and change the pieces. Then `join` puts them back together.

<> Syntax

Syntax pattern

```
string.split(separator)    // string to array
array.join(separator)      // array to string
```

Simple example

JavaScript

```
const csv = "Asha,15,Pokhara";

console.log(csv.split(","));
console.log("one two three".split(" "));
console.log("hey".split(""));
```

Output

```
[ 'Asha', '15', 'Pokhara' ]
[ 'one', 'two', 'three' ]
[ 'h', 'e', 'y' ]
```

Real-world example

Word count, title case, and a reversed word:

JavaScript

```
const sentence = "learning javascript is fun";
const words = sentence.split(" ");

console.log("Words:", words.length);

const title = words.map(w => w[0].toUpperCase() + w.slice(1)).join(" ");
console.log(title);

const reversed = "stressed".split("").reverse().join("");
console.log(reversed);
```

Output

```
Words: 4
Learning Javascript Is Fun
desserts
```

💡 Three more handy tools

`padStart` fills the start of a string until it has a given length (good for clocks and dates). `repeat` copies a string many times. `localeCompare` sorts words correctly, even with mixed capital letters:

JavaScript

```
console.log("7".padStart(2, "0") + ":05");  
console.log("=".repeat(10));  
console.log(["Zoya", "asha", "Ravi"].sort((a, b) => a.localeCompare(b)));
```

Output

```
07:05  
=====  
[ 'asha', 'Ravi', 'Zoya' ]
```

⚠ Common mistakes

- ✗ Forgetting the separator. `"a b".split()` gives an array with the whole string inside: `[ 'a b' ]`.
- ✗ Splitting on a single space when there are double spaces. You get empty strings in the array:

JavaScript

```
console.log("a  b".split(" "));
```

Output

```
[ 'a', '', 'b' ]
```

- ✗ Forgetting that the pieces are **strings**, even if they look like numbers. `"15" + 1` is `"151"`. Convert them first (Chapter 11).

🔑 Key points

- `split(separator)` turns a string into an array.
- `split("")` gives every character separately.
- `join(separator)` turns an array back into a string.
- The pieces from `split` are always strings.

🔗 Practice

- 1 Split `"red,green,blue"` and print how many colours there are.
- 2 Reverse the words (not the letters) of `"one two three"`.
- 3 Use `split`, `map`, and `join` to write every word of `"hello big world"` in capital letters.

📄 Mini project: Text Toolkit

A tiny text analyzer that cleans a sentence and reports facts about it. It uses `trim`, `split`, `map`, `slice`, `replaceAll`, `includes`, and `join` together.

JavaScript

```
const raw = " the quick brown fox jumps over the lazy dog ";

const clean = raw.trim();
const words = clean.split(" ");

console.log("Characters:", clean.length);
console.log("Words:", words.length);

const title = words.map(w => w[0].toUpperCase() + w.slice(1)).join(" ");
console.log("Title Case:", title);

const longest = words.reduce((best, w) => (w.length > best.length ? w : best), "");
console.log("Longest word:", longest);

console.log("Slug:", clean.replaceAll(" ", "-"));

let vowels = 0;
for (const ch of clean.toLowerCase()) {
  if ("aeiou".includes(ch)) {
    vowels++;
  }
}
console.log("Vowels:", vowels);

console.log("Reversed:", clean.split("").reverse().join(""));
```

Output

```
Characters: 43
Words: 9
Title Case: The Quick Brown Fox Jumps Over The Lazy Dog
Longest word: quick
Slug: the-quick-brown-fox-jumps-over-the-lazy-dog
Vowels: 11
Reversed: god yzal eht revo spmuj xof nworb kciuq eht
```

Challenge: add a check that tells you whether the text contains every letter of the alphabet (a "pangram"). Hint: loop over

`"abcdefghijklmnopqrstuvwxyz"` and use `includes`.

📖 Chapter 10 in one minute

- `length` counts characters. `string[0]` reads one. Strings cannot be changed in place.
- Escape sequences (`\n` , `\"` , `\\`) write special characters. Backticks allow many lines.
- `toUpperCase` , `toLowerCase` , and `trim` clean up text. They return new strings.
- `includes` , `startsWith` , `endsWith` , and `indexOf` search. Searches are case-sensitive.
- `slice(start, end)` cuts a part. The end index is not included.
- `replace` changes the first match, `replaceAll` changes all of them.
- `split` turns a string into an array, and `join` turns it back.

▼ Sample answers for Chapter 10

10.1, exercise 2 (first and last letter):

JavaScript

```
const word = "programming";  
  
console.log(word[0]);  
console.log(word[word.length - 1]);
```

Output

```
p  
g
```

10.3, exercise 3 (case-insensitive compare):

JavaScript

```
console.log("YES".toLowerCase() === "yes".toLowerCase());
```

Output

```
true
```

10.5, exercise 3 (before the @):

JavaScript

```
const email = "asha@example.com";  
  
console.log(email.slice(0, email.indexOf("@")));
```

Output

```
asha
```

10.6, exercise 1 (replace vs replaceAll):

JavaScript

```
const text = "red car, red bike";

console.log(text.replace("red", "blue"));
console.log(text.replaceAll("red", "blue"));
```

Output

```
blue car, red bike
blue car, blue bike
```

10.7, exercise 2 (reverse the words):

JavaScript

```
const sentence = "one two three";

console.log(sentence.split(" ").reverse().join(" "));
```

Output

```
three two one
```

10.7, exercise 3 (capital letters):

JavaScript

```
const shout = "hello big world".split(" ").map(w => w.toUpperCase()).join(" ");

console.log(shout);
```

Output

```
HELLO BIG WORLD
```

11

Chapter 11

Numbers and Math

Computers are great at maths, but JavaScript has a few surprises. Text from a form looks like a number but behaves like text. Some calculations give a strange result called `NaN`. Decimals are not always exact. In this chapter you will learn to convert text into numbers, catch bad numbers, round results, use the built-in `Math` object, and make random numbers for games and quizzes.

What you will learn

- Convert text into numbers with `Number`, `parseInt`, and `parseFloat`
- Understand `NaN` and check for it correctly
- Round numbers with `Math.round`, `Math.floor`, `Math.ceil`, and `Math.trunc`
- Use `Math.max`, `Math.min`, `Math.abs`, `Math.sqrt`, and `Math.PI`
- Create random numbers and random choices
- Format numbers with `toFixed` and `toLocaleString`, and understand why `0.1 + 0.2` is odd

11.1 Converting Text to Numbers

🕒 What is it?

Whatever a user types into a page arrives as **text**, even when it looks like a number. To do maths with it, you must convert it. JavaScript gives you three tools:

- `Number(text)` converts the whole text into a number.
- `parseInt(text)` reads a whole number from the **start** of the text and ignores the rest.
- `parseFloat(text)` reads a number with decimals from the start of the text.

🔗 Why do we use it?

Forms, prompts, and data files give you text. Without converting it, `"10" + "20"` becomes `"1020"`, not `30`.

<> Syntax

Syntax pattern

```
Number("42")           // 42
parseInt("42px")        // 42
parseFloat("3.14abc")   // 3.14
```

📝 Simple example

JavaScript

```
console.log(Number("42"));
console.log(Number("3.14"));
console.log(Number(" 7 "));
console.log(Number("abc"));
console.log(parseInt("42px"));
console.log(parseFloat("3.14abc"));
console.log(parseInt("3.99"));
```

Output

```
42
3.14
7
NaN
42
3.14
3
```

`Number` is strict: any letters make the result `NaN`. `parseInt` and `parseFloat` are gentle: they read as many characters as they can, and stop at the first wrong one. Note that `parseInt("3.99")` gives `3`. It cuts the decimals off, and does **not** round.

Real-world example

A form that adds two numbers. The two boxes give text, so the program must convert before adding:

JavaScript

```
const boxA = "10";
const boxB = "20";

console.log("Without converting:", boxA + boxB);
console.log("After converting:", Number(boxA) + Number(boxB));
```

Output

```
Without converting: 1020
After converting: 30
```

Common mistakes

- ✗ Adding text values with `+`. It **joins** them instead of adding them.
- ✗ Forgetting that an empty string becomes `0`, not an error:

JavaScript

```
console.log(Number(""));
console.log(Number("  "));
console.log(Number(true));
```

Output

```
0
0
1
```

- ✗ Using `parseInt` when the number has decimals. It throws the decimals away.
- ✗ Expecting `-`, `*`, and `/` to work like `+`. They convert text automatically (`"6" * "2"` is `12`), but `+` does not. Convert on purpose so nothing is left to chance.

🔑 Key points

- Input from users is always text. Convert it before doing maths.
- `Number(text)` is strict. `parseInt` and `parseFloat` read from the start and stop at the first wrong character.
- `parseInt` cuts off decimals. It does not round.
- `Number("")` is `0`, which can be a trap.

🔧 Practice

- 1 Convert `"250"` into a number and add 50 to it.
- 2 What do `Number("12abc")` and `parseInt("12abc")` give? Predict, then try.
- 3 Use `parseFloat` to get the number out of `"19.99 USD"`.

11.2 NaN and Checking Numbers

🎯 What is it?

`NaN` means "Not a Number". You get it when a calculation cannot give a real number, for example `Number("hello")` or `0 / 0`. Strangely, the type of `NaN` is `"number"`, and `NaN` is not equal to **anything**, not even itself. To test for it, use `Number.isNaN(value)`.

💡 Why do we use it?

Bad input is normal: users type letters where numbers should be. If you do not catch `NaN`, it spreads through your calculations and ruins every result.

<> Syntax

Syntax pattern

```
Number.isNaN(value)      // true only for NaN
Number.isInteger(value)  // true for whole numbers
```

✍ Simple example

JavaScript

```
const result = Number("hello");

console.log(result);
console.log(typeof result);
console.log(result === NaN);
console.log(Number.isNaN(result));
console.log(NaN + 5);
console.log(0 / 0, 10 / 0);
console.log(Number.isInteger(7), Number.isInteger(7.5));
```

Output

```
NaN
number
false
true
NaN
NaN Infinity
true false
```

Dividing a number by zero does not crash. It gives `Infinity`. And `NaN` is contagious: any calculation with `NaN` gives `NaN`.

🌐 Real-world example

An age checker. It rejects text that is not a number, empty text, and impossible ages:

JavaScript

```
function readAge(text) {
  if (text.trim() === "") {
    return "Please type your age.";
  }
  const age = Number(text);
  if (Number.isNaN(age)) {
    return "That is not a number.";
  }
  if (age < 0 || age > 120) {
    return "That age is out of range.";
  }
  return "Age accepted: " + age;
}

console.log(readAge("15"));
console.log(readAge("abc"));
console.log(readAge(""));
console.log(readAge("200"));
```


Output

```
Age accepted: 15
That is not a number.
Please type your age.
That age is out of range.
```

⚠ Common mistakes

- ✗ Testing with `value === NaN`. It is always `false`. Use `Number.isNaN(value)`.
- ✗ Forgetting that empty input becomes `0`, which passes as a number. Check for empty text first.
- ✗ Using the old global `isNaN("abc")`. It converts its input first, which can surprise you. `Number.isNaN` is more precise.

🔑 Key points

- `NaN` means the result is not a valid number.
- `NaN` is not equal to anything, so use `Number.isNaN()` to test for it.
- Any maths that involves `NaN` gives `NaN`.
- Dividing by zero gives `Infinity`, not an error.

☰ Practice

- 1 Print the result of `Number("12.5.3")` and check it with `Number.isNaN`.
- 2 Write `isValidScore(text)` that returns `true` only for numbers from 0 to 100.
- 3 Explain in one sentence why `NaN === NaN` is `false`.

11.3 Rounding Numbers

🕒 What is it?

The `Math` object gives you four ways to turn a decimal into a whole number:

Method	What it does	4.3	4.7	-4.3
<code>Math.round(x)</code>	nearest whole number	4	5	-4
<code>Math.floor(x)</code>	always down	4	4	-5
<code>Math.ceil(x)</code>	always up	5	5	-4
<code>Math.trunc(x)</code>	cut off the decimals	4	4	-4

💡 Why do we use it?

Prices, pages, boxes, and seats are whole things. You cannot sell 2.3 tickets or use 3.6 boxes. Rounding decides how to handle the extra part.

<> Syntax

Syntax pattern

```
Math.round(number)
Math.floor(number)
Math.ceil(number)
Math.trunc(number)
```

✍ Simple example

JavaScript

```
console.log(Math.round(4.5), Math.round(4.4));
console.log(Math.floor(4.9), Math.floor(-4.1));
console.log(Math.ceil(4.1), Math.ceil(-4.9));
console.log(Math.trunc(4.9), Math.trunc(-4.9));
```

Output

```
5 4
4 -5
5 -4
4 -4
```

🌐 Real-world example

Boxes and minutes. 10 friends need pizza, and one box feeds 4 people. How many boxes must you order? Also, how long is 125 seconds in minutes and seconds?

JavaScript

```
const boxes = Math.ceil(10 / 4);
console.log("Pizza boxes to order:", boxes);

const totalSeconds = 125;
const minutes = Math.floor(totalSeconds / 60);
const seconds = totalSeconds % 60;
console.log(`${totalSeconds} seconds = ${minutes} min ${seconds} sec`);
```

Output

```
Pizza boxes to order: 3
125 seconds = 2 min 5 sec
```

Here `ceil` is right, because 2.5 boxes means you need 3. And `floor` is right for minutes, because a minute is only counted once it is complete.

⚠️ Common mistakes

- ✗ Using `floor` when you need `ceil`, or the other way round. Ask: "Should the extra part count as a whole one?"
- ✗ Forgetting how negative numbers behave. `Math.floor(-4.1)` is `-5`, and `Math.round(-4.5)` is `-4`. Use `trunc` to simply cut.

✗ Trying to round to decimals with `Math.round(2.567, 2)`. It ignores the second argument. Multiply, round, and divide instead:

JavaScript

```
const price = 2.567;  
  
console.log(Math.round(price, 2));  
console.log(Math.round(price * 100) / 100);
```

Output

```
3  
2.57
```

🔑 Key points

- `round` goes to the nearest, `floor` goes down, `ceil` goes up, and `trunc` cuts off the decimals.
- For negative numbers, `floor` and `trunc` are different.
- To round to 2 decimals: `Math.round(x * 100) / 100`.
- Choose by asking what the extra part should mean.

🔗 Practice

- 1 A bus holds 40 students. How many buses are needed for 135 students? (Use `Math.ceil`.)
- 2 Round `7.456` to 2 decimal places using `Math.round`.
- 3 Convert 3725 seconds into hours, minutes, and seconds using `Math.floor` and `%`.

11.4 Useful Math Methods

🎯 What is it?

The `Math` object is a toolbox of maths helpers. It is not something you create. It is always there, ready to use.

Tool	What it gives
<code>Math.abs(x)</code>	the size of the number without its sign
<code>Math.max(a, b, ...)</code>	the biggest value
<code>Math.min(a, b, ...)</code>	the smallest value
<code>Math.sqrt(x)</code>	the square root
<code>Math.pow(x, y)</code> or <code>x ** y</code>	<code>x</code> to the power of <code>y</code>
<code>Math.PI</code>	the number pi (3.14159...)

💡 Why do we use it?

Distances, areas, discounts, scores, and limits all need these tools. They save you from writing them yourself.

<> Syntax

Syntax pattern

```
Math.max(3, 9, 4)
Math.max(...array)    // spread an array into separate values
Math.sqrt(49)
Math.PI
```

Simple example

JavaScript

```
console.log(Math.abs(-7));  
console.log(Math.max(3, 9, 4));  
console.log(Math.min(3, 9, 4));  
console.log(Math.sqrt(49));  
console.log(Math.pow(2, 5), 2 ** 5);  
console.log(Math.PI);
```

Output

```
7  
9  
3  
7  
32 32  
3.141592653589793
```

Real-world example

A circle, a distance, and the best score.

JavaScript

```
const radius = 5;  
const area = Math.PI * radius ** 2;  
console.log("Circle area:", Math.round(area * 100) / 100);  
  
const dx = 3 - 0;  
const dy = 4 - 0;  
console.log("Distance:", Math.sqrt(dx ** 2 + dy ** 2));  
  
const scores = [72, 95, 38, 60];  
console.log("Best:", Math.max(...scores));  
console.log("Worst:", Math.min(...scores));
```

Output

```
Circle area: 78.54
Distance: 5
Best: 95
Worst: 38
```

The three dots `...scores` spread the array into separate values, so `Math.max(...scores)` works like `Math.max(72, 95, 38, 60)`.

⚠ Common mistakes

- ✗ Passing an array directly: `Math.max([1, 2, 3])` gives `NaN`. Spread it with `Math.max(...array)`.
- ✗ Forgetting the capital `M` in `Math`.
- ✗ Writing `Math.PI()` with brackets. `PI` is a value, not a function.
- ✗ Forgetting that `Math.max()` with no values gives `-Infinity`.

🔑 Key points

- `Math` is a built-in toolbox. Do not create it, just use it.
- `max` and `min` take separate values. Use `...array` to spread an array.
- `Math.sqrt`, `Math.abs`, and `Math.PI` cover many everyday needs.
- `x ** y` is the same as `Math.pow(x, y)`.

☰ Practice

- 1 Find the area of a circle with radius 7, rounded to 1 decimal place.
- 2 Use `Math.max` and `Math.min` to find the highest and lowest of `[14, 3, 27, 9]`.
- 3 The temperature dropped from 5 to -3. Use `Math.abs` to print how many degrees it dropped.

11.5 Random Numbers

🎯 What is it?

`Math.random()` gives a random decimal number from `0` up to (but never including) `1`. By multiplying, adding, and rounding down, you can turn it into random whole numbers in any range.

💡 Why do we use it?

Games roll dice, quizzes shuffle questions, apps pick a "winner", and tools make sample data. All of them need randomness.

<> Syntax

Syntax pattern

```
Math.random()           // 0 to 0.999...
Math.floor(Math.random() * n) // 0 to n - 1
Math.floor(Math.random() * (max - min + 1)) + min // min to max
```

✍ Simple example

A dice roll is a random whole number from 1 to 6. Your numbers will be different each time you run this:

JavaScript

```
const roll = Math.floor(Math.random() * 6) + 1;
console.log("You rolled a", roll);
```

Output

```
You rolled a 4
```

Here is how it works: `Math.random() * 6` gives a decimal from 0 to 5.99..., `Math.floor` turns it into 0 to 5, and `+ 1` shifts it to 1 to 6.

🌐 Real-world example

Reusable random helpers. A game needs random integers, and a quiz needs a random item from a list. The outputs below are random, so this code prints checks that must always be true:

JavaScript

```
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

function pickRandom(list) {
  return list[randomInt(0, list.length - 1)];
}

// Roll a die 1000 times and collect what we see
const seen = [];
let allInRange = true;
for (let i = 0; i < 1000; i++) {
  const roll = randomInt(1, 6);
  if (!seen.includes(roll)) {
    seen.push(roll);
  }
  if (roll < 1 || roll > 6) {
    allInRange = false;
  }
}
console.log("Every roll was 1 to 6:", allInRange);
console.log("All six faces appeared:", seen.length === 6);

const colors = ["red", "green", "blue"];
console.log("Pick is from the list:", colors.includes(pickRandom(colors)));
```

Output

```
Every roll was 1 to 6: true
All six faces appeared: true
Pick is from the list: true
```

⚠ Common mistakes

- ✗ Forgetting `Math.floor`. You get decimals like `3.7215`, not whole numbers.
- ✗ Off-by-one errors. `Math.floor(Math.random() * 6)` gives 0 to 5. Add `+ 1` for a dice.
- ✗ Using `Math.round` instead of `Math.floor`. The smallest and largest values then come up only half as often.
- ✗ Using `Math.random()` for passwords or secret codes. It is fine for games, but it is **not secure**. Real security needs special tools built for the job.

🔑 Key points

- `Math.random()` gives a decimal from 0 up to (not including) 1.
- `Math.floor(Math.random() * n)` gives a whole number from 0 to `n - 1`.
- For a range, use `Math.floor(Math.random() * (max - min + 1)) + min`.
- Pick a random array item with a random index.

☰ Practice

- 1 Write `coinFlip()` that returns `"heads"` or `"tails"` with equal chances.
- 2 Print a random number between 10 and 20 (both included).
- 3 Pick a random name from an array of five names.

11.6 Formatting Numbers and Decimal Surprises

🎯 What is it?

- `toFixed(n)` gives a **string** with exactly `n` digits after the decimal point.
 - `toLocaleString()` writes a number the way people read it, with commas and even currency signs.
-

💡 Why do we use it?

Money should show two decimals (`5.50` , not `5.5`). Big numbers are easier to read with separators (`1,234,567`). Formatting makes numbers friendly.

<> Syntax

Syntax pattern

```
number.toFixed(2)
number.toLocaleString("en-US")
number.toLocaleString("en-US", { style: "currency", currency: "USD" })
```

✍ Simple example

JavaScript

```
const total = 1234.5;

console.log(total.toFixed(2));
console.log((3.14159).toFixed(2));
console.log((5).toFixed(2));
console.log(total.toLocaleString("en-US"));
console.log(total.toLocaleString("en-US", { minimumFractionDigits: 2 }));
console.log(total.toLocaleString("en-US", { style: "currency", currency: "USD" }));
```

Output

```
1234.50
3.14
5.00
1,234.5
1,234.50
$1,234.50
```

Notice that plain `toLocaleString` adds the comma but does not add a missing zero. Give it `minimumFractionDigits: 2` when you want two decimals every time.

🌐 Real-world example

The 0.1 + 0.2 surprise. Computers store decimals in binary, and many decimals cannot be stored exactly. So tiny errors appear:

JavaScript

```
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
console.log((0.1 + 0.2).toFixed(2));
console.log(Math.abs(0.1 + 0.2 - 0.3) < 0.0001);
```

Output

```
0.30000000000000004
false
0.30
true
```

This is not a JavaScript bug. Almost every programming language does it. For **money**, many programs store whole amounts (like cents or paisa) and divide only at the end. To compare decimals, check that they are **close enough**, as in the last line.

⚠ Common mistakes

- ✗ Forgetting that `toFixed` returns a **string**. `"3.14" + 1` gives `"3.141"`. Convert back with `Number(...)` if you need to do more maths.
- ✗ Calling `toFixed` on text. `"3.14159".toFixed(2)` is an error. Convert with `Number` first.
- ✗ Expecting `toFixed` to be perfect at rounding halves. Because of the binary storage, `(1.005).toFixed(2)` gives `"1.00"`, not `"1.01"`.
- ✗ Comparing decimals with `===` after calculations.

🔑 Key points

- `toFixed(n)` gives a string with `n` decimals.
- `toLocaleString()` adds separators and can format currency.
- Decimals are stored in binary, so `0.1 + 0.2` is not exactly `0.3`.
- Compare decimals with a small tolerance, and keep money in whole units when possible.

☰ Practice

- 1 Show the price `7` as `"7.00"`.
- 2 Format `9876543.21` with commas for `"en-US"`.
- 3 Write a check that says `0.1 * 3` and `0.3` are "close enough", using `Math.abs`.

🚩 Mini project: Bill Splitter and Dice Lab

Two small programs in one file. The first splits a restaurant bill, and it needs `Number`, rounding, and `toFixed`. The second rolls a die 600 times and counts how often each face comes up. The counts change every run, so we print facts that are always true.

JavaScript

```
// PART 1: the bill
const subtotal = Number("2450");
const tax = subtotal * 13 / 100;
const tip = subtotal * 10 / 100;
const total = subtotal + tax + tip;
const people = 4;
const share = Math.ceil(total / people);

console.log("Subtotal:", subtotal.toFixed(2));
console.log("Tax (13%):", tax.toFixed(2));
console.log("Tip (10%):", tip.toFixed(2));
console.log("Total:", total.toFixed(2));
console.log(`Each of ${people} people pays at least:`, share);

// PART 2: the dice
const counts = [0, 0, 0, 0, 0, 0];
for (let i = 0; i < 600; i++) {
  const face = Math.floor(Math.random() * 6) + 1;
  counts[face - 1]++;
}

const counted = counts.reduce((sum, c) => sum + c, 0);
console.log("Rolls counted:", counted);
console.log("Some face came up 100+ times:", Math.max(...counts) >= 100);
```

Output

```
Subtotal: 2450.00
Tax (13%): 318.50
Tip (10%): 245.00
Total: 3013.50
Each of 4 people pays at least: 754
Rolls counted: 600
Some face came up 100+ times: true
```

Challenge: print how many times each face appeared, using `counts.forEach`. With 600 rolls, each face should show up about 100 times. Then change the tip to a variable that the user can set.

🔖 Chapter 11 in one minute

- Text from forms must be converted: `Number(text)`, `parseInt(text)`, `parseFloat(text)`.
- NaN means "not a number". Check it with `Number.isNaN()`, never with `===`.
- `round`, `floor`, `ceil`, and `trunc` turn decimals into whole numbers in different ways.
- `Math.max`, `Math.min`, `Math.abs`, `Math.sqrt`, and `Math.PI` are everyday helpers. Spread arrays with `...`.
- `Math.floor(Math.random() * n)` gives 0 to `n - 1`.
- `toFixed(2)` returns a string. `toLocaleString()` adds commas and currency signs.
- Decimals are not always exact: `0.1 + 0.2` is `0.30000000000000004`.

▼ Sample answers for Chapter 11

11.1, exercise 2 (Number vs parseInt): `Number("12abc")` gives `NaN`, because it is strict. `parseInt("12abc")` gives `12`, because it reads until

the first wrong character.

JavaScript

```
console.log(Number("12abc"));  
console.log(parseInt("12abc"));
```

Output

```
NaN  
12
```

11.2, exercise 2 (isValidScore):

JavaScript

```
function isValidScore(text) {  
  const n = Number(text);  
  return text.trim() !== "" && !Number.isNaN(n) && n >= 0 && n <= 100;  
}  
  
console.log(isValidScore("85"));  
console.log(isValidScore("abc"));  
console.log(isValidScore("120"));  
console.log(isValidScore(""));
```

Output

```
true  
false  
false  
false
```

11.3, exercise 3 (hours, minutes, seconds):

JavaScript

```
const total = 3725;
const hours = Math.floor(total / 3600);
const minutes = Math.floor((total % 3600) / 60);
const seconds = total % 60;

console.log(`${hours}h ${minutes}m ${seconds}s`);
```

Output

1h 2m 5s

11.4, exercise 1 (circle with radius 7):

JavaScript

```
const area = Math.PI * 7 ** 2;

console.log(Math.round(area * 10) / 10);
```

Output

153.9

11.5, exercise 1 (coinFlip):

JavaScript

```
function coinFlip() {
  return Math.random() < 0.5 ? "heads" : "tails";
}

const result = coinFlip();
console.log(result === "heads" || result === "tails");
```

Output

true

11.6, exercise 3 (close enough):

JavaScript

```
console.log(Math.abs(0.1 * 3 - 0.3) < 0.0001);
```

Output

true

2

Project 2

Library Book Manager

Part 2 gave you the tools for handling data: arrays to hold lists, objects to describe things, strings to work with text, and numbers to count and calculate. Now you will build the heart of a small library system. It can list books, search them, lend and take back books, add new ones, and report statistics. This is the kind of data handling that real apps do all the time.

What you will learn

- Model real things as an **array of objects**
- Search text without worrying about capital letters
- Change an object that you found with `find`
- Use `map`, `filter`, `find`, `reduce`, and `sort` together
- Format output neatly with string and number tools

Step 1: Plan the library

Write down what the program must do before you code:

1. Keep a list of books. Each book has an `id`, `title`, `author`, `year`, `pages`, and `available`.
2. Show a book as one neat line of text.
3. Search by a word in the title **or** the author, ignoring capital letters.
4. Borrow and return a book by its `id`, with helpful messages when something is wrong.
5. Add a new book with the next free `id`.

6. Show statistics: number of books, how many are on the shelf, average pages, and the oldest book.

Step 2: The data

Each book is an object, and the whole library is an array of these objects.

JavaScript

```
const library = [
  { id: 1, title: "The River Map", author: "Mira Thapa", year: 2004, pages: 240,
    available: true },
  { id: 2, title: "Clockwork Garden", author: "Jonas Reed", year: 1998, pages:
    180, available: true },
  { id: 3, title: "Letters from the Hills", author: "Anita Rai", year: 1975,
    pages: 320, available: false },
  { id: 4, title: "Small Stars", author: "Mira Thapa", year: 2012, pages: 96,
    available: true },
  { id: 5, title: "The Last Bus Home", author: "Omar Haddad", year: 2020, pages:
    150, available: false }
];
```

Step 3: Show one book as text

`padStart` puts a zero in front of single-digit ids, so the list lines up. A ternary picks the status word.

JavaScript

```
function formatBook(book) {
  const id = String(book.id).padStart(2, "0");
  const status = book.available ? "in" : "OUT";
  return `${id}. ${book.title} by ${book.author} (${book.year}) [${status}]`;
}
```

Step 4: Search

The keyword and both the title and the author are turned into lower case, so "Mira", "MIRA", and "mira" all find the same books. `filter` keeps every book that matches.

JavaScript

```
function search(keyword) {  
  const word = keyword.trim().toLowerCase();  
  return library.filter(book =>  
    book.title.toLowerCase().includes(word) ||  
    book.author.toLowerCase().includes(word)  
  );  
}
```

Step 5: Borrow and return

`find` gives back the **actual object** inside the array, so changing `book.available` changes the library itself. Each function checks for the two possible problems first, and returns a message.

JavaScript

```
function checkOut(id) {
  const book = library.find(b => b.id === id);
  if (!book) {
    return `There is no book with id ${id}.`;
  }
  if (!book.available) {
    return `${book.title} is already checked out.`;
  }
  book.available = false;
  return `You borrowed "${book.title}".`;
}

function returnBook(id) {
  const book = library.find(b => b.id === id);
  if (!book) {
    return `There is no book with id ${id}.`;
  }
  if (book.available) {
    return `${book.title} is already on the shelf.`;
  }
  book.available = true;
  return `Thank you for returning "${book.title}".`;
}
```

Step 6: Add a book

The new `id` is one more than the biggest existing id. The three dots spread the list of ids into `Math.max`.

JavaScript

```
function addBook(title, author, year, pages) {
  const newId = Math.max(...library.map(b => b.id)) + 1;
  library.push({ id: newId, title: title, author: author, year: year, pages:
pages, available: true });
  return newId;
}
```

Step 7: Statistics

Four different array tools give four facts.

JavaScript

```
function printStats() {  
  const totalPages = library.reduce((sum, b) => sum + b.pages, 0);  
  const averagePages = Math.round(totalPages / library.length);  
  const oldestYear = Math.min(...library.map(b => b.year));  
  const oldest = library.find(b => b.year === oldestYear);  
  const onShelf = library.filter(b => b.available).length;  
  
  console.log("Books:", library.length);  
  console.log("On the shelf:", onShelf);  
  console.log("Average length:", averagePages, "pages");  
  console.log("Oldest book:", oldest.title, `${oldest.year}`);  
}
```

The complete program

Here is everything together, followed by what it prints:

JavaScript

```
const library = [
  { id: 1, title: "The River Map", author: "Mira Thapa", year: 2004, pages: 240,
    available: true },
  { id: 2, title: "Clockwork Garden", author: "Jonas Reed", year: 1998, pages:
    180, available: true },
  { id: 3, title: "Letters from the Hills", author: "Anita Rai", year: 1975,
    pages: 320, available: false },
  { id: 4, title: "Small Stars", author: "Mira Thapa", year: 2012, pages: 96,
    available: true },
  { id: 5, title: "The Last Bus Home", author: "Omar Haddad", year: 2020, pages:
    150, available: false }
];

function formatBook(book) {
  const id = String(book.id).padStart(2, "0");
  const status = book.available ? "in" : "OUT";
  return `${id}. ${book.title} by ${book.author} (${book.year}) [${status}]`;
}

function search(keyword) {
  const word = keyword.trim().toLowerCase();
  return library.filter(book =>
    book.title.toLowerCase().includes(word) ||
    book.author.toLowerCase().includes(word)
  );
}

function checkOut(id) {
  const book = library.find(b => b.id === id);
  if (!book) {
    return `There is no book with id ${id}.`;
  }
  if (!book.available) {
    return `"${book.title}" is already checked out.`;
  }
  book.available = false;
  return `You borrowed "${book.title}".`;
}

function returnBook(id) {
  const book = library.find(b => b.id === id);
  if (!book) {
```



```

    return `There is no book with id ${id}.`;
  }
  if (book.available) {
    return `${book.title}" is already on the shelf.`;
  }
  book.available = true;
  return `Thank you for returning "${book.title}".`;
}

function addBook(title, author, year, pages) {
  const newId = Math.max(...library.map(b => b.id)) + 1;
  library.push({ id: newId, title: title, author: author, year: year, pages:
pages, available: true });
  return newId;
}

function printStats() {
  const totalPages = library.reduce((sum, b) => sum + b.pages, 0);
  const averagePages = Math.round(totalPages / library.length);
  const oldestYear = Math.min(...library.map(b => b.year));
  const oldest = library.find(b => b.year === oldestYear);
  const onShelf = library.filter(b => b.available).length;

  console.log("Books:", library.length);
  console.log("On the shelf:", onShelf);
  console.log("Average length:", averagePages, "pages");
  console.log("Oldest book:", oldest.title, `(${oldest.year})`);
}

console.log("=== All books ===");
library.forEach(book => console.log(formatBook(book)));

console.log("");
console.log('=== Search: "mira" ===');
search("mira").forEach(book => console.log(formatBook(book)));

console.log("");
console.log("=== Borrowing ===");
console.log(checkOut(1));
console.log(checkOut(1));
console.log(checkOut(3));
console.log(checkOut(9));
console.log(returnBook(3));

```

```
console.log("");
console.log("=== Add a book ===");
const newId = addBook("Paper Boats", "Lina Gurung", 2023, 128);
console.log("Added book number", newId);

console.log("");
console.log("=== Sorted by year ===");
library.slice().sort((a, b) => a.year - b.year).forEach(book => {
  console.log(book.year, "-", book.title);
});

console.log("");
console.log("=== Statistics ===");
printStats();
```

Output

```
=== All books ===
01. The River Map by Mira Thapa (2004) [in]
02. Clockwork Garden by Jonas Reed (1998) [in]
03. Letters from the Hills by Anita Rai (1975) [OUT]
04. Small Stars by Mira Thapa (2012) [in]
05. The Last Bus Home by Omar Haddad (2020) [OUT]

=== Search: "mira" ===
01. The River Map by Mira Thapa (2004) [in]
04. Small Stars by Mira Thapa (2012) [in]

=== Borrowing ===
You borrowed "The River Map".
"The River Map" is already checked out.
"Letters from the Hills" is already checked out.
There is no book with id 9.
Thank you for returning "Letters from the Hills".

=== Add a book ===
Added book number 6

=== Sorted by year ===
1975 - Letters from the Hills
1998 - Clockwork Garden
2004 - The River Map
2012 - Small Stars
2020 - The Last Bus Home
2023 - Paper Boats

=== Statistics ===
Books: 6
On the shelf: 4
Average length: 186 pages
Oldest book: Letters from the Hills (1975)
```

Check a few numbers by hand. After the borrowing steps, books 2, 3 (returned), 4, and the new book 6 are on the shelf, so 4 books are in. The pages add up to 1114, and 1114 divided by 4 is about 278.5, which `Math.round` turns into 279.

Your turn: challenges

1. **Search by year.** Write `booksFromYear(year)` that returns all books published in or after that year.
2. **A full report line.** Make `formatBook` show the number of pages too, such as `(240 pages)`.
3. **Sort by title.** Print the books in alphabetical order. (Hint: `localeCompare`.)
4. **Prevent duplicates.** Make `addBook` refuse a book whose title already exists, and return `0` instead of a new id.
5. **A borrower name.** Change `available` into a `borrower` property that holds a name, or `null` when the book is on the shelf.

▼ Hint for challenge 3

Sort a **copy** with `localeCompare`, so the original order stays the same:

JavaScript

```
const books = [
  { title: "The River Map" },
  { title: "Clockwork Garden" },
  { title: "Small Stars" }
];

books
  .slice()
  .sort((a, b) => a.title.localeCompare(b.title))
  .forEach(book => console.log(book.title));
```

Output

```
Clockwork Garden
Small Stars
The River Map
```

What you practised

- Modelling data as an array of objects.
- `find` to reach one object, and changing it in place.
- `filter` , `map` , `reduce` , and `sort` on lists of objects.
- String methods (`toLowerCase` , `includes` , `padStart`) for searching and formatting.
- Number tools (`Math.max` , `Math.min` , `Math.round`) for statistics.

12

Chapter 12

The DOM

So far, your programs have printed text into a console. Now the fun begins: JavaScript can **change the web page itself**. It can rewrite a heading, fill a list, turn a word red, or build a whole table from data. The tool that makes this possible is the **DOM**. In this chapter you will learn to find elements on a page, and to change, create, and remove them.

What you will learn

- Explain what the DOM is and how `document` gives you access to it
- Select elements with `getElementById`, `querySelector`, and `querySelectorAll`
- Change text and HTML with `textContent` and `innerHTML`
- Read and change attributes and form values
- Add, remove, and toggle CSS classes, and set inline styles
- Create new elements and add them to the page, or remove them

How the live examples work

From here on, the examples are real web page pieces that run right inside the book. The dark box shows the code (press **Copy** to take it). The white box under it is the **live result**. Anything sent to `console.log` appears in a small dark **console** box at the bottom of the result. Press **Reset** to run the example again.

In your own files, put the code in an `.html` file, place the `<script>` just before `</body>`, and look in the browser's Console tab (press F12) for `console.log` messages.

12.1 What Is the DOM?

What is it?

When a browser loads your HTML, it does not keep it as plain text. It builds a **tree of objects** in memory, and this tree is the **DOM**: the **D**ocument **O**bject **M**odel. Every tag becomes an object called an **element**. The whole tree hangs from one object named `document`.

Diagram

```
document
├── html
│   ├── head
│   │   └── title
│   └── body
│       ├── h1
│       └── p
```

JavaScript can read this tree and change it. When the tree changes, the browser updates the screen immediately.

🔗 Why do we use it?

Without the DOM, JavaScript could only calculate things in secret. The DOM is the bridge between your code and what the user sees. Every interactive feature on the web (menus, likes, live search, games) works by changing the DOM.

<> Syntax

Syntax pattern

```
document           // the whole page
document.body      // the <body> element
document.title     // the text of the <title>
element.children   // the elements inside an element
```

✍ Simple example

The script below asks the DOM about itself. Watch the console box:

HTML

```
<h1>My page</h1>
<p>Hello!</p>
<script>
  console.log("Elements in the body:", document.body.children.length);
  console.log("First element says:",
document.body.firstChild.textContent);
  console.log("Is there a body?", document.body !== null);
</script>
```

The body has three elements: the heading, the paragraph, and the `<script>` itself.

🌐 Real-world example

A welcome banner built from data. A page starts with a placeholder heading, and the script fills in the real message. (You will learn `getElementById` and `textContent` in the next sections. For now, just watch what happens.)

HTML

```
<h2 id="welcome">Loading...</h2>
<script>
  const user = { name: "Asha", unread: 3 };
  document.getElementById("welcome").textContent =
    "Welcome back, " + user.name + "! You have " + user.unread + " new
  messages.";
</script>
```

⚠ Common mistakes

- ✗ Thinking the DOM is your `.html` file. It is a **live copy in memory**. When JavaScript changes the DOM, the page changes, but your file on disk stays exactly as it was.
- ✗ Trying to use `document` in Node.js. It belongs to the **browser**, so Node.js gives an error.
- ✗ Forgetting the `<script>` tag. Your JavaScript must be inside `<script>...</script>` or loaded from a file with `<script src="...">`.

🔑 Key points

- The DOM is a tree of objects that the browser builds from your HTML.
- `document` is the doorway to the whole tree.
- Changing the DOM changes what the user sees, but not your `.html` file.
- The DOM belongs to the browser, not to JavaScript itself.

🔧 Practice

- 1 Draw the DOM tree of a page that has a `<h1>`, and a `<ul>` with two `<li>` items.

- 2 Use `document.body.children.length` in a page with three paragraphs. What number do you expect? (Remember the `<script>.`)
- 3 Explain in one sentence why changing the DOM does not change the `.html` file.

12.2 Selecting Elements

🕒 What is it?

Before you can change an element, you have to **find** it. This is called **selecting**. There are three main tools:

Tool	What it gives you
<code>document.getElementById("id")</code>	the one element with that <code>id</code>
<code>document.querySelector("selector")</code>	the first element that matches a CSS selector
<code>document.querySelectorAll("selector")</code>	all matching elements, as a list

The selectors are the same ones you know from CSS: `#title` for an id, `.item` for a class, `li` for a tag, `ul li` for a tag inside a tag.

💡 Why do we use it?

Every DOM job starts with "find the element". Once you have it, you can read it, change it, move it, or remove it.

<> Syntax

Syntax pattern

```
const one = document.getElementById("title");           // no # sign here
const first = document.querySelector(".item");          // CSS selector
const all = document.querySelectorAll(".item");         // a list of all matches
```

Simple example

The heading, the first item, and the whole list of items are selected and reported:

HTML

```
<h2 id="title">Shopping list</h2>
<ul>
  <li class="item">Milk</li>
  <li class="item">Bread</li>
  <li class="item">Eggs</li>
</ul>
<script>
  const title = document.getElementById("title");
  const firstItem = document.querySelector(".item");
  const allItems = document.querySelectorAll(".item");

  console.log("Title:", title.textContent);
  console.log("First item:", firstItem.textContent);
  console.log("How many items:", allItems.length);
  console.log("Third item:", allItems[2].textContent);

  allItems.forEach(function (li) {
    console.log("-", li.textContent);
  });
</script>
```

`querySelectorAll` gives a **NodeList**. It looks like an array: it has `length`, numbers as indexes, `forEach`, and it works with `for...of`.

🌐 Real-world example

A cart total read from the page. The prices are written in the HTML, and JavaScript adds them up:

HTML

```
<ul id="cart">
  <li class="price">250</li>
  <li class="price">80</li>
  <li class="price">120</li>
</ul>
<p>Total: <b id="total">0</b></p>
<script>
  const prices = document.querySelectorAll("#cart .price");
  let total = 0;

  for (const p of prices) {
    total += Number(p.textContent);
  }

  document.querySelector("#total").textContent = total;
</script>
```

Remember from Chapter 11 that text must be converted with `Number(...)` before doing maths.

⚠️ Common mistakes

- ✗ Running the script **before** the element exists. The browser reads the page from top to bottom, so a script placed above the element cannot find it. It gets `null`, and using `null` causes an error:

HTML

```
<script>
  document.getElementById("msg").textContent = "Hello!";
</script>
<p id="msg">Waiting...</p>
```

The fix is to put the script **after** the element (usually just before `</body>`).

- ✗ Writing a `#` in `getElementById("#title")`. That method wants just the id: `"title"`. Only `querySelector` uses the CSS form `"#title"`.
- ✗ Forgetting the dot for classes in `querySelector("item")`. That looks for a `<item>` tag. Use `".item"`.
- ✗ Expecting `querySelector` to give many elements. It gives only the **first** match. Use `querySelectorAll` for all of them.
- ✗ Calling `map` or `filter` on a `NodeList`. It does not have them. Convert it to a real array first:

JavaScript

```
const items = document.querySelectorAll(".item");
const names = Array.from(items).map(li => li.textContent);
```

🔑 Key points

- `getElementById("id")` finds one element by its id (no `#`).
- `querySelector("css")` finds the first match. `querySelectorAll("css")` finds all matches.
- A `NodeList` has `length`, indexes, and `forEach`. Use `Array.from()` to get array methods.
- If nothing matches, you get `null` (or an empty list). Put your script after the HTML.

☰ Practice

- 1 Select the element with the id `"logo"` and print its `textContent`.
- 2 Select all the `<p>` elements in a page and print how many there are.
- 3 Use `querySelector` with a selector that finds the first `<li>` inside `<ul id="menu">`.

12.3 Changing Text and HTML

🎯 What is it?

Two properties change what is **inside** an element:

- `textContent` reads or sets the plain **text**. Any `<tags>` you put in are shown as ordinary characters.
- `innerHTML` reads or sets the inner **HTML**. Tags you put in become real elements.

💡 Why do we use it?

This is the most direct way to change what the user reads: update a score, show an error message, fill in a name, or write a whole new block of content.

<> Syntax

Syntax pattern

```
element.textContent = "new text";
element.innerHTML = "some <b>HTML</b>";
console.log(element.textContent); // reading works too
```

Simple example

HTML

```
<h2 id="heading">Old title</h2>
<p id="note">This text will change.</p>
<p id="box"></p>
<p id="plain"></p>
<script>
  document.getElementById("heading").textContent = "New title";
  document.getElementById("note").textContent = "Changed with JavaScript!";
  document.getElementById("box").innerHTML = "With innerHTML: <b>bold</b> and
  <i>italic</i>.";
  document.getElementById("plain").textContent = "With textContent: <b>bold</b>
  stays as text.";
</script>
```

Real-world example

A countdown timer. `setInterval` is a built-in function (like `setTimeout` from Chapter 7). It calls your callback again and again, every 1000 milliseconds. The script updates the heading every second, then stops the timer with `clearInterval`. Press **Reset** to watch it again:

HTML

```
<h2 id="clock">5</h2>
<script>
  let seconds = 5;
  const clock = document.getElementById("clock");

  const timer = setInterval(function () {
    seconds--;
    if (seconds > 0) {
      clock.textContent = seconds;
    } else {
      clock.textContent = "Time is up!";
      clearInterval(timer);
    }
  }, 1000);
</script>
```

⚠ Common mistakes

- ✗ Putting user-typed text into `innerHTML`. Hidden code inside that text could run on your page. This is a real security problem called **XSS**. For text that comes from a user, always use `textContent`:

JavaScript

```
const userText = "<img src=x onerror=alert('hacked')>";

box.innerHTML = userText;    // dangerous: the hidden code runs
box.textContent = userText;  // safe: it is shown as plain text
```

- ✗ Expecting `textContent` to create tags. `<b>` is shown as the characters `<b>`. Use `innerHTML` (with safe content) or create the elements (12.6).
- ✗ Changing the **variable** instead of the **property**. `heading = "New"` does nothing to the page. You must write `heading.textContent = "New"`.
- ✗ Using `innerHTML +=` again and again in a loop. It is slow, and it rebuilds all the elements every time.

🔑 Key points

- `textContent` sets or reads plain text. It is the safe default.
- `innerHTML` sets or reads HTML. Use it only with content you wrote yourself.
- Never put text from a user into `innerHTML`.
- You change the property of the element (`el.textContent`), not the variable.

🔧 Practice

- 1 Write `<p id="status">Loading...</p>` and change it to `"Ready!"` with JavaScript.
- 2 Use `innerHTML` to show a sentence with one **bold** word.
- 3 Explain why `textContent` is safer than `innerHTML` for text typed by a user.

12.4 Attributes and Form Values

🕒 What is it?

An **attribute** is extra information inside a tag: `href`, `src`, `id`, `class`, `type`, `disabled`, and more. JavaScript can read and change them:

- `getAttribute("name")` reads an attribute. `setAttribute("name", value)` sets it. `removeAttribute("name")` removes it.
- Many attributes also work as **properties**: `link.href`, `button.disabled`, `checkbox.checked`.
- For form fields, `input.value` is the text inside the field.
- `data-` **attributes** are your own custom labels, such as `data-price="250"`. Read them through `element.dataset.price`.

🔗 Why do we use it?

Attributes control where links go, whether a button works, and what a form field holds. `data-` attributes let you attach small pieces of information to elements.

<> Syntax

Syntax pattern

```
element.setAttribute("href", "https://example.com");
element.getAttribute("href");
input.value = "Asha";           // form field text
button.disabled = true;        // turn a property on
element.dataset.price           // reads data-price="..."
```

✍ Simple example

HTML

```
<a id="link">Visit a website</a>
<br><br>
<input id="username" placeholder="Type your name">
<button id="send">Send</button>
<script>
  const link = document.getElementById("link");
  link.setAttribute("href", "https://example.com");
  link.setAttribute("target", "_blank");
  console.log("Link goes to:", link.getAttribute("href"));

  const usernameBox = document.getElementById("username");
  usernameBox.value = "Asha";
  console.log("The box holds:", usernameBox.value);

  document.getElementById("send").disabled = true;
</script>
```

The link now works, the box is filled in, and the button is switched off.

🌐 Real-world example

Totals from `data-` attributes. Each item carries its price in a `data-price` attribute. The script adds them up:

HTML

```
<ul>
  <li data-price="250">Book</li>
  <li data-price="80">Pen</li>
  <li data-price="45">Eraser</li>
</ul>
<p id="sum"></p>
<script>
  let total = 0;

  document.querySelectorAll("li").forEach(function (li) {
    total += Number(li.dataset.price);
  });

  document.getElementById("sum").textContent = "Total price: " + total;
</script>
```

⚠ Common mistakes

- ✗ Forgetting that `input.value` is always **text**, even for number fields. Convert it with `Number(...)`.
- ✗ Trying to switch off a boolean attribute with `setAttribute("disabled", false)`. The attribute still **exists**, so the button stays disabled. Use the property instead:

JavaScript

```
button.setAttribute("disabled", false); // still disabled!
button.disabled = false;                // correct
```

- ✗ Using `textContent` to fill an input box. Use `.value`.
- ✗ Forgetting that the `class` attribute has its own tools (`classList`, next section).

🔑 Key points

- `getAttribute`, `setAttribute`, and `removeAttribute` work with any attribute.
- Use properties for common ones: `link.href`, `button.disabled`, `input.value`.
- `input.value` is always text.
- `data-price="250"` is read with `element.dataset.price`.

☰ Practice

- 1 Change the `href` of a link to point to another website.
- 2 Fill in an input with your name, and print its `value` with `console.log`.
- 3 Add `data-stock="5"` to a list item and print the stock number as a real number.

12.5 Classes and Styles

🕒 What is it?

Every element has a `classList` that lets you add, remove, and check CSS classes:

- `classList.add("name")` and `classList.remove("name")`
- `classList.toggle("name")` adds the class if it is missing, and removes it if it is there.
- `classList.contains("name")` tells you `true` or `false`.

You can also set a style directly with `element.style.property`. CSS names with a dash are written in **camelCase**: `background-color` becomes

backgroundColor .

💡 Why do we use it?

The best way to change how something looks is to write the styles in CSS once, and let JavaScript switch the **classes** on and off. Your design stays in your CSS, and your behaviour stays in JavaScript.

<> Syntax

Syntax pattern

```
element.classList.add("done");
element.classList.remove("done");
element.classList.toggle("done");
element.classList.contains("done");    // true or false
element.style.color = "red";
element.style.backgroundColor = "yellow"; // camelCase
```

Simple example

HTML

```
<style>
  .done { text-decoration: line-through; color: gray; }
  .important { background: #FFE14D; font-weight: bold; }
</style>
<ul>
  <li id="a">Wash the dishes</li>
  <li id="b">Finish homework</li>
  <li id="c">Call grandma</li>
</ul>
<script>
  document.getElementById("a").classList.add("done");
  document.getElementById("b").classList.add("important");

  const c = document.getElementById("c");
  c.style.color = "royalblue";
  c.style.fontSize = "24px";
</script>
```

Real-world example

Order status badges. One function puts the right class on a badge, after removing the old ones:

HTML

```
<style>
  .badge { display: inline-block; padding: 2px 12px; border-radius: 12px; color:
#fff; margin: 4px 0; }
  .ok { background: #1A7F37; }
  .late { background: #C62828; }
  .wait { background: #B26A00; }
</style>
<div id="o1" class="badge">Order 1</div><br>
<div id="o2" class="badge">Order 2</div><br>
<div id="o3" class="badge">Order 3</div>
<script>
  function setStatus(id, status) {
    const badge = document.getElementById(id);
    badge.classList.remove("ok", "late", "wait");
    badge.classList.add(status);
  }

  setStatus("o1", "ok");
  setStatus("o2", "late");
  setStatus("o3", "wait");
</script>
```

💡 toggle with a switch

`classList.toggle("on", true)` always **adds** the class, and `classList.toggle("on", false)` always **removes** it. So `element.classList.toggle("on", someCondition)` turns a class on or off to match a `true` or `false` value. You will use this in Chapter 13.

⚠ Common mistakes

- ✗ Writing the dot in the class name: `classList.add(".done")`. Use just `"done"`.
- ✗ Using `element.className = "done"`. It **replaces** every class the element already had. `classList.add` keeps the others.

- ✗ Writing CSS names with a dash: `element.style.background-color` is a syntax error. Use `backgroundColor`.
- ✗ Forgetting the unit: `style.fontSize = 24` does nothing. Write `"24px"`.
- ✗ Expecting `element.style` to show styles that come from a CSS file. It shows only **inline** styles that you have set.

🔑 Key points

- `classList` has `add`, `remove`, `toggle`, and `contains`.
- Prefer switching classes over setting many styles in JavaScript.
- `style` properties use camelCase, and values are text with units.
- `toggle(name, condition)` turns a class on or off to match the condition.

☰ Practice

- 1 Add the class `"highlight"` to a paragraph, and define `.highlight` in CSS.
- 2 Make a heading blue and 40px large using `style`.
- 3 Use `classList.contains` to print whether an element has the class `"done"`.

12.6 Creating, Adding, and Removing Elements

🕒 What is it?

You can build new elements from nothing:

- `document.createElement("tag")` makes a new, empty element. It is **not** on the page yet.
- `parent.append(child)` puts it at the end of the parent (and `parent.prepend(child)` puts it at the start).

- `element.remove()` takes an element off the page.

💡 Why do we use it?

Real pages are built from data: a list of products, a table of students, a feed of messages. You write the pattern once, and JavaScript creates as many elements as the data needs.

<> Syntax

Syntax pattern

```
const li = document.createElement("li"); // 1. create
li.textContent = "New item";             // 2. fill it
list.append(li);                         // 3. add it to the page
li.remove();                             // remove it later
```

✍ Simple example

A list is built from an array:

HTML

```
<h3>Fruits</h3>
<ul id="list"></ul>
<script>
  const fruits = ["Mango", "Banana", "Apple"];
  const list = document.getElementById("list");

  for (const fruit of fruits) {
    const li = document.createElement("li");
    li.textContent = fruit;
    list.append(li);
  }
</script>
```

🌐 Real-world example

A results table from data. Every student in the array becomes a table row. Failing marks get a red class:

HTML

```
<style>
  table { border-collapse: collapse; }
  th, td { border: 1px solid #C7D0E4; padding: 4px 14px; text-align: left; }
  .fail { color: #C62828; font-weight: bold; }
</style>
<table>
  <thead><tr><th>Name</th><th>Marks</th></tr></thead>
  <tbody id="rows"></tbody>
</table>
<script>
  const students = [
    { name: "Asha", marks: 80 },
    { name: "Ravi", marks: 35 },
    { name: "Sita", marks: 92 }
  ];
  const rows = document.getElementById("rows");

  students.forEach(function (student) {
    const row = document.createElement("tr");
    const nameCell = document.createElement("td");
    const marksCell = document.createElement("td");

    nameCell.textContent = student.name;
    marksCell.textContent = student.marks;
    if (student.marks < 40) {
      marksCell.classList.add("fail");
    }

    row.append(nameCell, marksCell);
    rows.append(row);
  });
</script>
```

To remove an element, select it and call `remove()`. To empty a whole container, set `container.innerHTML = ""`:

JavaScript

```
document.getElementById("b").remove();    // remove one element
document.getElementById("list").innerHTML = ""; // empty a container
```

⚠ Common mistakes

- ✗ Creating an element and forgetting to add it. Until you call `append`, it exists only in memory, and nobody can see it.
- ✗ Forgetting `document.` in front of `createElement`.
- ✗ Adding the **same** element twice and expecting two copies. An element can only be in one place, so it **moves**. Create a new element each time (for example, inside the loop).
- ✗ Building HTML by joining strings with user text. Create elements and use `textContent`.
- ✗ Confusing `append` and `appendChild`. `append` is newer and more flexible: it takes several items, and text too.

🔑 Key points

- `createElement` makes an element, `append` puts it on the page, and `remove` takes it away.
- Create a **new** element for every item.
- Use `textContent` to fill elements with text.
- Loop over an array of data to build lists and tables.

🔧 Practice

- 1 Make an empty `<ul id="days">` and fill it with the seven days of the week from an array.
- 2 Create a `<p>`, give it a class, put text in it, and add it to the `<body>`.
- 3 Add a "Delete" step: use `remove()` to delete the first `<li>` of a list.

🚩 Mini project: Profile Card Builder

Build a small profile card from a JavaScript object. The card is created with `createElement`, filled with `textContent`, and styled with classes. Change the data and the card changes.

HTML

```
<style>
  .card { border: 2px solid #0F1B3D; border-radius: 6px; padding: 12px
16px; width: 290px; }
  .card h2 { margin: 0 0 2px; }
  .role { color: #5F6B89; margin: 0 0 8px; }
  .skills { margin: 6px 0 10px; padding-left: 20px; }
  .badge { display: inline-block; padding: 2px 12px; border-radius: 12px;
color: #fff; font-size: 14px; }
  .on { background: #1A7F37; }
  .off { background: #7A7A7A; }
</style>
<div id="card" class="card"></div>
<script>
  const person = {
    name: "Asha Sharma",
    role: "Student developer",
    skills: ["HTML", "CSS", "JavaScript"],
    available: true
  };

  const card = document.getElementById("card");

  const title = document.createElement("h2");
  title.textContent = person.name;

  const role = document.createElement("p");
  role.className = "role";
  role.textContent = person.role;

  const skills = document.createElement("ul");
  skills.className = "skills";
  person.skills.forEach(function (skill) {
    const li = document.createElement("li");
    li.textContent = skill;
    skills.append(li);
  });

  const badge = document.createElement("span");
  badge.classList.add("badge", person.available ? "on" : "off");
  badge.textContent = person.available ? "Available" : "Busy";
```

```
card.append(title, role, skills, badge);  
</script>
```

Challenge: change `available` to `false` and watch the badge change. Then add a `city` property and show it under the role. Finally, make the whole thing a function `buildCard(person)`, and build cards for two different people.

📖 Chapter 12 in one minute

- The DOM is a live tree of objects that the browser builds from your HTML. `document` is the doorway.
- Select with `getElementById`, `querySelector` (first), and `querySelectorAll` (all).
- Change text with `textContent` (safe) or HTML with `innerHTML` (only for your own content).
- Attributes: `setAttribute`, properties like `input.value` and `button.disabled`, and `dataset` for data- attributes.
- Style with `classList` (`add`, `remove`, `toggle`, `contains`), and `style` in camelCase.
- Create with `createElement`, add with `append`, and take away with `remove`.
- Place your script after the HTML, or the elements will not be found.

▼ Sample answers for Chapter 12

12.3, exercise 1 (Loading to Ready):

HTML

```
<p id="status">Loading...</p>
<script>
  document.getElementById("status").textContent = "Ready!";
</script>
```

12.6, exercise 1 (days of the week):

HTML

```
<ul id="days"></ul>
<script>
  const week = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
  const days = document.getElementById("days");

  week.forEach(function (day) {
    const li = document.createElement("li");
    li.textContent = day;
    days.append(li);
  });
</script>
```

13

Chapter 13

Events

A page that only changes by itself is not very interesting. A page that **reacts to you** is. When you click a button, type in a box, press a key, or move the mouse, the browser announces it. That announcement is called an **event**. In this chapter you will learn to listen for events and run your code when they happen. This is the step that turns a web page into a real application.

What you will learn

- Listen for events with `addEventListener`
- Use the event object (`event.target` , `event.key` , and more)
- React to typing and choices with `input` and `change`
- Handle form submission and stop the page from reloading
- Respond to keyboard and mouse events
- Use one listener for many elements (event delegation)

13.1 Events and `addEventListener`

🕒 What is it?

An **event** is something that happens on the page: a click, a key press, a change in a text box, the page finishing loading. An **event listener** is a function that waits for one kind of event on one element, and runs when it happens.

You attach a listener with `addEventListener`. It takes two things: the **name of the event** (as text) and a **callback function** (Chapter 7) to run.

🔗 Why do we use it?

Without events, your code would run once at the start and then stop. Events let the user be in control: nothing happens until they click, type, or press something.

<> Syntax

Syntax pattern

```
element.addEventListener("click", function () {  
    // code to run on every click  
});  
  
// or with a separate named function  
function handleClick() { /* ... */ }  
element.addEventListener("click", handleClick);
```

✍ Simple example

A button counts its own clicks:

HTML

```
<button id="btn">Click me</button>  
<p id="msg">You have not clicked yet.</p>  
<script>  
    const btn = document.getElementById("btn");  
    const msg = document.getElementById("msg");  
    let count = 0;  
  
    btn.addEventListener("click", function () {  
        count++;  
        msg.textContent = "You clicked " + count + " times.";  
    });  
</script>
```

The callback does **not** run when the page loads. It runs later, on every click.

🌐 Real-world example

A like button. It remembers whether you liked the post, changes its text, and switches a CSS class on and off:

HTML

```
<style>
  #like { padding: 6px 18px; border: 2px solid #0F1B3D; background: #fff;
border-radius: 20px; }
  #like.liked { background: #FFE14D; }
</style>
<button id="like"></button>
<script>
  const likeButton = document.getElementById("like");
  let likes = 12;
  let liked = false;

  function draw() {
    likeButton.textContent = (liked ? "♥ Liked" : "♥ Like") + " (" + likes +
    ")";
    likeButton.classList.toggle("liked", liked);
  }

  likeButton.addEventListener("click", function () {
    liked = !liked;
    likes += liked ? 1 : -1;
    draw();
  });

  draw();
</script>
```

The function `draw()` shows the current state on the screen. The listener changes the state and calls `draw()`. This "change the data, then redraw" pattern is used in almost every interactive app.

⚠ Common mistakes

✗ **Calling** the function instead of **passing** it. This is the same mistake as with callbacks in Chapter 7:

JavaScript

```
button.addEventListener("click", showMessage()); // wrong: runs NOW, passes  
its result  
button.addEventListener("click", showMessage);   // correct: passes the  
function itself
```

- ✗ Writing `"onclick"` as the event name. The name is just `"click"`. (Other names are `"input"`, `"keydown"`, `"submit"`, and so on.)
- ✗ Misspelling the event name. `"clik"` gives no error. It just never fires.
- ✗ Adding the same listener many times (for example, inside another listener). Then one click runs your code many times.
- ✗ Selecting the element before it exists (see Chapter 12). Place your script after the HTML.

🔑 Key points

- An event is something that happens. A listener is a function that waits for it.
- `element.addEventListener("eventName", callback)` attaches one.
- The callback runs later, every time the event happens.
- Pass the function itself, without brackets.
- Keep the data in variables, and write a function that redraws the screen from that data.

🔧 Practice

- 1 Make a button that changes the text of a heading when clicked.

- 2 Make a button that hides and shows a paragraph each time you click it. (Hint: add a CSS class `.hidden { display: none; }` and use `classList.toggle()`.)
- 3 Explain the difference between `addEventListener("click", showMessage)` and `addEventListener("click", showMessage())`.

13.2 The Event Object

🕒 What is it?

When an event happens, the browser hands your callback an **event object** with details about what happened. You receive it as a parameter. Most people name it `event` (or `e`).

Property	What it tells you
<code>event.type</code>	the kind of event, such as <code>"click"</code>
<code>event.target</code>	the exact element that was clicked (or typed in)
<code>event.clientX</code> , <code>event.clientY</code>	mouse position in the window
<code>event.offsetX</code> , <code>event.offsetY</code>	mouse position inside the element
<code>event.key</code>	which key was pressed (for keyboard events)
<code>event.preventDefault()</code>	stops the browser's normal action

🔍 Why do we use it?

Often you need to know **which** element was clicked, **where** the mouse was, or **which key** was pressed. The event object answers all of these.

<> Syntax

Syntax pattern

```
element.addEventListener("click", function (event) {  
  console.log(event.type);      // "click"  
  console.log(event.target);    // the clicked element  
});
```

🔗 Simple example

One listener sits on the blue box. Click the buttons and the paragraph, and it reports which element you actually hit:

HTML

```
<div id="box" style="padding:14px;background:#E8EDFF;border:2px solid #2444D6">  
  <button>Button 1</button>  
  <button>Button 2</button>  
  <p>A paragraph inside the box. Click here too!</p>  
</div>  
<p id="report">Click something inside the blue box.</p>  
<script>  
  document.getElementById("box").addEventListener("click", function (event) {  
    const t = event.target;  
    document.getElementById("report").textContent =  
      "You clicked a <" + t.tagName.toLowerCase() + "> element.";  
  });  
</script>
```

🌐 Real-world example

A click position tracker. Games and drawing tools need to know exactly where the user clicked:

HTML

```
<div id="pad" style="height:90px;padding:8px;background:#FFF5BF;border:2px
dashed #7A5C00;cursor:crosshair">
  Click anywhere in this box
</div>
<p id="pos">No click yet.</p>
<script>
  const pad = document.getElementById("pad");

  pad.addEventListener("click", function (event) {
    document.getElementById("pos").textContent =
      "You clicked at x: " + event.offsetX + ", y: " + event.offsetY;
  });
</script>
```

⚠ Common mistakes

- ✗ Forgetting to write the parameter. `function () { event.target }` does not receive the event object, so it fails or uses an outdated global.
- ✗ Confusing `event.target` (the element that was actually clicked, which might be a child) with `event.currentTarget` (the element that has the listener).
- ✗ Using `this` inside an arrow function callback. Use `event.currentTarget` instead.
- ✗ Thinking the parameter must be called `event`. Any name works, but `event` or `e` are standard.

🔑 Key points

- The callback receives an event object as its first parameter.
- `event.target` is the element that triggered the event.
- `event.offsetX` and `event.offsetY` give the position inside the element.
- `event.preventDefault()` stops the default browser action.

☰ Practice

- 1 Print `event.type` in the console when you click a button.
- 2 Show the text of the clicked element with `event.target.textContent`.
- 3 Find the difference between `event.target` and `event.currentTarget` by clicking a `<span>` inside a `<button>` that has the listener.

13.3 Input and Change Events

🎯 What is it?

Two events tell you that the user changed a form field:

- `input` fires **every time** the value changes, even by a single letter.
- `change` fires when the user **finishes** a change: they leave a text box, pick an option, or tick a box.

Inside the handler, read the field's current value with `input.value` (or `checkbox.checked` for a tick box).

💡 Why do we use it?

Live features like search-as-you-type, character counters, instant previews, and "agree" boxes all depend on these two events.

<> Syntax

Syntax pattern

```
textBox.addEventListener("input", function () {
  console.log(textBox.value);
});

select.addEventListener("change", function () {
  console.log(select.value);
});
```

✎ Simple example

A greeting and a character counter that update with every key you type:

HTML

```
<input id="nameBox" placeholder="Type your name" maxlength="20">
<p id="greeting">Hello, stranger!</p>
<p id="count">0 / 20 characters</p>
<script>
  const nameBox = document.getElementById("nameBox");

  nameBox.addEventListener("input", function () {
    const text = nameBox.value.trim();
    document.getElementById("greeting").textContent = "Hello, " + (text ||
"stranger") + "!";
    document.getElementById("count").textContent = nameBox.value.length + " / 20
characters";
  });
</script>
```

Here is `change` with a dropdown list and a check box:

HTML

```
<label>Favourite colour:
  <select id="colorPick">
    <option value="">Choose...</option>
    <option value="tomato">Tomato</option>
    <option value="seagreen">Sea green</option>
    <option value="royalblue">Royal blue</option>
  </select>
</label>
<div id="swatch" style="height:44px;margin:10px 0;border:1px solid #999"></div>
<label><input type="checkbox" id="agree"> I agree to the rules</label>
<span id="agreeText">(not agreed)</span>
<script>
  const colorPick = document.getElementById("colorPick");
  const swatch = document.getElementById("swatch");

  colorPick.addEventListener("change", function () {
    swatch.style.backgroundColor = colorPick.value;
  });

  const agree = document.getElementById("agree");

  agree.addEventListener("change", function () {
    document.getElementById("agreeText").textContent = agree.checked ? "
(agreed)" : "(not agreed)";
  });
</script>
```

🌐 Real-world example

Search as you type. Every letter you type filters the list. Each item is hidden if it does not contain the search word. The `hidden` property hides an element completely:

HTML

```
<input id="search" placeholder="Search fruits...">
<ul id="fruits">
  <li>Mango</li>
  <li>Banana</li>
  <li>Apple</li>
  <li>Orange</li>
  <li>Grapes</li>
</ul>
<script>
  const search = document.getElementById("search");
  const items = document.querySelectorAll("#fruits li");

  search.addEventListener("input", function () {
    const word = search.value.trim().toLowerCase();

    items.forEach(function (li) {
      li.hidden = !li.textContent.toLowerCase().includes(word);
    });
  });
</script>
```

This uses string methods from Chapter 10: `trim`, `toLowerCase`, and `includes`.

⚠ Common mistakes

- ✗ Reading `input.value` **once**, outside the handler. It gives the value from the moment the page loaded. Read it **inside** the handler.
- ✗ Using `change` when you need instant updates. On a text box, `change` waits until the user leaves the box. Use `input`.
- ✗ Forgetting that `value` is always text. Convert numbers with `Number(...)`.
- ✗ Forgetting `trim()` and `toLowerCase()` when searching. `"Apple"` and `"apple"` are not equal.
- ✗ Using `keyup` to track typing. It misses pasting with the mouse. Use `input`.

🔑 Key points

- `input` fires on every change. `change` fires when a choice is finished.
- Read the current value inside the handler with `.value` (or `.checked` for check boxes).
- `value` is always text.
- Clean the text with `trim()` and `toLowerCase()` before comparing.

🔧 Practice

- 1 Show what the user types in a text box, live, inside a `<p>`.
- 2 Make a select box of three cities that shows the chosen city in a `<p>`.
- 3 Make a check box that shows or hides a paragraph.

13.4 Forms and the submit Event

🎯 What is it?

A `<form>` fires a `submit` event when the user presses its submit button **or** presses Enter inside a field. Normally the browser then sends the form away and **reloads the page**. To handle the form yourself, call `event.preventDefault()` at the start of the handler.

💡 Why do we use it?

Forms are the main way that users give information to a website. Listening for `submit` lets you check the values, show messages, and add the new data to the page, all without a reload.

<> Syntax

Syntax pattern

```
form.addEventListener("submit", function (event) {  
  event.preventDefault();      // stop the page from reloading  
  const value = input.value.trim();  
  // check it, use it, show a message  
});
```

Simple example

A signup form with two checks:

HTML

```
<form id="signup">
  <p><label>Username <input id="user"></label></p>
  <p><label>Age <input id="age" type="number"></label></p>
  <button>Join</button>
</form>
<p id="result"></p>
<script>
  const signup = document.getElementById("signup");

  signup.addEventListener("submit", function (event) {
    event.preventDefault();

    const user = document.getElementById("user").value.trim();
    const age = Number(document.getElementById("age").value);
    const result = document.getElementById("result");

    if (user.length < 3) {
      result.textContent = "Username needs at least 3 letters.";
      return;
    }
    if (!Number.isInteger(age) || age < 13) {
      result.textContent = "You must be 13 or older.";
      return;
    }
    result.textContent = "Welcome, " + user + "!";
  });
</script>
```

Try it: press Enter inside a box, or click **Join**. Both fire the same `submit` event.

🌐 Real-world example

A guest book. Each message is added to the top of a list. The text goes in with `textContent`, so anything a visitor types is shown safely. After adding, `form.reset()` clears the boxes:

HTML

```
<form id="guest">
  <p><input id="who" placeholder="Your name"></p>
  <p><input id="says" placeholder="Your message" size="28">
</button>Sign</button></p>
</form>
<ul id="entries"></ul>
<script>
  const guestForm = document.getElementById("guest");
  const entries = document.getElementById("entries");

  guestForm.addEventListener("submit", function (event) {
    event.preventDefault();

    const who = document.getElementById("who").value.trim();
    const says = document.getElementById("says").value.trim();
    if (who === "" || says === "") {
      return;
    }

    const li = document.createElement("li");
    const strong = document.createElement("strong");
    strong.textContent = who + ": ";
    li.append(strong, says);
    entries.prepend(li);

    guestForm.reset();
  });
</script>
```

⚠ Common mistakes

- ✗ Forgetting `event.preventDefault()`. The browser reloads the page, and all your work disappears.
- ✗ Listening for `click` on the submit button instead of `submit` on the form. Then pressing **Enter** does not work.
- ✗ Not checking the input before using it. Empty boxes, spaces only, and text where you expected numbers are all normal.

✗ Building the new content with `innerHTML` and the visitor's text. Use `createElement` and `textContent`.

🔑 Key points

- A form fires `submit` on the button and on the Enter key.
- Call `event.preventDefault()` first to stop the reload.
- Listen for `submit` on the **form**, not for `click` on the button.
- Always check and clean input (`trim`, `Number`, length checks) before using it.
- `form.reset()` clears the fields.

🔧 Practice

- 1 Make a form with one number box. When submitted, show the number doubled in a paragraph.
- 2 Add a check: show `"Please type a number."` when the box is empty or not a number.
- 3 Add a "message board" like the guest book, but with a single text box.

13.5 Keyboard Events

🎯 What is it?

Two events report key presses:

- `keydown` fires when a key goes **down** (and repeats while it is held).
- `keyup` fires when the key is **released**.

Inside the handler, `event.key` tells you which key: `"a"`, `"A"`, `"Enter"`, `"Escape"`, `"ArrowLeft"`, `" "` (the space bar), and so on. You can listen on a

single element, or on the whole `document` .

💡 Why do we use it?

Games use the arrow keys. Apps use shortcuts such as Escape to close a window, or Enter to send a message.

<> Syntax

Syntax pattern

```
document.addEventListener("keydown", function (event) {  
  if (event.key === "Enter") {  
    // Enter was pressed  
  }  
});
```

✍ Simple example

Use the arrow keys to move the blue square. Press Enter to reset it. **Click inside the box first**, so the keyboard input goes to this example. `Math.max` and `Math.min` (Chapter 11) keep the square inside its stage:

HTML

```
<p>Click here first, then press the arrow keys. Enter resets.</p>
<div id="stage" style="position:relative;width:300px;height:120px;border:2px
solid #0F1B3D;background:#EDF1F8">
  <div id="player"
style="position:absolute;left:10px;top:10px;width:30px;height:30px;background:#2
444D6;border-radius:6px"></div>
</div>
<p id="lastKey">Last key: none</p>
<script>
  const player = document.getElementById("player");
  let x = 10;
  let y = 10;

  document.addEventListener("keydown", function (event) {
    document.getElementById("lastKey").textContent =
      "Last key: " + (event.key === " " ? "Space" : event.key);

    if (event.key.startsWith("Arrow")) {
      event.preventDefault();    // stop the page from scrolling
    }
    if (event.key === "ArrowRight") { x += 10; }
    if (event.key === "ArrowLeft") { x -= 10; }
    if (event.key === "ArrowDown") { y += 10; }
    if (event.key === "ArrowUp") { y -= 10; }
    if (event.key === "Enter") { x = 10; y = 10; }

    x = Math.max(0, Math.min(x, 266));
    y = Math.max(0, Math.min(y, 86));
    player.style.left = x + "px";
    player.style.top = y + "px";
  });
</script>
```

🌐 Real-world example

Press Enter to add. A quick-add box for tasks, without a form:

HTML

```
<input id="quick" placeholder="Type a task, press Enter" size="28">
<ul id="quickList"></ul>
<script>
  const quick = document.getElementById("quick");
  const quickList = document.getElementById("quickList");

  quick.addEventListener("keydown", function (event) {
    if (event.key === "Enter" && quick.value.trim() !== "") {
      const li = document.createElement("li");
      li.textContent = quick.value.trim();
      quickList.append(li);
      quick.value = "";
    }
  });
</script>
```

⚠ Common mistakes

- ✗ Comparing with the wrong text. `event.key` is case-sensitive: `"Enter"` works, `"enter"` does not, and `"a"` and `"A"` are different.
- ✗ Using `event.keyCode`. It is old and deprecated. Use `event.key`.
- ✗ Expecting `keydown` to give the finished text. To read what was typed in a box, use the `input` event.
- ✗ Forgetting that a held key keeps firing `keydown`. You can check `event.repeat`.
- ✗ Forgetting that the page must have keyboard focus. Click on it first.

🔑 Key points

- `keydown` fires when a key goes down, and `keyup` when it is released.
- `event.key` says which key: `"Enter"`, `"Escape"`, `"ArrowLeft"`, `"a"`, and so on.
- Listen on `document` for keys anywhere on the page, or on one input for keys inside it.

- Use `event.preventDefault()` to stop default actions such as scrolling.

☰ Practice

- 1 Print every key the user presses with `console.log(event.key)`.
- 2 Show a message "Escape pressed!" only when the Escape key is pressed.
- 3 Make a text box that turns red when the user presses Enter while it is empty.

13.6 Mouse Events

🕒 What is it?

Besides `click`, elements can react to many mouse events:

Event	When it fires
<code>dblclick</code>	the element is double-clicked
<code>mouseenter</code>	the pointer moves onto the element
<code>mouseleave</code>	the pointer moves off the element
<code>mousemove</code>	the pointer moves while it is over the element
<code>mousedown</code> , <code>mouseup</code>	a mouse button goes down, or comes back up
<code>contextmenu</code>	the right mouse button is clicked

💡 Why do we use it?

Hover effects, tooltips, drag and drop, and drawing tools all use mouse events.

<> Syntax

Syntax pattern

```
element.addEventListener("mouseenter", function () { /* ... */ });
element.addEventListener("mouseleave", function () { /* ... */ });
element.addEventListener("mousemove", function (event) { /* event.offsetX ... */
});
```

Simple example

Move the mouse onto the card and off again, and double-click it:

HTML

```
<style>
  #card { width: 230px; padding: 16px; border: 2px solid #0F1B3D; background:
#fff; transition: 0.15s; }
  #card.hover { background: #FFE14D; transform: translateY(-3px); }
</style>
<div id="card">Hover over me. Double-click me!</div>
<p id="mouseInfo">Move the mouse onto the card.</p>
<script>
  const card = document.getElementById("card");
  const info = document.getElementById("mouseInfo");

  card.addEventListener("mouseenter", function () {
    card.classList.add("hover");
    info.textContent = "The mouse entered.";
  });

  card.addEventListener("mouseleave", function () {
    card.classList.remove("hover");
    info.textContent = "The mouse left.";
  });

  card.addEventListener("dblclick", function () {
    card.textContent = "You double-clicked!";
  });
</script>
```

🌐 Real-world example

A live coordinates display. The numbers change every time the mouse moves inside the green box:

HTML

```
<div id="track" style="height:90px;padding:8px;background:#DDF3E4;border:2px solid #1A7F37;cursor:crosshair">
  Move your mouse over this box
</div>
<p id="coords">x: -, y: -</p>
<script>
  const track = document.getElementById("track");

  track.addEventListener("mousemove", function (event) {
    document.getElementById("coords").textContent =
      "x: " + event.offsetX + ", y: " + event.offsetY;
  });

  track.addEventListener("mouseleave", function () {
    document.getElementById("coords").textContent = "x: -, y: -";
  });
</script>
```

⚠ Common mistakes

- ✗ Using `mouseover` and `mouseout` for hover effects on elements with children. Those events also fire when the pointer moves between the element and its children, which causes flicker. `mouseenter` and `mouseleave` do not.
- ✗ Doing heavy work in `mousemove`. It fires many times per second, so keep the handler small.
- ✗ Forgetting that phones and tablets have no hover. Important actions should also work with `click`.
- ✗ Forgetting that `offsetX` is relative to the **element**, and `clientX` is relative to the **window**.

🔑 Key points

- `mouseenter` and `mouseleave` are the best pair for hover effects.
- `mousemove` reports the pointer position through the event object.
- `dblclick` handles double-clicks.
- Keep `mousemove` handlers short, and do not rely on hover alone.

🔧 Practice

- 1 Make a box that turns green when the mouse enters and back to white when it leaves.
- 2 Show "Ouch!" when a paragraph is double-clicked.
- 3 Print the mouse position (`clientX` and `clientY`) when a button is right-clicked. (Hint: use `contextmenu` , and call `preventDefault()` to skip the browser menu.)

13.7 Events on Many Elements: Delegation

🎯 What is it?

When you click a child element, the event does not stop there. It **bubbles up**: the parent hears it, then the parent's parent, all the way to the top of the page. This lets you put **one** listener on a parent, and use `event.target` to find out which child was clicked. This technique is called **event delegation**.

Two helpful tools:

- `element.closest("selector")` finds the nearest parent (or the element itself) that matches the selector.
- `element.matches("selector")` tells you `true` or `false` whether the element matches.

💡 Why do we use it?

Real lists change: items are added and removed. If you attach a listener to every item, new items have none, and you need lots of listeners. A single listener on the parent handles all the current and future children.

<> Syntax

Syntax pattern

```
list.addEventListener("click", function (event) {
  const li = event.target.closest("li");
  if (!li) {
    return;           // the click was not on an item
  }
  // do something with li
});
```

✍ Simple example

Click an item to cross it out. Press **Add item** and try the new item too. It works with no extra code:

HTML

```
<style>
  li { cursor: pointer; padding: 2px 0; }
  li.done { text-decoration: line-through; color: #777; }
</style>
<button id="addBtn">Add item</button>
<ul id="items">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
<script>
  const items = document.getElementById("items");
  let n = 2;

  items.addEventListener("click", function (event) {
    const li = event.target.closest("li");
    if (li) {
      li.classList.toggle("done");
    }
  });

  document.getElementById("addBtn").addEventListener("click", function () {
    n++;
    const li = document.createElement("li");
    li.textContent = "Item " + n;
    items.append(li);
  });
</script>
```

🌐 Real-world example

Many buttons, one listener. A keypad with ten buttons needs only one listener. Each button carries its digit in a `data-` attribute:

HTML

```
<div id="pad" style="display:grid;grid-template-columns:repeat(5,44px);gap:6px">
  <button data-digit="1">1</button><button data-digit="2">2</button>
  <button data-digit="3">3</button><button data-digit="4">4</button>
  <button data-digit="5">5</button><button data-digit="6">6</button>
  <button data-digit="7">7</button><button data-digit="8">8</button>
  <button data-digit="9">9</button><button data-digit="0">0</button>
</div>
<p>Typed: <b id="typedDigits">-</b></p>
<script>
  const keypad = document.getElementById("pad");
  let typed = "";

  keypad.addEventListener("click", function (event) {
    if (event.target.dataset.digit === undefined) {
      return;
    }
    typed += event.target.dataset.digit;
    document.getElementById("typedDigits").textContent = typed;
  });
</script>
```

⚠ Common mistakes

- ✗ Adding a listener to each new element by hand, or forgetting to add it to new ones. Use delegation for lists that change.
- ✗ Using `event.target` directly when the item contains other tags. Clicking a `<b>` inside an `<li>` makes `target` the `<b>`. Use `event.target.closest("li")`.
- ✗ Forgetting the check for clicks on empty space. When the click is on the list itself (not on an item), `closest` gives `null`, so test for it.
- ✗ Expecting every event to bubble. `focus`, `blur`, and `mouseenter` do not. (Use `focusin`, `focusout`, or `mouseover` when you need delegation.)

🔑 Key points

- Events bubble from the clicked element up through its parents.

- One listener on a parent can serve many children, even ones added later.
- `event.target.closest("selector")` finds the item that was clicked.
- Always check for `null` in case the click missed every item.

☞ Practice

- 1 Put one listener on a `<ul>` that shows the text of whichever `<li>` is clicked.
- 2 Add a "Delete" button inside each `<li>`, and make it remove that `<li>` using delegation and `matches`.
- 3 Explain in one sentence why delegation works for elements created after the listener was added.

🚩 Mini project: Tip Calculator

A working app that uses **input** and **click** events, `data-` attributes, `Number`, `Math`, and `toFixed`. Change the bill, pick a tip button, or change the number of people, and the results update at once.

HTML

```
<style>
  .tip { width: 300px; border: 2px solid #0F1B3D; border-radius: 6px;
padding: 12px 16px; }
  .tip label { display: block; margin-bottom: 8px; }
  .tip input { width: 100px; padding: 4px; }
  .pct button { padding: 4px 10px; border: 2px solid #0F1B3D; background:
#fff; border-radius: 4px; }
  .pct button.on { background: #FFE14D; }
  .out p { margin: 4px 0; }
</style>
<div class="tip">
  <label>Bill amount <input id="bill" type="number" min="0" value="1200">
</label>
  <div class="pct">Tip:
    <button data-tip="5">5%</button>
    <button data-tip="10" class="on">10%</button>
    <button data-tip="15">15%</button>
    <button data-tip="20">20%</button>
  </div>
  <label style="margin-top: 10px">People <input id="people" type="number"
min="1" value="2"></label>
  <div class="out">
    <p>Tip: <b id="tip0Out"></b></p>
    <p>Total: <b id="total0Out"></b></p>
    <p>Each pays: <b id="each0Out"></b></p>
  </div>
</div>
<script>
  const bill = document.getElementById("bill");
  const people = document.getElementById("people");
  const buttons = document.querySelectorAll(".pct button");
  let tipPercent = 10;

  function calculate() {
    const amount = Number(bill.value) || 0; // NaN
    becomes 0
    const count = Math.max(1, Math.floor(Number(people.value)) || 1);
    const tip = amount * tipPercent / 100;
    const total = amount + tip;

    document.getElementById("tip0Out").textContent = tip.toFixed(2);
    document.getElementById("total0Out").textContent = total.toFixed(2);
```

```

        document.getElementById("eachOut").textContent = (total /
count).toFixed(2);
    }

    buttons.forEach(function (button) {
        button.addEventListener("click", function () {
            tipPercent = Number(button.dataset.tip);
            buttons.forEach(function (b) {
                b.classList.toggle("on", b === button);
            });
            calculate();
        });
    });

    bill.addEventListener("input", calculate);
    people.addEventListener("input", calculate);
    calculate();
</script>

```

Challenge: add a "Custom tip" box. Then round each person's share **up** with `Math.ceil`, and show a small message when the bill is empty.

🔖 Chapter 13 in one minute

- An event is something that happens.
`element.addEventListener("click", callback)` runs your function when it does.
- Pass the function (`handler`), do not call it (`handler()`).
- The callback receives an event object: `event.target` , `event.key` , `event.offsetX` , `event.preventDefault()` .
- `input` fires on every change. `change` fires when a choice is finished.
- Listen for `submit` on the form, and call `event.preventDefault()` to stop the reload.
- `keydown` and `event.key` handle the keyboard. `mouseenter` , `mouseleave` , and `mousemove` handle the mouse.
- Events bubble. One listener on a parent plus `event.target.closest(...)` handles many children.
- Keep your data in variables, and redraw the screen from that data.

▼ Sample answers for Chapter 13

13.1, exercise 2 (hide and show):

HTML

```
<style> .hidden { display: none; } </style>
<button id="toggle">Show / hide</button>
<p id="secret">The secret message!</p>
<script>
  document.getElementById("toggle").addEventListener("click", function () {
    document.getElementById("secret").classList.toggle("hidden");
  });
</script>
```

13.3, exercise 1 (live mirror):

HTML

```
<input id="typed" placeholder="Type here">
<p>You typed: <span id="echo"></span></p>
<script>
  const typed = document.getElementById("typed");

  typed.addEventListener("input", function () {
    document.getElementById("echo").textContent = typed.value;
  });
</script>
```

13.4, exercise 1 (double the number):

HTML

```
<form id="doubler">
  <input id="num" type="number" placeholder="A number">
  <button>Double it</button>
</form>
<p id="answer"></p>
<script>
  document.getElementById("doubler").addEventListener("submit", function
(event) {
    event.preventDefault();
    const n = Number(document.getElementById("num").value);
    document.getElementById("answer").textContent = n * 2;
  });
</script>
```

3

Project 3

To-Do List App

This is the biggest project of the book so far, and a real application. You will build a to-do list that lets you add tasks, tick them off, delete them, filter them, and see how many are left. It uses everything from Part 3: selecting elements, creating elements, classes, events, forms, and event delegation. It also uses the arrays and objects of Part 2 to keep the **data**.

What you will learn

- Keep an app's data in an array of objects
- Write a `render()` function that redraws the screen from the data
- Handle a form with `submit`
- Use event delegation for buttons that are created later
- Combine classes, `data-` attributes, and events in one working app

Step 1: Plan the app

Write down what the app must do:

1. Show a list of tasks. Each task has a **text**, and can be **done** or not.
2. A form adds a new task (and ignores empty ones).
3. Clicking a task's text ticks it off, or un-ticks it. A small **x** button deletes it.
4. A counter shows how many tasks are **left** (not done).
5. Three filter buttons show **All**, **Active**, or **Done** tasks.

💡 The most important idea: data first, screen second

Do not change the page directly when something happens. Instead, **change the data** (the `tasks` array), and then call one function, `render()`, that redraws the whole list from the data. The screen is always a picture of the data, so the two can never disagree. Most modern web apps are built on this idea.

Step 2: The HTML

The page needs a heading, a form, an empty list to fill, a message for an empty list, and a bar with the counter and the filter buttons. Notice that the list is empty, because JavaScript will build the items.

HTML

```
<div class="todo">
  <h2>My tasks</h2>
  <form id="taskForm">
    <input id="taskInput" placeholder="What needs doing?" autocomplete="off">
    <button>Add</button>
  </form>
  <ul id="taskList"></ul>
  <p id="emptyMsg" class="empty">Nothing to show here.</p>
  <div class="bar">
    <span id="counter"></span>
    <span class="filters">
      <button data-filter="all" class="on">All</button>
      <button data-filter="active">Active</button>
      <button data-filter="done">Done</button>
    </span>
  </div>
</div>
```

Step 3: The data and render()

The tasks live in an array of objects. Each object has an `id` (so we can tell tasks apart), the `text`, and a `done` flag. The variable `filter` remembers which filter is active.

JavaScript

```
let tasks = [
  { id: 1, text: "Read Chapter 12", done: true },
  { id: 2, text: "Practice events", done: false },
  { id: 3, text: "Build the to-do app", done: false }
];
let nextId = 4;
let filter = "all";

const form = document.getElementById("taskForm");
const input = document.getElementById("taskInput");
const list = document.getElementById("taskList");
const counter = document.getElementById("counter");
const emptyMsg = document.getElementById("emptyMsg");
```

`render()` is the heart of the app. It empties the list, builds one `<li>` for every visible task, and updates the counter, the empty message, and the filter buttons. The `data-id` attribute stores each task's id on its `<li>`, so a click can later tell which task it belongs to.

JavaScript

```
function visibleTasks() {
  if (filter === "active") {
    return tasks.filter(t => !t.done);
  }
  if (filter === "done") {
    return tasks.filter(t => t.done);
  }
  return tasks;
}

function render() {
  const visible = visibleTasks();
  list.innerHTML = "";

  visible.forEach(function (task) {
    const li = document.createElement("li");
    li.dataset.id = task.id;
    li.classList.toggle("done", task.done);

    const text = document.createElement("span");
    text.className = "text";
    text.textContent = task.text;

    const del = document.createElement("button");
    del.className = "delete";
    del.textContent = "x";
    del.setAttribute("aria-label", "Delete task");

    li.append(text, del);
    list.append(li);
  });

  const left = tasks.filter(t => !t.done).length;
  counter.textContent = left + (left === 1 ? " task left" : " tasks left");
  emptyMsg.hidden = visible.length > 0;

  document.querySelectorAll(".filters button").forEach(function (b) {
    b.classList.toggle("on", b.dataset.filter === filter);
  });
}
```

Step 4: Adding tasks

The form's `submit` event stops the page reload, cleans the text, ignores empty text, adds a new object to the array, clears the box, and redraws.

JavaScript

```
form.addEventListener("submit", function (event) {
  event.preventDefault();

  const text = input.value.trim();
  if (text === "") {
    return;
  }

  tasks.push({ id: nextId, text: text, done: false });
  nextId++;
  input.value = "";
  render();
});
```

Step 5: Completing and deleting tasks

There is **one** listener on the `<ul>`. It finds which `<li>` was clicked, reads its id from `data-id`, and then either deletes that task (if the click was on the `×`) or flips its `done` flag. Because we use delegation, new tasks work automatically.

JavaScript

```
list.addEventListener("click", function (event) {
  const li = event.target.closest("li");
  if (!li) {
    return;
  }
  const id = Number(li.dataset.id);

  if (event.target.matches(".delete")) {
    tasks = tasks.filter(t => t.id !== id);
  } else {
    const task = tasks.find(t => t.id === id);
    task.done = !task.done;
  }
  render();
});
```

Step 6: The filters and the first draw

The filter bar has one listener too. It reads the `data-filter` value of the clicked button, saves it in `filter`, and redraws. The last line, `render()`, draws the list for the first time.

JavaScript

```
document.querySelector(".filters").addEventListener("click", function (event) {
  if (event.target.dataset.filter) {
    filter = event.target.dataset.filter;
    render();
  }
});

render();
```

The complete app

Here is everything together. Add a task, tick one off, delete one, and try the filters. Press **Reset** to start again:

HTML

```
<style>
  .todo { max-width: 420px; }
  .todo h2 { margin: 0 0 10px; }
  #taskForm { display: flex; gap: 6px; margin-bottom: 10px; }
  #taskInput { flex: 1; padding: 6px 8px; }
  #taskList { list-style: none; padding: 0; margin: 0; }
  #taskList li { display: flex; align-items: center; gap: 8px; padding: 6px 4px;
border-bottom: 1px solid #DDE3F0; }
  #taskList .text { flex: 1; cursor: pointer; }
  #taskList li.done .text { text-decoration: line-through; color: #777; }
  #taskList .delete { border: 0; background: none; color: #C62828; font-size:
20px; line-height: 1; }
  .empty { color: #777; padding: 6px 4px; margin: 0; }
  .bar { display: flex; justify-content: space-between; align-items: center;
margin-top: 10px; font-size: 14px; }
  .filters button { border: 1px solid #0F1B3D; background: #fff; border-radius:
4px; padding: 2px 8px; }
  .filters button.on { background: #FFE14D; }
</style>
<div class="todo">
  <h2>My tasks</h2>
  <form id="taskForm">
    <input id="taskInput" placeholder="What needs doing?" autocomplete="off">
    <button>Add</button>
  </form>
  <ul id="taskList"></ul>
  <p id="emptyMsg" class="empty">Nothing to show here.</p>
  <div class="bar">
    <span id="counter"></span>
    <span class="filters">
      <button data-filter="all" class="on">All</button>
      <button data-filter="active">Active</button>
      <button data-filter="done">Done</button>
    </span>
  </div>
</div>
<script>
  let tasks = [
    { id: 1, text: "Read Chapter 12", done: true },
    { id: 2, text: "Practice events", done: false },
    { id: 3, text: "Build the to-do app", done: false }
  ];
```

```

let nextId = 4;
let filter = "all";

const form = document.getElementById("taskForm");
const input = document.getElementById("taskInput");
const list = document.getElementById("taskList");
const counter = document.getElementById("counter");
const emptyMsg = document.getElementById("emptyMsg");

function visibleTasks() {
  if (filter === "active") {
    return tasks.filter(t => !t.done);
  }
  if (filter === "done") {
    return tasks.filter(t => t.done);
  }
  return tasks;
}

function render() {
  const visible = visibleTasks();
  list.innerHTML = "";

  visible.forEach(function (task) {
    const li = document.createElement("li");
    li.dataset.id = task.id;
    li.classList.toggle("done", task.done);

    const text = document.createElement("span");
    text.className = "text";
    text.textContent = task.text;

    const del = document.createElement("button");
    del.className = "delete";
    del.textContent = "×";
    del.setAttribute("aria-label", "Delete task");

    li.append(text, del);
    list.append(li);
  });

  const left = tasks.filter(t => !t.done).length;
  counter.textContent = left + (left === 1 ? " task left" : " tasks left");
  emptyMsg.hidden = visible.length > 0;
}

```

```

        document.querySelectorAll(".filters button").forEach(function (b) {
            b.classList.toggle("on", b.dataset.filter === filter);
        });
    }

    form.addEventListener("submit", function (event) {
        event.preventDefault();

        const text = input.value.trim();
        if (text === "") {
            return;
        }

        tasks.push({ id: nextId, text: text, done: false });
        nextId++;
        input.value = "";
        render();
    });

    list.addEventListener("click", function (event) {
        const li = event.target.closest("li");
        if (!li) {
            return;
        }
        const id = Number(li.dataset.id);

        if (event.target.matches(".delete")) {
            tasks = tasks.filter(t => t.id !== id);
        } else {
            const task = tasks.find(t => t.id === id);
            task.done = !task.done;
        }
        render();
    });

    document.querySelector(".filters").addEventListener("click", function (event) {
    {
        if (event.target.dataset.filter) {
            filter = event.target.dataset.filter;
            render();
        }
    }
    });

```



```
render();  
</script>
```

Your turn: challenges

1. **Clear completed.** Add a button that removes all the done tasks in one click. (Hint: `tasks = tasks.filter(t => !t.done)` .)
2. **Edit a task.** Double-click a task's text to change it. (Hint: `dblclick` , and `prompt` or a small input.)
3. **Save your tasks.** Make the tasks survive a page reload with `localStorage` (see the note below).
4. **Task count per filter.** Show the number of tasks on each filter button, such as `Active (2)` .
5. **Priority.** Add a `high` property to tasks, and show high-priority tasks with a coloured border.

About saving with localStorage

Browsers have a small storage called `localStorage` that keeps text even after the page is closed. It works in your own `.html` files, but it does **not** work inside the live boxes of this book, so try it in your own file. Objects must be turned into text first with `JSON.stringify`, and back again with `JSON.parse` :

JavaScript

```
function save() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
}

function load() {
  const saved = localStorage.getItem("tasks");
  return saved ? JSON.parse(saved) : [];
}

// call save() at the end of render(), and start with: let tasks = load();
```

What you practised

- Keeping the **data** in one place and redrawing the screen from it.
- Building elements with `createElement`, `append`, and `textContent`.
- `classList.toggle(name, condition)` and `data-` attributes.
- A form with `submit` and `preventDefault`.
- Event delegation with `closest` and `matches`.
- Filtering arrays of objects with `filter`, and finding with `find`.